

PART 1 — VISION, STRATEGY, AND FOUNDATIONAL PRINCIPLES

Document: Investment Time Machine Founder Specification

Part: 1 of 27

Status: Approved

Version: 1.0

Author: Saman Sepehr

Last Updated: June 2026

1. Executive Summary

Investment Time Machine is a historical investment intelligence platform designed to help users understand financial decisions through simulation, analysis, and education.

Unlike brokerages, trading platforms, and stock recommendation tools, Investment Time Machine focuses exclusively on understanding historical outcomes rather than predicting future performance.

The platform allows users to simulate investment decisions across time using real market data and receive quantitative analysis, portfolio insights, risk assessments, and AI-generated educational explanations.

The long-term objective is to build the most comprehensive educational platform for historical investment analysis while maintaining a strong emphasis on transparency, reproducibility, and engineering quality.

2. Mission Statement

Modern investment education is heavily influenced by speculation, social media content, news cycles, and hindsight bias.

Many individuals encounter investing through:

- Social media influencers

- Online communities
- Financial news
- Content creators
- Short-form video platforms

These sources frequently emphasize predictions and emotional narratives while providing limited context regarding historical outcomes, risk exposure, and long-term performance.

Investment Time Machine exists to address this gap.

The mission of the platform is to provide an evidence-based environment where users can explore historical investment decisions and understand the consequences of those decisions through accurate simulations and educational analysis.

Every calculation, chart, report, and AI-generated explanation must be grounded in verifiable historical market data.

3. Product Vision

The long-term vision is to create the world's most comprehensive historical investment exploration platform.

Users should be able to investigate historical investment decisions with the same ease that users explore locations on Google Maps.

The platform should allow users to answer questions such as:

- What if I invested \$1,000 into Bitcoin in 2011?
- What if I invested every month for the last ten years?
- Would Nvidia have outperformed Tesla?
- How would a diversified portfolio have changed my outcome?
- How much risk would I have experienced during market crashes?
- What would my investment be worth after adjusting for inflation?

Investment Time Machine is designed to become the definitive platform for answering historical investment questions.

4. Problem Statement

Most investment platforms are optimized for trading activity rather than financial understanding.

Current solutions generally fall into one of four categories:

Brokerage Platforms

Examples:

- Robinhood
- Fidelity
- Charles Schwab

Primary Goal:

- Execute trades

Limitations:

- Limited educational simulation capabilities
- Minimal historical decision analysis

Market Data Platforms

Examples:

- Yahoo Finance
- TradingView

Primary Goal:

- Provide charts and market information

Limitations:

- Require financial literacy
- Limited contextual education

Research Platforms

Examples:

- Seeking Alpha
- Morningstar

Primary Goal:

- Provide research content

Limitations:

- Often overwhelming for beginners
- Limited interactive simulation tools

Portfolio Backtesting Platforms

Examples:

- Portfolio Visualizer

Primary Goal:

- Historical portfolio analysis

Limitations:

- Outdated user experience
- Limited educational guidance
- No integrated AI explanation system

The market lacks a platform that combines historical simulation, educational analysis, portfolio research, and AI-assisted explanation into a single experience.

5. Product Philosophy

Investment Time Machine is built around four foundational beliefs.

Belief 1: Understanding Is More Valuable Than Prediction

The platform prioritizes understanding historical outcomes rather than forecasting future results.

Belief 2: Transparency Creates Trust

Users should understand how calculations are performed and where data originates.

Belief 3: Education Should Be Interactive

Users learn more effectively by exploring scenarios than by reading static content.

Belief 4: Evidence Matters

Every statement generated by the system should be traceable to verifiable data.

6. Product Principles

Every future feature must satisfy at least one of the following principles.

Principle 1 — Increase Understanding

The feature helps users understand investments, markets, or financial outcomes.

Principle 2 — Increase Transparency

The feature makes calculations, assumptions, or data sources easier to understand.

Principle 3 — Increase Accuracy

The feature improves correctness, reliability, or data quality.

Principle 4 — Increase Educational Value

The feature teaches a concept that users can apply elsewhere.

Features that do not satisfy these principles should not be developed.

7. Product Scope

The scope of Version 1 is intentionally limited.

The platform will provide:

- Historical asset simulations
- Portfolio simulations
- Asset comparisons

- Risk analysis
- Educational AI reports
- Historical market exploration

The platform will not provide:

- Real-money trading
 - Brokerage integrations
 - Investment recommendations
 - Financial planning services
 - Tax advice
 - Personalized investment guidance
-

8. Target Users

Primary User Group

College Students

Characteristics:

- Learning personal finance
- Interested in investing
- Limited investing experience
- Interested in understanding compounding and opportunity cost

Representative Questions:

- What if I invested my internship income?
 - What if I started investing at age 18?
-

Secondary User Group

Retail Investors

Characteristics:

- Interested in historical performance

- Interested in portfolio analysis
- Interested in risk-adjusted returns

Representative Questions:

- Was this investment worth the risk?
 - Would diversification have improved my outcome?
-

Tertiary User Group

Researchers and Enthusiasts

Characteristics:

- Interested in historical market behavior
- Interested in investment comparisons
- Interested in portfolio construction

Representative Questions:

- How did technology stocks perform during recessions?
 - Which assets provided the strongest risk-adjusted returns?
-

9. Competitive Analysis

Yahoo Finance

Strengths:

- Large market coverage
- Historical data
- News aggregation

Weaknesses:

- Weak simulation experience
- Limited educational tooling

Lessons Adopted:

- Broad asset coverage

Lessons Avoided:

- Information overload
-

TradingView

Strengths:

- Excellent charting
- Strong visualization

Weaknesses:

- Steep learning curve
- Trader-focused experience

Lessons Adopted:

- Strong visual exploration

Lessons Avoided:

- Complexity for beginners
-

Portfolio Visualizer

Strengths:

- Excellent calculations
- Powerful backtesting

Weaknesses:

- Outdated user experience
- Limited accessibility

Lessons Adopted:

- Analytical depth

Lessons Avoided:

- Poor usability
-

10. Free-Tier Technology Strategy

The project should remain buildable with minimal cost during development.

Target Monthly Cost:

\$0–\$10

Preferred Technologies:

Backend:

- FastAPI
- PostgreSQL
- SQLAlchemy
- Alembic

Data Sources:

- yfinance
- CoinGecko
- FRED

Infrastructure:

- GitHub
- Docker
- Railway
- Render
- Vercel

The platform should not depend on paid infrastructure during the MVP stage.

11. Engineering Principles

The following principles govern all technical decisions.

Accuracy Over Aesthetics

Correctness is more important than visual presentation.

Backend Before Frontend

Core functionality must exist before interface development.

Local Data Ownership

Historical market data should be stored locally whenever practical.

Reproducibility

Every calculation should be independently verifiable.

Provider Independence

The system should support multiple data providers through abstraction layers.

Documentation First

Major decisions must be documented before implementation.

12. Project Commandments

1. Never allow AI to invent market data.
2. Never hide calculations from users.
3. Never sacrifice accuracy for convenience.
4. Prefer free infrastructure whenever possible.
5. Every feature must improve educational value.
6. Every calculation must be reproducible.
7. Build for users and recruiters simultaneously.
8. Treat documentation as part of the product.
9. Store historical data locally whenever feasible.

10. Optimize for long-term maintainability.

13. Success Metrics

Technical Success

The platform successfully:

- Stores historical market data
- Performs accurate simulations
- Generates reproducible analytics
- Maintains stable API performance
- Achieves strong automated test coverage

Product Success

Users can:

- Simulate historical investments
- Compare multiple assets
- Analyze portfolio performance
- Understand investment risk
- Receive useful educational insights

Recruiter Success

A recruiter reviewing the project should immediately recognize:

- Backend Engineering
 - Data Engineering
 - Database Design
 - System Design
 - Financial Analytics
 - AI Integration
 - Software Architecture
-

14. Long-Term Vision

Version 1

Historical Investment Simulator

Version 2

Advanced Portfolio Analysis

Version 3

AI Research Assistant

Version 4

Market Intelligence Platform

Version 5

Comprehensive Financial Education Ecosystem

The ultimate objective is to create the most trusted platform for exploring and understanding historical investment decisions.

End of Part 1

Investment Time Machine

Founder Specification

Part 2 — System Architecture

2.1 Architectural Philosophy

Investment Time Machine is designed around a single principle:

Historical investment analysis must be accurate, reproducible, and explainable.

The platform is not intended to be another financial dashboard or speculative AI chatbot. Its primary purpose is to provide users with historically accurate investment simulations and educational insights based on verified market data.

Every architectural decision must support this objective.

Principle 1 — Historical Truth Is Sacred

The platform shall never fabricate financial information.

All investment calculations must originate from verified historical datasets stored within the system.

This includes:

- Asset prices
- Dividends
- Stock splits
- Inflation data
- Economic indicators

The system must never estimate missing financial data without explicitly marking the result as incomplete.

Rationale

Trust is the core product feature.

Users must be able to reproduce results independently and verify every calculation performed by the platform.

Tradeoffs

Maintaining local historical datasets increases storage requirements and operational complexity.

This tradeoff is accepted because accuracy is more valuable than infrastructure simplicity.

Principle 2 — Local Data Ownership

Historical market data should be stored internally whenever practical.

The platform should not depend on external APIs during simulation execution.

External providers should only be used for:

- Initial data ingestion
- Scheduled updates
- Data verification

Rationale

External APIs introduce:

- Downtime risk
- Rate limits
- Vendor lock-in
- Data inconsistency

Local ownership improves reliability and reproducibility.

Tradeoffs

Additional database storage and maintenance requirements are introduced.

This tradeoff is accepted.

Principle 3 — AI Never Generates Financial Facts

Artificial intelligence is an educational layer, not a source of financial truth.

AI may:

- Explain results
- Summarize market conditions
- Provide educational context
- Compare investment outcomes

AI may not:

- Invent asset prices
- Invent returns
- Invent dividends
- Invent performance statistics
- Modify simulation outputs

Rationale

Large language models are probabilistic systems and cannot guarantee factual correctness.

Financial calculations must remain deterministic.

Tradeoffs

Additional validation and architecture separation are required.

This tradeoff is accepted.

Principle 4 — Simulation Logic Must Be Independent

The investment simulation engine must function entirely without AI.

A simulation should produce identical results regardless of whether AI services are available.

Rationale

Financial correctness must never depend on model behavior or third-party AI providers.

Tradeoffs

Requires additional service separation and engineering discipline.

This tradeoff is accepted.

2.2 High-Level System Architecture

The platform is composed of four primary services.

1. API Gateway
2. Historical Data Service
3. Simulation Engine
4. AI Intelligence Service

Each service has a single responsibility and communicates through well-defined interfaces.

The architecture prioritizes maintainability, auditability, and future scalability.

2.3 Core Services

API Gateway

The API Gateway serves as the primary entry point for all client requests.

Responsibilities

- Authentication
- Authorization
- Request validation
- Input sanitization
- Rate limiting
- API orchestration
- Response formatting

Technology

- FastAPI

Rationale

FastAPI provides:

- Excellent performance
 - Strong typing
 - Automatic API documentation
 - Simple integration with Python services
-

Historical Data Service

The Historical Data Service is responsible for all market and economic data retrieval.

Responsibilities

- Historical price retrieval
- Dividend retrieval
- Stock split retrieval
- Inflation data retrieval
- Economic data retrieval
- Cache management

Inputs

- Asset identifier
- Asset type
- Date range

Outputs

- Normalized historical dataset

Supported Assets

Phase 1

- US Stocks
- ETFs
- Cryptocurrencies

Phase 2

- Mutual Funds
- Commodities
- Bonds

Phase 3

- International Markets

Rationale

Separating data retrieval from calculations prevents business logic from becoming tightly coupled to data storage systems.

Simulation Engine

The Simulation Engine is the most critical component of the platform.

Its sole responsibility is producing accurate investment calculations.

Responsibilities

- Investment growth calculations
- Dividend reinvestment calculations
- CAGR calculations
- Inflation-adjusted returns
- Portfolio performance calculations

Requirements

Deterministic

Identical inputs must always produce identical outputs.

Auditable

Every simulation must store:

- Inputs
- Assumptions
- Data references
- Formulas used
- Outputs

Explainable

Users must be able to inspect:

- Calculation methods
- Data sources
- Assumptions

Future Expansion

The architecture must support:

- Dollar-cost averaging simulations

- Portfolio simulations
- Retirement planning
- Tax modeling
- Risk analysis

without requiring major redesign.

AI Intelligence Service

The AI Intelligence Service provides educational context for simulation results.

It is intentionally separated from financial calculations.

Responsibilities

- Educational explanations
- Historical context generation
- Market event summaries
- Investment concept explanations

Inputs

- Simulation outputs
- Historical events
- Economic indicators

Outputs

- Human-readable educational content

Restrictions

The AI service must never:

- Calculate returns
- Generate financial data
- Modify simulation results
- Access raw databases directly

Rationale

Separating educational functionality from financial calculations preserves accuracy while allowing future AI enhancements.

2.4 Internal Data Flow

The platform follows a predictable request lifecycle.

Example Request

User Input:

- Asset: Tesla (TSLA)
- Investment Amount: \$5,000
- Start Date: January 1, 2018

Processing Flow

1. Request arrives at API Gateway.
2. Gateway validates all inputs.
3. Historical Data Service retrieves required datasets.
4. Simulation Engine performs calculations.
5. Results are stored for auditing purposes.
6. AI Intelligence Service optionally generates educational insights.
7. Final response is returned to the user.

This flow ensures calculations remain isolated from presentation and AI systems.

2.5 Service Boundaries

Strict service boundaries are mandatory.

Historical Data Service Must Not

- Generate explanations
- Perform investment calculations

- Interact with language models
-

Simulation Engine Must Not

- Generate text
 - Call AI services
 - Modify historical datasets
-

AI Intelligence Service Must Not

- Calculate returns
 - Modify financial outputs
 - Access raw databases directly
 - Become a source of financial truth
-

API Gateway Must Not

- Perform investment calculations
 - Store business logic
 - Act as a reporting engine
-

Architectural Benefits

This separation provides:

- Easier testing
- Easier debugging
- Independent scaling
- Financial correctness
- Reduced AI risk
- Improved maintainability

Most importantly, it ensures that investment results remain independently verifiable even if every AI component is removed from the system.

Part 2 Completion Status

Completed Sections:

- 2.1 Architectural Philosophy
- 2.2 High-Level System Architecture
- 2.3 Core Services
- 2.4 Internal Data Flow
- 2.5 Service Boundaries

Status: Approved Draft

Next Section:

Part 2.6 — Database Architecture

Topics to Define:

- PostgreSQL schema design
- Asset catalog structure
- Historical pricing tables
- Dividend storage
- Split storage
- User tables
- Simulation tables
- Audit logging
- Indexing strategy
- Data retention strategy
- Future scalability considerations

End of part 2

Investment Time Machine

Founder Specification

Part 2.6 — Database Architecture

Status: Approved

2.6.1 Database Philosophy

The database is a core component of the Investment Time Machine platform and serves as the authoritative source of historical investment data.

The database architecture shall prioritize:

- Accuracy
- Reproducibility
- Auditability
- Data Integrity
- Scalability
- Long-Term Maintainability

Historical financial data is considered a product asset and must be treated accordingly.

2.6.2 Primary Database Decision

Decision

PostgreSQL shall serve as the primary production database.

Rationale

PostgreSQL provides:

- Strong relational integrity
- Mature indexing capabilities
- Excellent date and time support
- Reliable transaction handling
- Native JSON support
- Broad ecosystem compatibility

Approved Technology

- PostgreSQL
 - SQLAlchemy
 - Alembic
-

2.6.3 Database Domains

The platform database shall be organized into the following logical domains:

1. Asset Catalog
2. Historical Prices
3. Dividends
4. Stock Splits
5. Economic Indicators
6. Users
7. Simulations
8. Audit Logs
9. AI Explanations

Each domain shall remain logically separated while maintaining relational consistency.

2.6.4 Asset Catalog

The Asset Catalog shall act as the authoritative registry of all supported financial assets.

Supported Asset Types

- Stocks
- ETFs
- Cryptocurrencies
- Market Indexes
- Economic Indicators

Purpose

The Asset Catalog exists to:

- Eliminate duplicate asset definitions

- Normalize provider-specific identifiers
- Maintain consistent asset metadata
- Enable future multi-provider support

Example Attributes

- Symbol
 - Asset Name
 - Asset Type
 - Exchange
 - Currency
 - Country
 - Active Status
-

2.6.5 Historical Prices

Historical Prices shall store daily market pricing information.

Purpose

Support all simulation and performance calculations.

Stored Data

- Open Price
- High Price
- Low Price
- Close Price
- Adjusted Close Price
- Volume
- Data Source

Data Integrity Requirement

Only one historical price record may exist per:

- Asset
- Date
- Data Source

This constraint is mandatory.

2.6.6 Dividends

Dividend information shall be stored independently from price records.

Purpose

Support:

- Dividend analysis
- Dividend reinvestment calculations
- Total return calculations

Stored Data

- Ex-Dividend Date
 - Payment Date
 - Dividend Amount
 - Currency
 - Data Source
-

2.6.7 Stock Splits

Stock split events shall be maintained in a dedicated dataset.

Purpose

Track corporate actions affecting share quantities.

Examples

4-for-1 Split:

split_ratio = 4.0

1-for-10 Reverse Split:

split_ratio = 0.1

Approved MVP Strategy

The simulation engine shall primarily use adjusted close prices while retaining split data for auditability and future advanced simulations.

2.6.8 Economic Indicators

Macroeconomic datasets shall support educational and inflation-adjusted analysis.

Initial Sources

- FRED

Initial Indicators

- CPI
- Inflation Rate
- Federal Funds Rate
- GDP
- Unemployment Rate

Purpose

Provide economic context and support inflation-adjusted investment calculations.

2.6.9 Users

User data shall remain intentionally minimal during MVP development.

Initial User Information

- Email
- Password Hash
- Display Name
- Creation Timestamp

Design Principle

The platform's primary objective is investment simulation, not social functionality.

2.6.10 Simulations

Every simulation generated by the platform shall be persisted.

Purpose

Enable:

- Reproducibility
- User History
- Auditing
- Debugging
- Future Sharing Features

Stored Information

- Asset
 - Investment Amount
 - Start Date
 - End Date
 - Dividend Settings
 - Inflation Settings
 - Shares Purchased
 - Final Value
 - CAGR
 - Total Return
 - Data Snapshot Reference
-

2.6.11 Audit Logs

Audit logging is mandatory.

Purpose

Provide complete traceability of platform activity.

Example Events

- Simulation Created
- Asset Added
- Data Updated
- Data Import Failure
- AI Explanation Generated

Requirement

Every significant system action must be traceable through audit records.

2.6.12 AI Explanations

AI-generated content shall be stored independently from simulation results.

Purpose

Preserve separation between:

- Deterministic financial calculations
- Generated educational content

Stored Information

- Simulation Reference
- Model Name
- Prompt Version
- Generated Output
- Creation Timestamp

2.6.13 Indexing Strategy

Indexes shall prioritize historical data retrieval performance.

Required Indexes

- Asset Symbol
- Historical Price Asset ID
- Historical Price Date
- Asset ID + Date Composite Index
- Dividend Date Index
- Split Date Index
- User Simulation History Index
- Economic Indicator Date Index

Design Goal

Optimize the platform's most common query:

Retrieve historical data for a specific asset across a date range.

2.6.14 Partitioning Strategy

Partitioning is deferred during MVP development.

Decision

Do not introduce partitioning until performance requirements justify additional complexity.

Future Candidates

- Historical Prices
 - Audit Logs
 - Economic Indicator Values
-

2.6.15 Data Source Tracking

Every imported financial record must contain source attribution.

Approved Sources

- yfinance
- CoinGecko
- FRED
- Manual Import

Requirement

Source attribution is mandatory for transparency and auditing.

2.6.16 Migration Strategy

Alembic shall be used for schema migrations.

Requirements

Every schema modification must include:

- Migration File
- Version Control Record
- Documentation Update
- Rollback Consideration

Database changes shall never occur outside the migration system.

2.6.17 Architecture Decisions Summary

Approved Decisions:

- PostgreSQL selected as primary database.

- Historical market data stored locally.
 - Dividends stored separately from prices.
 - Stock splits stored separately from prices.
 - Simulations persisted for reproducibility.
 - Audit logging required.
 - AI outputs isolated from financial calculations.
 - Data source attribution mandatory.
 - Partitioning deferred until necessary.
 - Alembic selected for migration management.
-

Status: Approved

Part 2.6 Complete.

Investment Time Machine

Founder Specification

Part 2.6.18 — Physical Schema Design

Status: Draft

Purpose

This section establishes the database standards that all production tables must follow.

The goal of the Physical Schema Design is to ensure:

- Consistency
- Predictability
- Scalability
- Auditability
- Data Integrity

These standards apply to every database object created within the Investment Time Machine platform.

2.6.18.1 Physical Schema Philosophy

The database is the source of truth for all historical investment calculations.

Schema design must prioritize:

1. Accuracy over convenience
2. Explicit structure over flexibility
3. Data integrity over development speed
4. Reproducibility over optimization
5. Long-term maintainability over short-term simplicity

Every table must be understandable without requiring application code.

The schema itself should communicate intent.

2.6.18.2 Primary Key Strategy

Decision

All primary keys shall use UUIDs.

Example:

```
id UUID PRIMARY KEY
```

Rationale

UUIDs provide:

- Global uniqueness

- Safer future microservice expansion
- Easier data migration
- Better support for distributed systems

Future database merges become significantly easier.

Tradeoffs

UUIDs require more storage than integers.

This tradeoff is accepted because scalability and portability are more important than small storage savings.

2.6.18.3 Foreign Key Standards

All relationships must be enforced using database-level foreign keys.

Example:

```
asset_id UUID NOT NULL
```

```
REFERENCES assets(id)
```

Rules

Foreign keys are mandatory whenever a relationship exists.

Application-level relationships alone are prohibited.

Rationale

Database constraints prevent orphaned records and ensure referential integrity.

2.6.18.4 Timestamp Standards

Every production table must contain:

created_at TIMESTAMP WITH TIME ZONE

updated_at TIMESTAMP WITH TIME ZONE

Rules

created_at

- Set once
- Never modified

updated_at

- Updated automatically whenever a record changes

Time Zone Policy

All timestamps must be stored in UTC.

Local time conversion occurs only in the application layer.

Rationale

UTC storage eliminates timezone ambiguity and simplifies historical analysis.

2.6.18.5 Naming Conventions

Table Names

All table names must be:

lowercase

plural

snake_case

Examples:

assets

historical_prices

stock_splits

audit_logs

Column Names

All column names must use:

snake_case

Examples:

asset_id

price_date

created_at

Index Names

Format:

idx_<table>_<column>

Examples:

idx_assets_symbol

idx_prices_asset_date

Foreign Key Names

Format:

fk_<table>_<referenced_table>

Examples:

fk_prices_assets

fk_dividends_assets

2.6.18.6 Soft Delete Policy

Decision

MVP tables shall not use soft deletes by default.

Records should be physically removed when deletion is appropriate.

Exception

Historical financial records should almost never be deleted.

Instead:

is_active BOOLEAN

may be used where deactivation is preferred.

Rationale

Soft deletes introduce:

- Query complexity
- Additional indexes
- Hidden bugs

Avoid until a real requirement exists.

2.6.18.7 Data Source Attribution Standard

All imported financial data must include:

data_source VARCHAR(50)

Approved Initial Sources

yfinance

coingecko

fred

manual_import

Rationale

Every financial value must be traceable to its origin.

Users and developers must be able to determine:

- where data came from
- when it was imported
- which provider supplied it

2.6.18.8 Enum Strategy

Decision

Use PostgreSQL ENUM types for controlled values.

Approved Initial Enums

asset_type_enum

stock

etf

crypto

index

economic_indicator

simulation_status_enum

pending

completed

failed

event_type_enum

Future audit event categories.

Rationale

Enums prevent invalid values while improving query consistency.

Tradeoffs

Enums require migrations when values change.

Accepted.

Controlled values are preferred over unrestricted strings.

2.6.18.9 Numeric Data Standards

Financial calculations require precision.

Currency Values

Use:

NUMERIC(20,8)

Examples:

price

dividend_amount

portfolio_value

Percentage Values

Use:

NUMERIC(10,6)

Examples:

cagr

return_percentage

inflation_rate

Prohibited

FLOAT

REAL

DOUBLE PRECISION

for financial calculations.

Rationale

Floating-point rounding errors are unacceptable in investment calculations.

2.6.18.10 Constraint Standards

Every table must define constraints whenever possible.

Examples

Unique Symbols

UNIQUE(symbol)

Unique Historical Prices

UNIQUE(asset_id, price_date, data_source)

Positive Investment Amounts

CHECK(investment_amount > 0)

Positive Prices

CHECK(close_price >= 0)

Rationale

Invalid data should be rejected by the database itself.

2.6.18.11 Indexing Standards

Indexes shall be created only for:

- frequent lookups
 - joins
 - filtering operations
 - range queries
-

Required Categories

Primary Keys

id

Foreign Keys

asset_id

user_id

simulation_id

Date-Based Queries

price_date

created_at

Composite Query Paths

asset_id + price_date

Rule

Indexes must have a documented purpose.

Indexes may not be added without justification.

2.6.18.12 Auditability Requirements

All business-critical entities must be traceable.

The system must always be capable of answering:

What happened?

When did it happen?

Who triggered it?

Which data was affected?

Required Audit Coverage

- Asset creation
 - Data imports
 - Simulation generation
 - AI explanation generation
 - Administrative actions
-

2.6.18.13 Migration Standards

Schema changes must only occur through Alembic migrations.

Direct database modifications are prohibited.

Every Migration Must Include

- Migration identifier
 - Description
 - Version control record
 - Rollback awareness
-

Rationale

The database schema must evolve in a controlled and reproducible manner.

2.6.18.14 Future Scalability Standards

All schemas must be designed with future expansion in mind.

Future additions may include:

- Portfolio management
- Watchlists
- Retirement planning
- Tax analysis

- Alternative assets
- International markets

Current schema decisions should avoid blocking future growth.

2.6.18.15 Physical Schema Design Summary

Approved Standards:

- UUID primary keys
- Database-enforced foreign keys
- UTC timestamps
- Snake_case naming conventions
- Mandatory source attribution
- PostgreSQL enums
- NUMERIC financial precision
- Constraint-first validation
- Purpose-driven indexing
- Auditability requirements
- Alembic-only schema changes

These standards shall govern every database table defined in subsequent sections of Part 2.6.

Status:

Part 2.6.18 Complete

Ready for Part 2.6.19 — Asset Table Specification

Investment Time Machine

Founder Specification

Part 2.6.19 — Asset Table Specification

Status: Draft

Purpose

The Asset Table is the master registry for every financial instrument supported by the platform.

All other financial datasets reference assets through foreign keys.

Examples:

- AAPL
- MSFT
- SPY
- BTC
- ETH

The Asset Table serves as the authoritative identity layer for market data.

Table Definition

Table Name

assets

Primary Key

id UUID PRIMARY KEY

Columns

Column	Type	Constraints
id	UUID	PK
symbol	VARCHAR(20)	NOT NULL
name	VARCHAR(255)	NOT NULL
asset_type	asset_type_enum	NOT NULL
exchange	VARCHAR(50)	NULL
currency	VARCHAR(10)	NOT NULL
country	VARCHAR(50)	NULL
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL

Asset Type Enum

stock

etf

crypto

index

economic_indicator

Future additions:

bond

commodity

mutual_fund

forex

Adding enum values requires a migration.

Constraints

Unique Symbol Constraint

UNIQUE(symbol)

MVP assumption:

Each symbol is globally unique.

Examples:

AAPL

BTC

SPY

Active Status Constraint

Assets should not be deleted unless data corruption occurs.

Preferred behavior:

`is_active = false`

for delisted or unsupported assets.

Indexes

Symbol Lookup

`idx_assets_symbol`

Supports:

Find asset by ticker

Asset Type Lookup

`idx_assets_asset_type`

Supports:

List all ETFs

List all Cryptocurrencies

Relationships

One Asset → Many Historical Prices

assets.id



historical_prices.asset_id

One Asset → Many Dividends

assets.id



dividends.asset_id

One Asset → Many Stock Splits

assets.id



stock_splits.asset_id

One Asset → Many Simulations

assets.id



simulations.asset_id

Design Decisions

UUIDs Instead of Integers

Chosen because:

- globally unique
- easier future migrations
- better long-term scalability

Storage overhead is acceptable.

Separate Asset Metadata

Do not duplicate:

symbol

name

currency

inside other tables.

All market datasets should reference:

asset_id

This prevents inconsistencies and reduces storage requirements.

Asset Registry as Single Source of Truth

No price, dividend, split, or simulation record may exist without a valid asset record.

Foreign key enforcement is mandatory.

Future Expansion

Potential future columns:

sector

industry

market_cap

description

website

logo_url

provider_symbol

These are intentionally excluded from MVP to keep the schema focused on simulation requirements.

Approved Design Summary

The Asset Table serves as:

- Asset identity registry
- Metadata source
- Foreign key anchor point

All financial datasets depend on this table.

The Asset Table is therefore considered a Tier 1 Core Table within the Investment Time Machine database architecture.

Status:

Part 2.6.19 Complete

Ready for Part 2.6.20 — Historical Price Table Specification

Investment Time Machine

Founder Specification

Part 2.6.20 — Historical Price Table Specification

Status: Draft

Purpose

The Historical Price Table stores daily market pricing data for all supported assets.

This table is the foundation of the simulation engine and is the most frequently queried table in the entire platform.

All investment calculations ultimately depend on the accuracy of this dataset.

Table Definition

Table Name

historical_prices

Primary Key

id UUID PRIMARY KEY

Columns

Column	Type	Constraints
id	UUID	PRIMARY KEY
asset_id	UUID	NOT NULL, FK assets(id)
price_date	DATE	NOT NULL
open_price	NUMERIC(20,8)	NOT NULL
high_price	NUMERIC(20,8)	NOT NULL
low_price	NUMERIC(20,8)	NOT NULL
close_price	NUMERIC(20,8)	NOT NULL
adjusted_close_price	NUMERIC(20,8)	NOT NULL
volume	BIGINT	NULL
data_source	VARCHAR(50)	NOT NULL
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL

Constraints

Foreign Key

asset_id REFERENCES assets(id)

Every price record must belong to a valid asset.

Uniqueness Constraint

UNIQUE(asset_id, price_date, data_source)

Prevents duplicate records for the same asset and trading day.

Price Validation

CHECK(open_price >= 0)
CHECK(high_price >= 0)
CHECK(low_price >= 0)
CHECK(close_price >= 0)
CHECK(adjusted_close_price >= 0)

Negative market prices are not allowed.

Volume Validation

CHECK(volume >= 0)

Indexes

Primary Query Index

idx_historical_prices_asset_date
(asset_id, price_date)

Supports the platform's most common query:

Retrieve an asset's historical prices over a date range.

Date Index

idx_historical_prices_date
(price_date)

Supports:

- historical imports

- market-wide analysis
 - maintenance operations
-

Asset Index

idx_historical_prices_asset
(asset_id)

Supports asset-specific lookups.

Data Source Strategy

Each price record must include source attribution.

Approved MVP sources:

yfinance
coingecko
manual_import

Future sources may be added without schema changes.

Adjusted Close Policy

The platform shall use:

adjusted_close_price

as the default price for simulations.

Adjusted prices automatically account for:

- stock splits
- dividends (provider dependent)
- corporate actions

This simplifies MVP calculations while maintaining historical accuracy.

Raw price fields are retained for transparency and future advanced simulations.

Data Granularity

MVP Scope

Daily market data only.

Supported frequency:

1 Day

Not supported:

1 Minute

5 Minute

15 Minute

Hourly

Tick Data

Rationale

Daily data provides:

- educational value
- historical accuracy
- manageable storage costs

while avoiding the complexity of high-frequency datasets.

Expected Scale

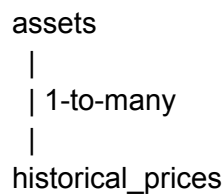
Estimated records:

Asset Count	Years	Approximate Rows
1,000	10	~2.5 million
5,000	20	~25 million
10,000	20	~50 million

PostgreSQL can comfortably support this scale with proper indexing.

Partitioning remains unnecessary for MVP.

Relationship Diagram



One asset can have thousands of historical price records.

Each historical price record belongs to exactly one asset.

Design Decisions

Store Adjusted and Raw Prices

Both datasets are preserved.

Benefits:

- transparent calculations
- easier auditing
- future support for advanced simulation modes

Storage overhead is acceptable.

Separate Prices From Corporate Actions

Dividends and stock splits remain in dedicated tables.

Benefits:

- cleaner data model
 - easier validation
 - simpler future expansion
-

Local Historical Ownership

Historical prices are stored permanently after ingestion.

Simulations must never depend on live API calls.

This guarantees:

- reproducibility
 - reliability
 - performance
-

Future Expansion

Potential future columns:

market_cap
shares_outstanding
vwap
trading_status
after_hours_price

These fields are intentionally excluded from MVP.

The table should remain focused on simulation requirements.

Approved Design Summary

The Historical Price Table is the most critical dataset in the platform.

It serves as:

- the foundation of all simulations
- the primary source of historical market truth
- the largest financial dataset in the database

Because every investment calculation depends on this table, it is classified as a Tier 1 Core Table.

Status:

Part 2.6.20 Complete

Next: Part 2.6.21 — Dividend Table Specification

Investment Time Machine

Founder Specification

Part 2.6.21 — Dividend Table Specification

Status: Draft

Purpose

The Dividend Table stores cash distributions paid by assets.

This table exists separately from historical prices because dividends represent a distinct financial event and are required for accurate total-return calculations.

The table enables:

- Dividend reinvestment simulations
- Total return calculations
- Dividend income analysis
- Future dividend-focused features

Table Definition

Table Name

dividends

Primary Key

id UUID PRIMARY KEY

Columns

Column		Type	Constraints
id	UUID		PRIMARY KEY
asset_id	UUID		NOT NULL, FK assets(id)
ex_dividend_date	DATE		NOT NULL
payment_date	DATE		NULL
dividend_amount	NUMERIC(20,8)		NOT NULL
currency	VARCHAR(10)		NOT NULL
data_source	VARCHAR(50)		NOT NULL
created_at	TIMESTAMP WITH TIME ZONE		NOT NULL
updated_at	TIMESTAMP WITH TIME ZONE		NOT NULL

Constraints

Foreign Key

asset_id REFERENCES assets(id)

Every dividend must belong to a valid asset.

Uniqueness Constraint

UNIQUE(asset_id, ex_dividend_date, dividend_amount)

Prevents duplicate dividend records.

Dividend Validation

CHECK(dividend_amount >= 0)

Dividend values cannot be negative.

Indexes

Primary Dividend Lookup

idx_dividends_asset_date
(asset_id, ex_dividend_date)

Supports:

- dividend reinvestment calculations
 - historical dividend retrieval
 - total return analysis
-

Payment Date Index

idx_dividends_payment_date
(payment_date)

Supports future dividend-calendar features.

Date Definitions

Ex-Dividend Date

The date on which an investor must already own shares to receive the dividend.

This is the primary date used during simulations.

Payment Date

The date cash is distributed to shareholders.

Stored for completeness and future analytics.

Not required for MVP calculations.

Currency Policy

Dividend currency must always be stored explicitly.

Examples:

USD
EUR
GBP
JPY

This supports future international assets and avoids hidden assumptions.

Data Source Strategy

Approved MVP sources:

yfinance
manual_import

Every dividend record must preserve its source.

Source attribution is mandatory for auditability.

Simulation Usage

When dividend reinvestment is enabled:

1. Dividend events are retrieved for the simulation period.
2. Cash dividends are calculated based on shares owned.
3. Additional shares are purchased using the dividend proceeds.
4. Share count increases over time.
5. Final portfolio value reflects reinvested distributions.

When dividend reinvestment is disabled:

- Dividends are ignored for growth calculations.
 - Price appreciation alone determines performance.
-

Relationship Diagram

assets
|
| 1-to-many
|
dividends

One asset may have many dividend records.

Each dividend record belongs to exactly one asset.

Design Decisions

Separate Dividend Storage

Dividend information shall not be embedded inside historical price records.

Benefits:

- cleaner schema
 - easier validation
 - simpler simulation logic
 - support for dividend-specific analytics
-

Preserve Raw Dividend Events

Dividend records should remain unchanged after ingestion.

The simulation engine performs calculations using stored events rather than precomputed totals.

This ensures:

- transparency
 - reproducibility
 - easier auditing
-

Store Dividend Currency

Although MVP assets will primarily use USD, currency storage is required from day one.

Future international expansion should not require schema changes.

Expected Scale

Estimated dividend volume:

Asset Type	Typical Events
Quarterly Stock	4 per year
Monthly ETF	12 per year
Growth Stock	0 per year

Compared to historical prices, dividend records represent a very small dataset.

No special scaling strategy is required.

Future Expansion

Potential future columns:

record_date
declaration_date
dividend_type
special_dividend_flag
withholding_tax_rate

These fields are intentionally excluded from MVP.

The table should remain focused on supporting accurate historical simulations.

Approved Design Summary

The Dividend Table serves as the authoritative record of shareholder cash distributions.

It supports:

- dividend reinvestment

- total return calculations
- income analysis
- future dividend-focused features

By separating dividends from price history, the platform preserves accuracy, transparency, and long-term flexibility.

Status:

Part 2.6.21 Complete

Next: Part 2.6.22 — Stock Split Table Specification

Investment Time Machine

Founder Specification

Part 2.6.22 — Stock Split Table Specification

Status: Draft

Purpose

The Stock Split Table stores corporate actions that change the number of shares outstanding without changing the total value of the investment.

Examples include:

- Stock splits (4-for-1)
- Reverse splits (1-for-10)

This table enables accurate historical share tracking and supports future advanced simulation modes.

Table Definition

Table: stock_splits

Primary Key:

id UUID PRIMARY KEY

Columns

Column	Type	Constraints
id	UUID	PRIMARY KEY
asset_id	UUID	NOT NULL, FK assets(id)
split_date	DATE	NOT NULL
split_ratio	NUMERIC(10,4)	NOT NULL
data_source	VARCHAR(50)	NOT NULL
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL

Constraints

Foreign Key

asset_id REFERENCES assets(id)

Every split event must belong to a valid asset.

Uniqueness Constraint

UNIQUE(asset_id, split_date, split_ratio)

Prevents duplicate split events.

Ratio Validation

CHECK(split_ratio > 0)

Split ratios must always be positive.

Split Ratio Standard

The platform shall store splits as multiplication factors.

Examples:

Event	Stored Ratio
2-for-1 Split	2.0
3-for-1 Split	3.0
4-for-1 Split	4.0
1-for-2 Reverse Split	0.5
1-for-10 Reverse Split	0.1

This standard simplifies calculations and reduces ambiguity.

Indexes

Primary Lookup Index

idx_stock_splits_asset_date
(asset_id, split_date)

Supports:

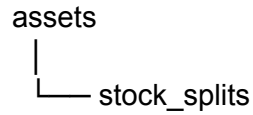
- simulation calculations
- historical corporate action retrieval
- audit operations

Date Index

idx_stock_splits_date
(split_date)

Supports bulk imports and maintenance operations.

Relationship



One asset may have many split events.

Each split event belongs to exactly one asset.

Simulation Usage

Stock splits affect:

share_count

Stock splits do not directly affect:

investment_value

Example:

Before Split:

100 shares × \$40 = \$4,000

After 4-for-1 Split:

400 shares × \$10 = \$4,000

Total value remains unchanged.

The simulation engine must adjust share counts accordingly.

Adjusted Price Policy

For MVP simulations, the system shall primarily use:

adjusted_close_price

from the Historical Price Table.

Because adjusted prices already account for most split events, the Stock Split Table exists primarily for:

- auditability
- transparency
- future advanced simulations
- corporate action analysis

This prevents double-adjustment errors.

Data Source Strategy

Approved MVP sources:

yfinance
manual_import

Every split event must preserve source attribution.

Expected Scale

Split events are rare compared to price records.

Example:

Asset Type	Typical Split Frequency
Large-Cap Stock	Every few years
ETF	Rare
Cryptocurrency	Extremely rare

Storage and performance impact are negligible.

Design Decisions

Separate Corporate Action Storage

Stock splits are stored independently from price records.

Benefits:

- cleaner schema
- easier validation
- future support for corporate action analytics
- simpler simulation logic

Preserve Historical Events

Split events should never be overwritten by calculated adjustments.

The database should retain the original corporate action record for auditing and verification purposes.

Future Expansion

Potential future columns:

announcement_date
record_date
split_type
notes
source_reference

These fields are intentionally excluded from MVP.

Summary

The Stock Split Table serves as the authoritative record of share-adjustment events.

It supports:

- accurate historical simulations
- corporate action tracking
- future advanced analytics
- transparent auditing

Although relatively small, this table is essential for maintaining historical correctness across long investment periods.

Status: Part 2.6.22 Complete

Next: Part 2.6.23 — User Table Specification

Investment Time Machine

Founder Specification

Part 2.6.23 — User Table Specification

Status: Draft

Purpose

The User Table stores account and authentication information for platform users.

The MVP should keep user data intentionally minimal. The platform's primary focus is historical investment simulation, not social networking or profile management.

The table must support:

- Authentication
- Simulation ownership
- Saved history
- Future personalization features

Table Definition

Table: users

Primary Key:

id UUID PRIMARY KEY

Columns

Column	Type	Constraints
id	UUID	PRIMARY KEY
email	VARCHAR(255)	NOT NULL
password_hash	VARCHAR(255)	NOT NULL
display_name	VARCHAR(100)	NOT NULL
is_active	BOOLEAN	DEFAULT TRUE
is_admin	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL

Constraints

Email Uniqueness

UNIQUE(email)

A single email address may only belong to one account.

Email Validation

Email format validation should occur at the application layer.

The database is responsible only for storage and uniqueness.

Account Status

is_active BOOLEAN DEFAULT TRUE

Inactive users cannot access authenticated features.

Indexes

Email Lookup

idx_users_email
(email)

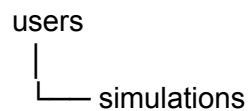
Supports login and account retrieval.

Account Status

idx_users_is_active
(is_active)

Supports administrative queries and user management.

Relationships



One user may create many simulations.

Each simulation belongs to exactly one user.

Future relationships:



These are not part of MVP.

Authentication Policy

The database shall never store:

plain_text_password
password_hint
security_answers

Only password hashes may be stored.

Approved authentication methods:

email_password
google_oauth
github_oauth

MVP may launch with email/password only.

User Data Philosophy

Only collect information that directly supports platform functionality.

MVP user data should remain limited to:

- Identity
- Authentication
- Simulation ownership

Avoid collecting unnecessary profile information.

Examples intentionally excluded:

phone_number
address
birth_date
occupation
investment_profile

These fields provide little value to the MVP and increase privacy obligations.

Administrative Access

Administrative privileges shall be controlled using:

is_admin BOOLEAN DEFAULT FALSE

Administrative users may eventually access:

- Data ingestion controls
- Audit logs
- Asset management
- System monitoring

Role complexity should remain minimal until justified.

Security Requirements

User records are considered sensitive data.

Requirements:

- Password hashing required
- HTTPS required in production
- No plaintext credentials
- No credential logging
- Principle of least privilege

Authentication logic belongs in the application layer, not the database.

Design Decisions

Minimal User Model

The MVP prioritizes simulation functionality over account complexity.

Benefits:

- simpler architecture
- lower security risk

- faster development
- easier compliance

UUID-Based Identity

User IDs follow the platform-wide UUID standard.

This ensures consistency across all tables and future services.

Soft Deactivation

Users should generally be deactivated rather than deleted.

Benefits:

- preserves simulation ownership
 - preserves audit history
 - simplifies data integrity
-

Future Expansion

Potential future columns:

profile_image_url
timezone
preferred_currency
theme_preference
notification_settings
last_login_at
email_verified_at

Potential future tables:

user_preferences
user_sessions
user_notifications

These features are intentionally deferred until user demand justifies additional complexity.

Summary

The User Table provides identity and ownership within the platform.

It supports:

- authentication
- simulation history
- account management
- future personalization

The MVP design intentionally minimizes stored user data while preserving a clear path for future expansion.

Status: Part 2.6.23 Complete

Next: Part 2.6.24 — Simulation Table Specification

Investment Time Machine

Founder Specification

Part 2.6.24 — Simulation Table Specification

Status: Draft

Purpose

The Simulation Table stores every investment simulation generated by the platform. This table is critical because simulations must be reproducible, auditable, and explainable.

A simulation record must preserve:

- User inputs
- Calculation assumptions
- Key output values
- Asset reference
- Data source context
- Execution status

The simulation engine must be able to reproduce or explain a result from stored data without relying on AI-generated content.

Table Definition

Table: simulations

Primary Key:
id UUID PRIMARY KEY

Columns

Column	Type	Constraints
id	UUID	PRIMARY KEY
user_id	UUID	NULL, FK users(id)
asset_id	UUID	NOT NULL, FK assets(id)
investment_amount	NUMERIC(20,8)	NOT NULL
start_date	DATE	NOT NULL
end_date	DATE	NOT NULL
include_dividends	BOOLEAN	DEFAULT FALSE
adjust_for_inflation	BOOLEAN	DEFAULT FALSE
initial_price	NUMERIC(20,8)	NOT NULL
final_price	NUMERIC(20,8)	NOT NULL
shares_purchased	NUMERIC(20,8)	NOT NULL
final_value	NUMERIC(20,8)	NOT NULL
total_return_percent	NUMERIC(10,6)	NOT NULL
cagr	NUMERIC(10,6)	NULL
inflation_adjusted_value	NUMERIC(20,8)	NULL
simulation_status	simulation_status_enum	NOT NULL
data_source	VARCHAR(50)	NOT NULL
error_message	TEXT	NULL

created_at	TIMESTAMP WITH TIME ZONE	NOT NULL
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL

Constraints

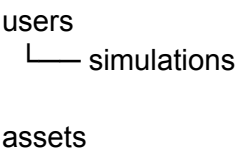
user_id REFERENCES users(id)
asset_id REFERENCES assets(id)
CHECK(investment_amount > 0)
CHECK(end_date >= start_date)
CHECK(initial_price >= 0)
CHECK(final_price >= 0)
CHECK(shares_purchased >= 0)
CHECK(final_value >= 0)

user_id is nullable so anonymous simulations can exist during MVP if authentication is delayed.

Indexes

Index	Columns	Purpose
idx_simulations_user_created	user_id, created_at	User simulation history
idx_simulations_asset_created	asset_id, created_at	Asset-level simulation analytics
idx_simulations_status	simulation_status	Debugging failed or pending simulations
idx_simulations_created_at	created_at	Admin review and chronological queries

Relationships



└─ simulations

simulations

└─ ai_explanations

One user may have many simulations. One asset may appear in many simulations. One simulation may have zero or many AI explanations.

Simulation Status Enum

pending

completed

failed

pending supports asynchronous execution. **completed** means all deterministic calculations finished successfully. **failed** means the simulation could not be completed and must store an error message.

Stored Inputs

The simulation record must preserve the original request inputs:

- Asset
- Investment amount
- Start date
- End date
- Dividend inclusion setting
- Inflation adjustment setting

These inputs are required for reproducibility and user history.

Stored Outputs

The simulation record must preserve deterministic outputs:

- Initial price

- Final price
- Shares purchased
- Final value
- Total return percentage
- CAGR
- Inflation-adjusted value when enabled

AI explanations are not stored in this table.

Reproducibility Policy

A simulation must never depend on a live API call after creation. Once a simulation is stored, the platform should be able to explain what was calculated and which assumptions were used.

Future versions should add a dedicated `data_snapshot_id` or equivalent structure to preserve exact historical data versions used during calculation.

For MVP, the simulation table stores core calculated outputs directly, while historical data remains traceable through asset, date range, and data source.

Design Decisions

Store Inputs and Outputs Together

The table stores both request inputs and calculation outputs. This improves debugging, user history, and reproducibility.

Allow Anonymous Simulations

`user_id` is nullable to support a public simulator experience without requiring account creation.

Separate AI From Simulation Results

AI explanations must remain outside the simulation table. This protects the distinction between deterministic financial results and generated educational content.

Include Failure State

Failed simulations should be stored when useful for debugging, especially when missing data or invalid date ranges prevent calculation.

Future Expansion

Potential future columns:

data_snapshot_id
calculation_version
dividend_reinvestment_value
inflation_rate_used
fees_applied
taxes_applied
portfolio_id

Potential future tables:

simulation_inputs
simulation_outputs
simulation_events
simulation_data_snapshots

These are intentionally deferred until simulations become more complex.

Summary

The Simulation Table is the platform's record of historical investment calculations. It supports reproducibility, user history, debugging, analytics, and future sharing features.

It is classified as a Tier 1 Core Table because every user-facing investment result depends on it.

Status: Part 2.6.24 Complete

Next: Part 2.6.25 — Audit Log Table Specification

Investment Time Machine

Founder Specification

Part 2.6.25 — Audit Log Table Specification

Status: Draft

Purpose

The Audit Log Table records important system events so the platform can explain what happened, when it happened, who triggered it, and which data was affected.

Audit logging is required because Investment Time Machine depends on trust, reproducibility, and traceability.

The table supports:

- Debugging
- Data integrity review
- Administrative monitoring
- Simulation traceability
- Future compliance needs

Table Definition

Table: audit_logs

Primary Key:

id UUID PRIMARY KEY

Columns

Column	Type	Constraints
id	UUID	PRIMARY KEY
event_type	event_type_enum	NOT NULL
entity_type	VARCHAR(100)	NOT NULL
entity_id	UUID	NULL
user_id	UUID	NULL, FK users(id)
details	JSONB	NULL

ip_address	VARCHAR(45)	NULL
user_agent	TEXT	NULL
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL

Constraints

user_id REFERENCES users(id)

entity_id is nullable because some events may describe system-level activity rather than a specific record.

details uses JSONB to store event-specific metadata without requiring a new table for every audit event type.

Indexes

Index	Columns	Purpose
idx_audit_logs_event_type	event_type	Filter by event category
idx_audit_logs_entity	entity_type, entity_id	Trace activity for a specific record
idx_audit_logs_user_created	user_id, created_at	Review user-triggered events
idx_audit_logs_created_at	created_at	Chronological review and admin queries

Event Types

Initial audit events should include:

- asset_created
- asset_updated
- historical_price_imported
- historical_price_import_failed
- dividend_imported
- split_imported
- simulation_created

simulation_failed
ai_explanation_generated
user_created
admin_action

Additional event types may be added through migrations.

Audit Detail Examples

For a simulation event:

```
{  
  "asset_symbol": "AAPL",  
  "investment_amount": "1000.00",  
  "start_date": "2015-01-01",  
  "end_date": "2025-01-01",  
  "include_dividends": true  
}
```

For a data import failure:

```
{  
  "data_source": "yfinance",  
  "asset_symbol": "TSLA",  
  "error": "missing adjusted close price",  
  "import_date": "2026-06-01"  
}
```

Relationships

users
└── audit_logs

A user may trigger many audit events. Some audit events may be system-generated and have no user.

Design Decisions

JSONB for Details

Audit events vary widely, so `details` should use JSONB instead of forcing every event into rigid columns. This keeps the audit table flexible while preserving structured metadata.

No Updated Timestamp

Audit records should be immutable. The table stores `created_at` only. Audit events should not be edited after creation.

Nullable User

System jobs, scheduled imports, and background workers may create audit events without a user account.

Entity-Based Tracking

`entity_type` and `entity_id` allow the system to trace events related to assets, simulations, users, imports, and AI explanations without requiring separate audit tables for each entity.

Security Requirements

Audit logs must not store:

- passwords
- password_hashes
- API keys
- access tokens
- refresh tokens
- private credentials

Sensitive values should be redacted before being written to `details`.

Future Expansion

Potential future columns:

- request_id
- session_id
- severity
- service_name
- correlation_id

Potential future tables:

audit_log_archives
system_events
security_events
data_import_runs

These are deferred until operational complexity requires them.

Summary

The Audit Log Table provides traceability across the platform. It supports debugging, data integrity review, administrative monitoring, and future compliance needs.

It is classified as a Tier 1 Core Table because platform trust depends on being able to explain system actions.

Status: Part 2.6.25 Complete

Next: Part 2.6.26 — AI Explanation Table Specification

Investment Time Machine

Founder Specification

Part 2.6.26 — AI Explanation Table Specification

Status: Draft

Purpose

The AI Explanation Table stores educational content generated by the AI Intelligence Service.

This table exists to maintain strict separation between:

- Deterministic financial calculations
- AI-generated educational content

Simulation results are considered financial truth.

AI explanations are considered supplemental educational content.

The system must never treat AI-generated text as financial data.

Table Definition

Table: ai_explanations

Primary Key:
id UUID PRIMARY KEY

Columns

Column	Type	Constraints
id	UUID	PRIMARY KEY
simulation_id	UUID	NOT NULL, FK simulations(id)
model_name	VARCHAR(100)	NOT NULL
prompt_version	VARCHAR(50)	NOT NULL
input_summary	JSONB	NOT NULL
explanation_text	TEXT	NOT NULL
generation_status	ai_generation_status_enum	NOT NULL
token_count	INTEGER	NULL
generation_time_ms	INTEGER	NULL
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL

Constraints

simulation_id REFERENCES simulations(id)

Every AI explanation must be attached to a valid simulation.

AI explanations may never exist independently of simulation results.

Indexes

Index	Columns	Purpose
idx_ai_explanations_simulation	simulation_id	Retrieve explanations for a simulation
idx_ai_explanations_model	model_name	Model performance analysis
idx_ai_explanations_created	created_at	Historical review
idx_ai_explanations_status	generation_statuses	Failure monitoring

Generation Status Enum

pending
completed
failed

pending

Explanation request created but not yet generated.

completed

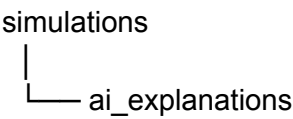
Explanation successfully generated and stored.

failed

Generation attempt failed.

Failure reason should be recorded in operational logs.

Relationships



One simulation may have:

- zero explanations
- one explanation
- multiple explanations

Multiple explanations support future features such as:

- prompt improvements
 - model comparisons
 - regenerated reports
-

Input Summary

The AI service should never receive unrestricted database access.

Instead, it receives a controlled summary of simulation results.

Example:

```
{  
  "asset": "AAPL",  
  "investment_amount": 1000,  
  "start_date": "2015-01-01",  
  "end_date": "2025-01-01",  
  "final_value": 7350,  
  "total_return_percent": 635  
}
```

This summary is stored for transparency and debugging.

AI Data Access Policy

The AI service may access:

- simulation outputs
- historical events
- economic indicators
- educational reference content

The AI service may not access:

- raw database tables
- user passwords
- administrative systems
- unrestricted financial datasets

All AI inputs must pass through controlled application-layer validation.

Prompt Versioning

Every explanation must store:

prompt_version

Example:

v1.0

v1.1

v2.0

This allows the platform to determine:

- which prompt generated a response
- when prompt changes improved quality
- whether a response should be regenerated

Prompt versioning is mandatory.

Model Tracking

Every explanation must store:

model_name

Examples:

gpt-5.5

gpt-6
claude-x
local-model-v1

This enables:

- quality comparisons
 - cost analysis
 - debugging
 - future model migrations
-

Design Decisions

Store AI Separately From Simulations

Simulation tables contain financial truth.

AI tables contain generated commentary.

Mixing the two would blur the distinction between facts and generated text.

Store Inputs Used By AI

The platform should preserve the exact information provided to the model.

Benefits:

- reproducibility
 - debugging
 - hallucination investigation
 - quality evaluation
-

Support Multiple Explanations

A simulation should support multiple explanation records.

This allows future experimentation without modifying historical simulation results.

Immutable AI Records

Generated explanations should not be edited after creation.

If content must change:

- create a new explanation record
- preserve the original record

This supports auditing and model evaluation.

Security Requirements

AI explanations must never store:

passwords
authentication tokens
private credentials
sensitive system configuration

User-generated content included in prompts should be sanitized before storage.

Future Expansion

Potential future columns:

confidence_score
cost_usd
temperature
provider_name
completion_tokens
prompt_tokens
feedback_score

Potential future tables:

ai_prompts

ai_models
ai_feedback
ai_generation_jobs

These features are intentionally deferred until AI usage patterns are established.

Summary

The AI Explanation Table stores educational content generated from simulation results while maintaining strict separation between financial calculations and AI-generated text.

It supports:

- explanation history
- prompt versioning
- model tracking
- auditing
- future AI experimentation

The table is classified as a Tier 2 Supporting Table because it enhances user understanding but is not required for simulation correctness.

Status: Part 2.6.26 Complete

Next: Part 2.6.27 — Entity Relationship Design

Investment Time Machine

Founder Specification

Part 2.6.26 — AI Explanation Table Specification

Status: Draft

Purpose

The AI Explanation Table stores educational content generated by the AI Intelligence Service.

This table exists to maintain strict separation between:

- Deterministic financial calculations
- AI-generated educational content

Simulation results are considered financial truth.

AI explanations are considered supplemental educational content.

The system must never treat AI-generated text as financial data.

Table Definition

Table: ai_explanations

Primary Key:

id UUID PRIMARY KEY

Columns

Column	Type	Constraints
id	UUID	PRIMARY KEY
simulation_id	UUID	NOT NULL, FK simulations(id)
model_name	VARCHAR(100)	NOT NULL
prompt_version	VARCHAR(50)	NOT NULL
input_summary	JSONB	NOT NULL

explanation_text	TEXT	NOT NULL
generation_status	ai_generation_status_enum	NOT NULL
token_count	INTEGER	NULL
generation_time_ms	INTEGER	NULL
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL

Constraints

simulation_id REFERENCES simulations(id)

Every AI explanation must be attached to a valid simulation.

AI explanations may never exist independently of simulation results.

Indexes

Index	Columns	Purpose
idx_ai_explanations_simulation	simulation_id	Retrieve explanations for a simulation

idx_ai_explanations_model	model_name	Model performance analysis
idx_ai_explanations_created	created_at	Historical review
idx_ai_explanations_status	generation_statuses	Failure monitoring

Generation Status Enum

pending

completed

failed

pending

Explanation request created but not yet generated.

completed

Explanation successfully generated and stored.

failed

Generation attempt failed.

Failure reason should be recorded in operational logs.

Relationships

simulations

|
└─ ai_explanations

One simulation may have:

- zero explanations
- one explanation
- multiple explanations

Multiple explanations support future features such as:

- prompt improvements
 - model comparisons
 - regenerated reports
-

Input Summary

The AI service should never receive unrestricted database access.

Instead, it receives a controlled summary of simulation results.

Example:

```
{  
  "asset": "AAPL",  
  "investment_amount": 1000,  
  "start_date": "2015-01-01",  
  "end_date": "2025-01-01",  
  "final_value": 7350,  
  "total_return_percent": 635  
}
```

This summary is stored for transparency and debugging.

AI Data Access Policy

The AI service may access:

- simulation outputs
- historical events
- economic indicators
- educational reference content

The AI service may not access:

- raw database tables
- user passwords
- administrative systems
- unrestricted financial datasets

All AI inputs must pass through controlled application-layer validation.

Prompt Versioning

Every explanation must store:

prompt_version

Example:

v1.0

v1.1

v2.0

This allows the platform to determine:

- which prompt generated a response
- when prompt changes improved quality
- whether a response should be regenerated

Prompt versioning is mandatory.

Model Tracking

Every explanation must store:

model_name

Examples:

gpt-5.5

gpt-6

claude-x

local-model-v1

This enables:

- quality comparisons
 - cost analysis
 - debugging
 - future model migrations
-

Design Decisions

Store AI Separately From Simulations

Simulation tables contain financial truth.

AI tables contain generated commentary.

Mixing the two would blur the distinction between facts and generated text.

Store Inputs Used By AI

The platform should preserve the exact information provided to the model.

Benefits:

- reproducibility
 - debugging
 - hallucination investigation
 - quality evaluation
-

Support Multiple Explanations

A simulation should support multiple explanation records.

This allows future experimentation without modifying historical simulation results.

Immutable AI Records

Generated explanations should not be edited after creation.

If content must change:

- create a new explanation record
- preserve the original record

This supports auditing and model evaluation.

Security Requirements

AI explanations must never store:

passwords

authentication tokens

private credentials

sensitive system configuration

User-generated content included in prompts should be sanitized before storage.

Future Expansion

Potential future columns:

confidence_score

cost_usd

temperature

provider_name

completion_tokens

prompt_tokens

feedback_score

Potential future tables:

ai_prompts

ai_models

ai_feedback

ai_generation_jobs

These features are intentionally deferred until AI usage patterns are established.

Summary

The AI Explanation Table stores educational content generated from simulation results while maintaining strict separation between financial calculations and AI-generated text.

It supports:

- explanation history
- prompt versioning
- model tracking
- auditing
- future AI experimentation

The table is classified as a Tier 2 Supporting Table because it enhances user understanding but is not required for simulation correctness.

Status: Part 2.6.26 Complete

Next: Part 2.6.27 — Entity Relationship Design

Investment Time Machine

Founder Specification

Part 2.6.28 — Future Database Expansion

Status: Draft

Purpose

This section documents anticipated database growth beyond the MVP.

The objective is not to design future features in detail, but to ensure current database decisions do not block future platform evolution.

The MVP database should be simple, but it should also provide a clear path toward becoming a comprehensive investment intelligence platform.

Expansion Philosophy

Future database growth should follow three principles:

Preserve Existing Data

New features should extend the schema rather than require destructive redesigns.

Maintain Backward Compatibility

Existing simulations and historical data should remain valid after future upgrades.

Avoid Premature Complexity

Future structures should be documented but not implemented until justified by product needs.

Portfolio Management

Purpose

Allow users to simulate and analyze collections of assets rather than individual investments.

Potential Tables

portfolios

portfolio_holdings

portfolio_snapshots

portfolio_performance

Example Relationships

users

|

└─ portfolios

|

└─ portfolio_holdings

|

└─ assets

Supported Features

- Multi-asset portfolios
 - Asset allocation analysis
 - Rebalancing simulations
 - Risk-adjusted performance
-

Watchlists

Purpose

Allow users to save assets for future analysis.

Potential Tables

watchlists

watchlist_assets

Example Relationships

users

|

└─ watchlists

|

└─ assets

Supported Features

- Saved assets
 - Asset tracking
 - Personalized dashboards
-

Saved Reports

Purpose

Allow users to preserve simulation outputs and AI-generated reports.

Potential Tables

saved_reports

report_versions

Supported Features

- Shareable reports
 - Historical report history
 - Research workflows
-

Retirement Planning

Purpose

Support long-term wealth planning simulations.

Potential Tables

retirement_plans

retirement_contributions

retirement_assumptions

Supported Features

- Retirement projections
- Contribution schedules
- Inflation assumptions
- Withdrawal modeling

Tax Modeling

Purpose

Support advanced after-tax investment analysis.

Potential Tables

tax_profiles

tax_events

tax_assumptions

Supported Features

- Capital gains estimation
- Dividend taxation
- Tax-adjusted returns

This functionality is explicitly out of scope for MVP.

Recurring Investment Simulations

Purpose

Support dollar-cost averaging and contribution schedules.

Potential Tables

investment_schedules

scheduled_contributions

Supported Features

- Monthly investing
 - Weekly investing
 - Custom contribution plans
-

Alternative Assets

Purpose

Expand beyond stocks, ETFs, and cryptocurrencies.

Potential Asset Types

bonds

commodities

mutual_funds

forex

real_estate

private_equity

Current asset architecture should support these additions through enum expansion rather than schema redesign.

International Markets

Purpose

Expand historical coverage beyond US-focused assets.

Additional Requirements

multiple_currencies

exchange_metadata

country_specific_assets

market_calendars

Potential Tables

currencies

exchange_rates

exchanges

market_holidays

AI Research Layer

Purpose

Support future AI-powered research capabilities.

Potential Tables

ai_prompts

ai_feedback

research_sessions

research_sources

Supported Features

- AI research assistants
 - Prompt experimentation
 - Feedback collection
 - Retrieval-augmented generation
-

Data Versioning

Purpose

Guarantee long-term reproducibility.

Potential Tables

data_snapshots

snapshot_assets

snapshot_prices

Benefits

- Exact simulation replay
- Historical auditability
- Dataset version tracking

This will become increasingly important as data providers evolve.

Analytics and Monitoring

Purpose

Support operational visibility and platform growth.

Potential Tables

usage_metrics

api_metrics

system_events

performance_metrics

Supported Features

- User analytics
 - System monitoring
 - Performance optimization
 - Capacity planning
-

Scaling Considerations

The current architecture should comfortably support:

10,000+ assets

50M+ historical price records

millions of simulations

Before introducing major infrastructure changes, the platform should first measure:

- Storage growth
- Query performance
- Import performance
- User activity patterns

Scaling decisions should be data-driven.

Deferred Features

The following are intentionally excluded from early versions:

social features

messaging

user feeds

copy trading

brokerage integrations

real-money investing

investment recommendations

These features do not align with the platform's educational mission.

Long-Term Database Vision

The database should evolve through the following stages:

Stage 1

Historical Investment Simulator

Core focus:

- assets
- historical prices
- simulations

Stage 2

Portfolio Analysis Platform

Core focus:

- portfolios
- allocation analysis
- risk metrics

Stage 3

Investment Research Platform

Core focus:

- saved reports
- AI research
- historical intelligence

Stage 4

Market Intelligence Platform

Core focus:

- large-scale analytics
 - advanced research tools
 - educational ecosystem
-

Summary

The MVP database architecture has been intentionally designed to support future growth without requiring major redesign.

Planned expansion areas include:

- Portfolio management
- Watchlists
- Saved reports
- Retirement planning
- Tax analysis
- Alternative assets
- International markets
- AI research
- Data versioning
- Platform analytics

These features are documented to guide future development but remain outside the scope of Version 1.

Status: Part 2.6.28 Complete

Part 2.6 — Database Architecture Complete

Next: Part 2.7 — AI Architecture

Investment Time Machine

Founder Specification

Part 2.7 — AI Architecture

Status: Draft

2.7.1 Purpose

The AI Architecture defines how artificial intelligence is integrated into the platform.

AI exists to improve understanding, not to generate financial truth.

The AI system serves as an educational layer built on top of deterministic simulation results.

The platform must remain fully functional even if every AI component is removed.

2.7.2 AI Philosophy

Investment Time Machine follows a strict separation between:

Financial Truth

and

Educational Interpretation

Financial Truth is produced by:

- Historical datasets
- Simulation engine
- Mathematical calculations
- Economic datasets

Educational Interpretation is produced by:

- Large language models
- Explanation systems
- Research assistance features

The AI layer may explain facts but may never create facts.

2.7.3 Core Principle

AI Is Not a Calculation Engine

The AI system must never perform authoritative financial calculations.

All investment metrics must originate from deterministic services.

Examples:

Metric	Source
Final Value	Simulation Engine
CAGR	Simulation Engine
Inflation Adjustment	Simulation Engine
Dividend Reinvestment	Simulation Engine
AI Explanation	AI Service

This separation is mandatory.

2.7.4 Responsibilities

The AI service is allowed to:

Explain Results

Example:

Why did NVIDIA outperform SPY during this period?

Explain Historical Events

Example:

What happened during the 2008 financial crisis?

Explain Investment Concepts

Example:

What is CAGR?

Summarize Economic Context

Example:

How did interest rates affect this investment?

Generate Educational Reports

Example:

Historical Performance Summary

Risk Summary

Market Context Summary

2.7.5 Prohibited Behaviors

The AI service must never:

Invent Market Data

Prohibited:

AAPL closed at \$212.45

unless supplied by verified data.

Invent Returns

Prohibited:

Your investment would have returned 845%.

unless provided by the simulation engine.

Modify Calculated Results

Prohibited:

Changing simulation outputs.

Execute Investment Logic

Prohibited:

Calculating CAGR

Calculating returns

Calculating dividends

Calculating inflation

Provide Investment Advice

Prohibited:

Buy NVIDIA.

Sell Tesla.

Invest in Bitcoin.

The platform is educational, not advisory.

2.7.6 AI Service Inputs

The AI service should only receive approved inputs.

Simulation Outputs

Example:

```
{  
  "asset": "AAPL",  
  "investment_amount": 1000,
```

```
"final_value": 7350,  
"cagr": 22.4  
}
```

Historical Context

Examples:

- Major market events
- Economic indicators
- Historical timelines

Educational Content

Examples:

- Financial definitions
- Investment concepts
- Platform documentation

2.7.7 AI Data Access Model

Approved Access

Simulation Outputs

Economic Indicators

Historical Event Database

Educational Knowledge Base

Restricted Access

Raw Database Access

User Passwords

Administrative Systems

Infrastructure Secrets

API Keys

AI should never have unrestricted access to production systems.

All inputs must pass through controlled application services.

2.7.8 Explanation Generation Pipeline

The explanation process follows a controlled workflow.

User Request



Simulation Engine



Validated Results



AI Service



Educational Explanation



Stored Explanation

At no point may AI modify simulation outputs.

The simulation engine always remains the source of truth.

2.7.9 Hallucination Prevention Strategy

Rule 1

Never ask AI to generate financial facts.

Instead:

Provide facts

Ask AI to explain facts

Rule 2

Use structured inputs.

Preferred:

```
{  
  "final_value": 5000,  
  "return": 250  
}
```

Avoid:

Tell me what happened.

Rule 3

Separate facts from narrative.

Facts:

Generated by software.

Narrative:

Generated by AI.

Rule 4

Store AI inputs.

Every explanation should preserve:

- Prompt version
- Model version
- Input summary

This enables auditing and debugging.

2.7.10 Prompt Versioning

Every AI-generated explanation must include:

`prompt_version`

Examples:

v1.0

v1.1

v2.0

Benefits:

- Reproducibility
- Quality tracking
- Easier experimentation
- Safer updates

Prompt versioning is mandatory.

2.7.11 Model Version Tracking

Every explanation must record:

model_name

Examples:

gpt-5.5

future_model

local_model_v1

Benefits:

- Performance comparison
 - Cost tracking
 - Migration planning
 - Quality analysis
-

2.7.12 AI Audit Requirements

The system must be able to answer:

Which model generated this?

Which prompt generated this?

Which data was supplied?

When was it generated?

Every explanation should therefore store:

- Model
 - Prompt version
 - Input summary
 - Timestamp
-

2.7.13 Failure Handling

AI failures must never prevent simulations from completing.

Example:

Simulation Success

AI Failure

Result:

Simulation returned successfully.

AI explanation unavailable.

Financial functionality takes priority over AI functionality.

2.7.14 Future RAG Architecture

Future versions may implement Retrieval-Augmented Generation (RAG).

Potential knowledge sources:

Historical Events

Economic Research

Investment Concepts

Platform Documentation

Educational Content

RAG should enhance educational quality without changing simulation results.

2.7.15 Model Provider Strategy

The architecture should remain provider-independent.

Potential providers:

OpenAI

Anthropic

Google

Local Models

Future Providers

The platform should communicate through an abstraction layer rather than directly depending on a specific vendor.

Benefits:

- Easier migration
 - Cost flexibility
 - Reduced vendor lock-in
-

2.7.16 Security Boundaries

AI services must never:

- Access production secrets
- Access database credentials
- Execute administrative actions
- Modify historical datasets
- Modify simulation results

AI should operate with the minimum permissions required.

2.7.17 Summary

The AI Architecture is built around a single rule:

AI explains financial facts. AI does not create financial facts.

Key principles:

- Deterministic calculations first
- AI explanations second
- Strict separation of responsibilities
- No investment advice
- No generated financial data
- Mandatory auditing
- Provider independence
- Failure isolation

The AI layer enhances understanding while preserving the platform's commitment to accuracy, transparency, and reproducibility.

Status: Part 2.7 Complete

Next: Part 2.8 — Security Architecture

Investment Time Machine

Founder Specification

Part 2.8 — Security Architecture

Status: Draft

2.8.1 Purpose

The Security Architecture defines how the platform protects:

- User accounts
- Historical datasets
- Simulation results
- Infrastructure
- Administrative systems

The objective is to build a secure platform without introducing unnecessary complexity during the MVP phase.

Security decisions should align with the platform's core principles:

- Accuracy
 - Trust
 - Reproducibility
 - Maintainability
-

2.8.2 Security Philosophy

Investment Time Machine is an educational platform, not a banking platform.

However, users must still trust:

- their accounts
- their data
- the integrity of simulation results

The system should follow:

Simple Security First

rather than:

Enterprise Complexity First

Security should be proportional to actual risk.

2.8.3 Security Goals

The platform must protect against:

Unauthorized Access

Examples:

- account takeover
- credential theft
- unauthorized admin access

Data Tampering

Examples:

- modified historical prices
- altered simulation results
- corrupted datasets

Infrastructure Exposure

Examples:

- exposed API keys
- leaked database credentials
- misconfigured deployments

Abuse

Examples:

- API abuse
- automated scraping
- denial-of-service attempts

2.8.4 Authentication Strategy

MVP Authentication

Supported:

Email + Password

Future:

Google OAuth

GitHub OAuth

Authentication should be handled by the application layer.

The database should only store credential hashes.

2.8.5 Password Security

Passwords must never be stored in plaintext.

Required:

Argon2

Acceptable fallback:

bcrypt

Stored value:

password_hash

Never store:

password

password_hint

2.8.6 Authorization Model

Standard User

Permissions:

- Create simulations
- View own history
- Generate AI reports

Administrator

Permissions:

- Manage assets
- Trigger data imports
- Access operational dashboards
- Review audit logs

Administrative access should be limited to a small number of trusted accounts.

2.8.7 Principle of Least Privilege

Every system component should operate with the minimum permissions required.

Examples:

AI Service

Allowed:

Read simulation outputs

Generate explanations

Not Allowed:

Modify prices

Modify simulations

Access credentials

Data Import Service

Allowed:

Insert historical data

Update market datasets

Not Allowed:

Manage users

Access admin controls

2.8.8 API Security

All production APIs must require:

HTTPS

Sensitive endpoints should require authentication.

Examples:

Create Simulation

User History

Admin Functions

Public endpoints should remain read-only.

2.8.9 Secret Management

Secrets must never be stored:

- In source code
- In Git repositories
- In documentation

Examples:

OPENAI_API_KEY

DATABASE_URL

JWT_SECRET

Secrets should be provided through environment variables.

2.8.10 Database Security

Database Access

Only backend services should communicate directly with PostgreSQL.

The frontend must never connect directly to the database.

Access Separation

Separate credentials should be used for:

Application Access

Administrative Access

Migration Access

This reduces blast radius during compromise.

2.8.11 Historical Data Protection

Historical market data is a core platform asset.

The platform must protect against:

- accidental deletion
- unauthorized modification
- corrupted imports

Recommended protections:

Foreign Keys

Constraints

Audit Logs

Import Validation

Historical truth must remain protected.

2.8.12 Input Validation

All user input must be validated before processing.

Examples:

Investment Amount

Must be greater than zero

Dates

Must be valid dates

Asset Symbols

Must exist in asset catalog

Validation should occur before simulation execution.

2.8.13 Rate Limiting

Rate limiting protects the platform from abuse.

Recommended MVP limits:

Simulation Endpoints

60 requests per minute

Authentication Endpoints

10 requests per minute

AI Endpoints

20 requests per minute

Values may be adjusted based on usage.

2.8.14 Logging and Monitoring

Security-relevant events should be logged.

Examples:

Login Attempts

Failed Authentication

Admin Actions

Data Import Failures

Permission Violations

Sensitive information must never appear in logs.

2.8.15 Backup Strategy

Critical data must be recoverable.

Required backups:

PostgreSQL Database

Historical Market Data

Migration History

Recommended MVP:

Daily Automated Backup

Backups should be tested periodically.

2.8.16 Dependency Security

The platform relies on third-party libraries.

Dependencies should be:

- regularly updated
- reviewed periodically
- removed when unused

Minimize unnecessary packages.

Every dependency increases attack surface.

2.8.17 Security Incident Response

If a security issue is detected:

1. Identify affected systems.
2. Stop further exposure.
3. Preserve logs.
4. Assess impact.
5. Apply remediation.
6. Document lessons learned.

Future versions may formalize incident procedures.

2.8.18 Future Security Enhancements

Potential future additions:

Multi-Factor Authentication

SSO

Advanced Role Management

Audit Dashboards

Security Alerts

IP Reputation Checks

These features are intentionally deferred until justified by scale.

2.8.19 Security Decisions Summary

Approved Decisions:

- HTTPS required in production
 - Password hashing required
 - Argon2 preferred
 - Principle of least privilege enforced
 - Frontend never accesses database directly
 - Secrets stored outside source code
 - Audit logging required
 - Input validation mandatory
 - Rate limiting enabled
 - Daily backups recommended
-

2.8.20 Summary

The Security Architecture prioritizes practical protection without introducing unnecessary complexity.

The platform should remain:

- Secure
- Maintainable
- Auditable
- Scalable

Security exists to protect trust, and trust is one of the platform's most important assets.

Status: Part 2.8 Complete

Next: Part 2.9 — API Architecture

Investment Time Machine

Founder Specification

Part 2.8 — Security Architecture

Status: Draft

2.8.1 Purpose

The Security Architecture defines how the platform protects:

- User accounts
- Historical datasets
- Simulation results
- Infrastructure
- Administrative systems

The objective is to build a secure platform without introducing unnecessary complexity during the MVP phase.

Security decisions should align with the platform's core principles:

- Accuracy
 - Trust
 - Reproducibility
 - Maintainability
-

2.8.2 Security Philosophy

Investment Time Machine is an educational platform, not a banking platform.

However, users must still trust:

- their accounts
- their data
- the integrity of simulation results

The system should follow:

Simple Security First

rather than:

Enterprise Complexity First

Security should be proportional to actual risk.

2.8.3 Security Goals

The platform must protect against:

Unauthorized Access

Examples:

- account takeover
- credential theft
- unauthorized admin access

Data Tampering

Examples:

- modified historical prices
- altered simulation results
- corrupted datasets

Infrastructure Exposure

Examples:

- exposed API keys
- leaked database credentials
- misconfigured deployments

Abuse

Examples:

- API abuse
- automated scraping
- denial-of-service attempts

2.8.4 Authentication Strategy

MVP Authentication

Supported:

Email + Password

Future:

Google OAuth

GitHub OAuth

Authentication should be handled by the application layer.

The database should only store credential hashes.

2.8.5 Password Security

Passwords must never be stored in plaintext.

Required:

Argon2

Acceptable fallback:

bcrypt

Stored value:

password_hash

Never store:

password

password_hint

security_answers

2.8.6 Authorization Model

Standard User

Permissions:

- Create simulations
- View own history
- Generate AI reports

Administrator

Permissions:

- Manage assets
- Trigger data imports
- Access operational dashboards
- Review audit logs

Administrative access should be limited to a small number of trusted accounts.

2.8.7 Principle of Least Privilege

Every system component should operate with the minimum permissions required.

Examples:

AI Service

Allowed:

Read simulation outputs
Generate explanations

Not Allowed:

Modify prices
Modify simulations
Access credentials

Data Import Service

Allowed:

Insert historical data

Update market datasets

Not Allowed:

Manage users

Access admin controls

2.8.8 API Security

All production APIs must require:

HTTPS

Sensitive endpoints should require authentication.

Examples:

Create Simulation

User History

Admin Functions

Public endpoints should remain read-only.

2.8.9 Secret Management

Secrets must never be stored:

- In source code
- In Git repositories
- In documentation

Examples:

OPENAI_API_KEY

DATABASE_URL

JWT_SECRET

Secrets should be provided through environment variables.

2.8.10 Database Security

Database Access

Only backend services should communicate directly with PostgreSQL.

The frontend must never connect directly to the database.

Access Separation

Separate credentials should be used for:

Application Access
Administrative Access
Migration Access

This reduces blast radius during compromise.

2.8.11 Historical Data Protection

Historical market data is a core platform asset.

The platform must protect against:

- accidental deletion
- unauthorized modification
- corrupted imports

Recommended protections:

Foreign Keys
Constraints
Audit Logs
Import Validation

Historical truth must remain protected.

2.8.12 Input Validation

All user input must be validated before processing.

Examples:

Investment Amount

Must be greater than zero

Dates

Must be valid dates

Asset Symbols

Must exist in asset catalog

Validation should occur before simulation execution.

2.8.13 Rate Limiting

Rate limiting protects the platform from abuse.

Recommended MVP limits:

Simulation Endpoints

60 requests per minute

Authentication Endpoints

10 requests per minute

AI Endpoints

20 requests per minute

Values may be adjusted based on usage.

2.8.14 Logging and Monitoring

Security-relevant events should be logged.

Examples:

Login Attempts
Failed Authentication
Admin Actions
Data Import Failures
Permission Violations

Sensitive information must never appear in logs.

2.8.15 Backup Strategy

Critical data must be recoverable.

Required backups:

PostgreSQL Database
Historical Market Data
Migration History

Recommended MVP:

Daily Automated Backup

Backups should be tested periodically.

2.8.16 Dependency Security

The platform relies on third-party libraries.

Dependencies should be:

- regularly updated
- reviewed periodically
- removed when unused

Minimize unnecessary packages.

Every dependency increases attack surface.

2.8.17 Security Incident Response

If a security issue is detected:

1. Identify affected systems.
2. Stop further exposure.
3. Preserve logs.
4. Assess impact.
5. Apply remediation.
6. Document lessons learned.

Future versions may formalize incident procedures.

2.8.18 Future Security Enhancements

Potential future additions:

Multi-Factor Authentication
SSO
Advanced Role Management
Audit Dashboards
Security Alerts
IP Reputation Checks

These features are intentionally deferred until justified by scale.

2.8.19 Security Decisions Summary

Approved Decisions:

- HTTPS required in production
 - Password hashing required
 - Argon2 preferred
 - Principle of least privilege enforced
 - Frontend never accesses database directly
 - Secrets stored outside source code
 - Audit logging required
 - Input validation mandatory
 - Rate limiting enabled
 - Daily backups recommended
-

2.8.20 Summary

The Security Architecture prioritizes practical protection without introducing unnecessary complexity.

The platform should remain:

- Secure
- Maintainable
- Auditable
- Scalable

Security exists to protect trust, and trust is one of the platform's most important assets.

Status: Part 2.8 Complete

Next: Part 2.9 — API Architecture

Investment Time Machine

Founder Specification

Part 2.10 — Infrastructure Architecture

Status: Draft

2.10.1 Purpose

The Infrastructure Architecture defines how the platform is hosted, deployed, operated, and scaled.

The infrastructure must support the project's core principles:

- Accuracy over aesthetics
- Backend before frontend
- Reproducibility
- Local data ownership
- Low operational cost
- Long-term maintainability

The MVP should prioritize simplicity and reliability over enterprise-scale complexity.

2.10.2 Infrastructure Philosophy

The platform should follow:

Simple Architecture First
Scale When Necessary

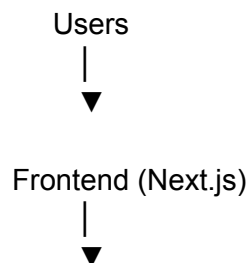
The goal is not to build infrastructure for millions of users before the product exists.

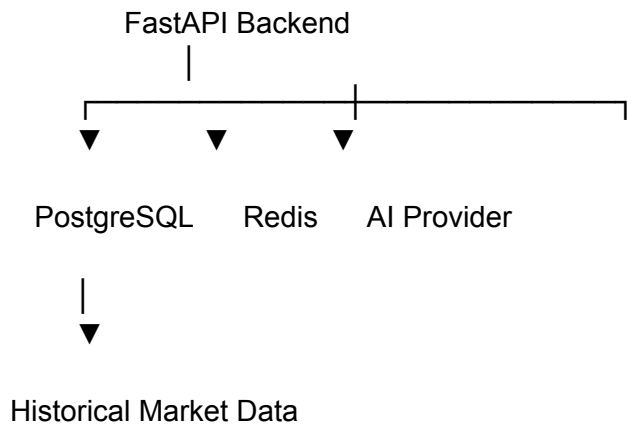
Infrastructure should evolve in response to:

- User growth
- Data growth
- Performance requirements

Premature infrastructure complexity is prohibited.

2.10.3 High-Level Infrastructure





This architecture satisfies MVP requirements while remaining easy to understand and operate.

2.10.4 Core Components

Component	Technology	Purpose
Frontend	Next.js	User interface
Backend	FastAPI	Business logic and APIs
Database	PostgreSQL	Persistent storage
Cache	Redis	Performance optimization
AI Layer	External LLM Provider	Educational explanations
Containerization	Docker	Deployment consistency
Source Control	GitHub	Version management

2.10.5 Hosting Strategy

Frontend

Recommended:

Vercel

Benefits:

- Excellent Next.js support
 - Free MVP tier
 - Simple deployments
 - Built-in CDN
-

Backend

Recommended:

Railway

Alternative:

Render

Benefits:

- Easy deployment
 - PostgreSQL integration
 - Low operational overhead
 - Suitable MVP pricing
-

Database

Recommended:

PostgreSQL

Hosted on:

Railway

or

Render

The database remains the platform's most important infrastructure component.

2.10.6 Environment Strategy

Three environments should exist.

Environment	Purpose
Development	Local development
Staging	Pre-production testing
Production	Live platform

Benefits:

- Safer deployments
- Better testing
- Reduced production risk

The MVP may initially combine Development and Staging if resources are limited.

2.10.7 Docker Standard

All major services should be containerized.

Required services:

frontend
backend
postgres
redis

Benefits:

- Consistent environments
- Easier onboarding
- Reproducible deployments
- Simpler migration between providers

Docker becomes the deployment standard for the platform.

2.10.8 Database Infrastructure

PostgreSQL is classified as critical infrastructure.

Requirements:

- Daily backups
- Migration management
- Connection pooling
- Access controls

The database should be treated as a protected asset.

Historical financial data should never depend exclusively on external providers.

2.10.9 Redis Infrastructure

Redis is optional during early MVP development but recommended.

Primary use cases:

- Asset lookup caching
- Historical price caching
- API response caching
- Rate limiting

Redis should never become the primary source of truth.

PostgreSQL remains authoritative.

2.10.10 Storage Strategy

The platform stores:

Structured Data

Stored in PostgreSQL:

assets
prices
dividends

splits
simulations
users

Generated Content

Stored in PostgreSQL:

ai_explanations
audit_logs

Static Assets

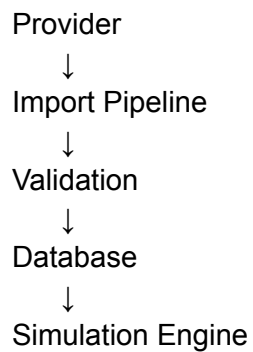
Stored through hosting provider infrastructure.

Examples:

logos
icons
images

2.10.11 Historical Data Infrastructure

Historical market data should follow:



The simulation engine should never rely on live provider calls.

Benefits:

- Reproducibility

- Faster simulations
 - Provider independence
-

2.10.12 Scaling Strategy

Scaling should occur in stages.

Stage 1

Single backend instance.

Single PostgreSQL instance.

Target:

0–1,000 users

Stage 2

Multiple backend instances.

Redis enabled.

Target:

1,000–10,000 users

Stage 3

Database optimization.

Read replicas.

Background processing.

Target:

10,000+ users

Infrastructure should scale only when supported by actual usage metrics.

2.10.13 Background Jobs

Certain tasks should eventually execute asynchronously.

Examples:

- Historical data imports
- AI explanation generation
- Bulk calculations
- Report generation

Potential future tools:

Celery
RQ
Dramatiq

Not required for initial MVP.

2.10.14 Vendor Independence

The architecture should avoid dependency on any single provider.

Examples:

Layer	Replaceable
AI Provider	Yes
Hosting Provider	Yes
Database Host	Yes
Cache Provider	Yes

Vendor lock-in should be minimized whenever practical.

2.10.15 Infrastructure Monitoring

Infrastructure should track:

- API uptime
- Database health
- Import failures
- Error rates
- Response times

The MVP may begin with basic provider monitoring before adopting dedicated observability tools.

2.10.16 Disaster Recovery

Critical assets:

Database
Historical Data
Migration History
Source Code

Recovery requirements:

- Automated backups
- Recovery procedures
- Source control protection

Historical financial data should always be recoverable.

2.10.17 Cost Strategy

Target MVP operating cost:

\$0–\$10/month

Preferred free-tier tools:

Category	Tool
Frontend	Vercel
Backend	Railway / Render
Database	Railway / Render
Source Control	GitHub
Containerization	Docker

Infrastructure decisions should favor sustainability during development.

2.10.18 Future Infrastructure Expansion

Potential future additions:

Kubernetes
Managed Redis
Read Replicas
Object Storage
CDN Optimization
Background Workers
Data Warehouse

These are intentionally deferred.

The MVP should remain operationally simple.

2.10.19 Summary

The Infrastructure Architecture prioritizes simplicity, reliability, and low operational cost.

Key decisions:

- Next.js frontend
- FastAPI backend

- PostgreSQL primary database
- Redis cache layer
- Docker standardization
- Railway/Render hosting
- Vercel frontend deployment
- Local ownership of historical data
- Provider independence

The infrastructure should remain lean until user growth justifies additional complexity.

Status: Part 2.10 Complete

Next: Part 2.11 — Deployment Architecture

Investment Time Machine

Founder Specification

Part 2.11 — Deployment Architecture

Status: Draft

2.11.1 Purpose

Deployment Architecture defines how code moves from development to production safely and repeatably. The deployment process must support reliable releases, controlled migrations, environment separation, and fast rollback when needed.

Core goals:

- Reproducible deployments
- Minimal manual steps
- Safe database migrations
- Clear environment separation
- Low-cost MVP operation

2.11.2 Deployment Philosophy

Deployments should be boring, predictable, and reversible. The platform should avoid complex release systems until user scale requires them.

Key principles:

Principle	Meaning
Automate repeatable steps	Manual deployment should be avoided where practical
Keep environments consistent	Development, staging, and production should use similar configuration
Separate code from configuration	Secrets and environment-specific values must not be committed
Migrate carefully	Database changes require versioned migrations
Prefer simple rollback	A failed release should be recoverable quickly

2.11.3 Source Control Strategy

GitHub is the authoritative source for application code, infrastructure files, documentation, and migration history.

Recommended branches:

Branch	Purpose
main	Production-ready code
develop	Integration branch for upcoming work
feature/*	Individual feature development
hotfix/*	Urgent production fixes

MVP may use a simpler `main` + `feature/*` workflow until the project requires more structure.

2.11.4 Deployment Environments

Environment	Purpose	Deployment Trigger
Development	Local engineering and testing	Manual
Staging	Pre-production validation	Merge to develop or manual trigger
Production	Live user-facing platform	Merge to main or tagged release

Staging should mirror production configuration as closely as possible while using separate databases, secrets, and API keys.

2.11.5 Frontend Deployment

Frontend deployment should use Vercel for the Next.js application.

Deployment behavior:

Action	Result
Pull request opened	Preview deployment
Merge to main	Production deployment
Environment variables updated	Rebuild if required

Benefits:

- Native Next.js support
- Automatic preview URLs
- Simple rollback
- Free-tier friendly

2.11.6 Backend Deployment

Backend deployment should use Railway or Render for the FastAPI service.

Deployment requirements:

- Build from GitHub repository
- Run through Docker when practical
- Load environment variables from provider settings
- Expose only required public endpoints
- Restart safely after deployment

Backend deployment must never require manually copying code to servers.

2.11.7 Database Migration Deployment

Schema changes must use Alembic migrations and run as part of a controlled deployment process.

Required migration process:

1. Generate migration locally.
2. Review migration for correctness.
3. Commit migration file to GitHub.
4. Apply migration to staging.
5. Validate staging behavior.
6. Apply migration to production.
7. Monitor application and database health.

Dangerous migrations, such as dropping columns or rewriting large tables, require explicit review and backup confirmation before production execution.

2.11.8 Environment Variable Strategy

All environment-specific configuration must live outside source code.

Common variables:

Variable	Purpose
DATABASE_URL	PostgreSQL connection
REDIS_URL	Redis connection
JWT_SECRET	Token signing
AI_PROVIDER_API_KEY	AI provider access
ENVIRONMENT	development, staging, production
CORS_ALLOWED_ORIGINS	Allowed frontend origins

Secrets must never be committed to GitHub, screenshots, documentation examples, or logs.

2.11.9 Release Process

Recommended MVP release flow:

feature branch

- > pull request
- > automated checks
- > staging deployment
- > manual review

- > production deployment
- > post-deploy monitoring

The process should remain lightweight while still protecting production data and user trust.

2.11.10 Automated Checks

Before production deployment, the system should run:

- Backend tests
- Frontend build checks
- Type checks
- Linting
- Database migration checks
- Basic API smoke tests

MVP may begin with backend tests and frontend build checks, then expand automation over time.

2.11.11 Rollback Strategy

Rollback must be planned before deployment.

Rollback options:

Layer	Rollback Method
Frontend	Revert to previous Vercel deployment
Backend	Redeploy previous commit
Database	Use backup restore or corrective migration
Configuration	Revert environment variable changes

Database rollback is the hardest category and should be treated carefully. Forward corrective migrations are often safer than attempting to reverse complex production schema changes.

2.11.12 Deployment Safety Rules

Production deployments must follow these rules:

- Never deploy unreviewed migrations.
- Never run destructive migrations without backup confirmation.
- Never expose secrets through build logs.
- Never deploy directly from an untracked local machine.
- Never mix staging and production databases.
- Never allow frontend code to connect directly to PostgreSQL.

2.11.13 Data Import Deployment

Market data import jobs should be deployed separately from user-facing API behavior when possible.

Data import changes require validation because incorrect imports can corrupt the platform's historical truth.

Required import safeguards:

- Validate record counts.
- Validate date ranges.
- Validate null values.
- Log import source.
- Record import failures.
- Preserve audit trails.

2.11.14 Deployment Monitoring

After each production deployment, monitor:

- API health
- Error rate
- Database connectivity
- Simulation success rate
- Authentication failures
- Data import failures
- AI explanation failures

A deployment is not complete until the system is confirmed stable.

2.11.15 MVP Deployment Stack

Layer	Recommended Tool
Source Control	GitHub

Frontend Hosting	Vercel
Backend Hosting	Railway or Render
Database Hosting	Railway or Render PostgreSQL
Containerization	Docker
Migrations	Alembic
CI/CD	GitHub Actions

This stack supports low-cost development while remaining professional and recruiter-visible.

2.11.16 Future Deployment Expansion

Future additions may include blue-green deployments, canary releases, infrastructure-as-code, managed secrets, background worker deployment pipelines, and dedicated observability tooling.

These are intentionally deferred until the platform has enough traffic, operational complexity, or team size to justify them.

2.11.17 Summary

Deployment Architecture ensures that Investment Time Machine can ship safely without unnecessary operational complexity.

Approved decisions:

- GitHub is the source of truth.
- Vercel hosts the frontend.
- Railway or Render hosts the backend and database.
- Alembic manages migrations.
- Environment variables manage secrets.
- Staging validation happens before production.
- Rollback planning is required for production releases.
- Deployment monitoring is mandatory.

Status: Part 2.11 Complete

Next: Part 2.12 — Monitoring and Observability Architecture

Investment Time Machine

Founder Specification

Part 2.12 — Monitoring and Observability Architecture

Status: Draft

2.12.1 Purpose

Monitoring and Observability ensure the platform remains reliable, debuggable, and maintainable as it grows.

The system must provide visibility into:

- Application health
- Database health
- Simulation performance
- Data imports
- AI services
- User-facing failures

The goal is to identify issues before they significantly impact users.

2.12.2 Observability Philosophy

The platform should always be able to answer:

- Is the system healthy?
- What failed?
- When did it fail?
- Why did it fail?
- Who was affected?
- How often is it happening?

Observability is not a luxury feature. It is a core operational requirement.

2.12.3 Monitoring Categories

The platform should monitor five primary areas:

Category	Purpose
Infrastructure	Servers, containers, databases
Application	API performance and errors
Data Pipeline	Historical data imports
Simulation Engine	Calculation performance
AI Services	Explanation generation health

2.12.4 Infrastructure Monitoring

The system should track:

Metric	Purpose
CPU Usage	Resource consumption
Memory Usage	Capacity planning
Disk Usage	Storage health
Network Traffic	Connectivity analysis
Container Health	Service availability

Alert thresholds should be established as the platform grows.

2.12.5 API Monitoring

The backend API should track:

Metric	Purpose
Request Count	Traffic analysis

Response Time	Performance measurement
Error Rate	Reliability monitoring
Success Rate	Service quality
Endpoint Usage	Product insights

Critical endpoints include:

/api/v1/simulations
/api/v1/assets
/api/v1/auth
/api/v1/ai

2.12.6 Database Monitoring

PostgreSQL should track:

Metric	Purpose
Active Connections	Capacity monitoring
Query Latency	Performance analysis
Slow Queries	Optimization targets
Database Size	Growth tracking
Backup Status	Recovery readiness

The database is a critical dependency and requires continuous visibility.

2.12.7 Simulation Monitoring

The Simulation Engine should record:

Metric	Purpose
Simulations Executed	Usage measurement
Average Runtime	Performance monitoring
Failure Rate	Reliability monitoring
Most Simulated Assets	Product insights
Calculation Errors	Debugging support

Simulation correctness remains more important than simulation speed.

2.12.8 Historical Data Monitoring

Data import systems should track:

Metric	Purpose
Import Success Rate	Reliability
Records Imported	Volume tracking
Failed Imports	Error detection
Import Duration	Performance
Provider Availability	Dependency health

Import failures must generate audit records.

2.12.9 AI Monitoring

The AI service should track:

Metric	Purpose
--------	---------

Explanation Requests	Usage analysis
Generation Success Rate	Reliability
Generation Time	Performance
Token Usage	Cost tracking
Model Distribution	Provider analysis

AI failures should not affect simulation availability.

2.12.10 Logging Standards

Application logs should be structured and searchable.

Every significant event should include:

timestamp
service_name
event_type
severity
request_id

Structured logs simplify debugging and future analytics.

2.12.11 Log Levels

The platform should standardize log severity.

Level	Usage
DEBUG	Development diagnostics
INFO	Normal operations
WARNING	Recoverable issues
ERROR	Failed operations

CRITICAL Service-threatening
failures

Consistent logging improves operational visibility.

2.12.12 Error Tracking

Errors should capture:

- Error type
- Error message
- Request context
- Timestamp
- Service origin

Errors must be actionable.

Avoid generic messages such as:

Something went wrong.

Prefer:

Simulation failed due to missing historical price data for the requested date range.

2.12.13 Health Checks

Every service should expose a health endpoint.

Example:

GET /health

Typical response:

```
{  
  "status": "healthy"  
}
```

Health checks support:

- Monitoring
 - Deployment validation
 - Automated recovery
-

2.12.14 Alerting Strategy

Alerts should be triggered for:

- API downtime
- Database outages
- Failed imports
- High error rates
- Backup failures
- Authentication anomalies

Alert fatigue should be avoided.

Only actionable issues should generate alerts.

2.12.15 Audit Integration

Monitoring and audit systems should complement each other.

System	Purpose
Monitoring	Operational health
Audit Logs	Historical traceability

Monitoring answers:

What is failing now?

Audit logs answer:

What happened previously?

2.12.16 MVP Monitoring Stack

Recommended MVP tools:

Need	Tool
Application Logs	Platform Logs
Error Tracking	Sentry
Uptime Monitoring	UptimeRobot
Metrics	Hosting Provider Metrics
Auditability	PostgreSQL Audit Tables

The MVP should leverage existing provider capabilities before introducing complex observability platforms.

2.12.17 Performance Targets

Initial targets:

Metric	Target
API Response Time	< 500ms
Simulation Execution	< 2 seconds
Asset Search	< 250ms
Database Query Time	< 100ms
Uptime	99%+

These targets may evolve as the platform grows.

2.12.18 Future Expansion

Potential future additions:

Prometheus
Grafana
OpenTelemetry
Distributed Tracing
Custom Dashboards
Business Intelligence Metrics

These should be introduced only when operational complexity justifies them.

2.12.19 Summary

Monitoring and Observability provide visibility into system health, performance, reliability, and operational behavior.

Approved decisions:

- Health checks required
- Structured logging required
- Error tracking required
- Database monitoring required
- Import monitoring required
- AI monitoring required
- Audit logs remain separate from operational monitoring
- Sentry and provider monitoring are sufficient for MVP

A system that cannot be observed cannot be reliably operated.

Status: Part 2.12 Complete

Next: Part 2.13 — Data Ingestion Architecture

Investment Time Machine

Founder Specification

Part 2.13 — Data Ingestion Architecture

Status: Draft

2.13.1 Purpose

The Data Ingestion Architecture defines how historical market data enters the platform.

Because historical data is the foundation of every simulation, the ingestion system must prioritize:

- Accuracy
- Validation
- Traceability
- Reproducibility
- Provider independence

A bad import can corrupt thousands of simulations. Therefore, data ingestion is considered a critical platform function.

2.13.2 Ingestion Philosophy

The platform follows a simple rule:

Never trust external data until it has been validated.

External providers are sources of information, not sources of truth.

Every imported dataset must pass through:

Provider

→ Validation

→ Normalization

→ Storage

→ Simulation Engine

Simulation calculations must only use validated data stored locally.

2.13.3 Supported Data Sources

MVP Sources

Provider	Purpose
yfinance	Stocks and ETFs
CoinGecko	Cryptocurrency data
FRED	Economic indicators

These providers are selected because they are widely used, free, and sufficient for MVP requirements.

2.13.4 Ingestion Components

The ingestion system consists of four layers.

Layer	Responsibility
Provider Layer	Retrieve external data
Validation Layer	Verify correctness
Transformation Layer	Normalize format
Storage Layer	Persist validated data

Each layer should remain independent.

2.13.5 Import Workflow

Standard import process:

External Provider



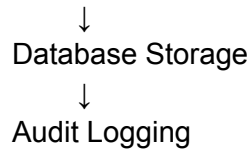
Data Retrieval



Validation



Normalization

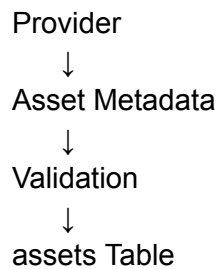


If validation fails, data must not be stored.

2.13.6 Asset Discovery

The platform must maintain an asset catalog.

Asset discovery process:



Examples:

AAPL
MSFT
SPY
BTC
ETH

Assets should be imported before price history.

2.13.7 Historical Price Imports

Historical price imports are the most important ingestion workflow.

Imported fields:

Field

Open

High

Low

Close

Adjusted Close

Volume

Date

Validation requirements:

- Date exists
- Prices are non-negative
- $\text{High} \geq \text{Low}$
- $\text{Volume} \geq 0$
- Asset exists

Invalid records must be rejected.

2.13.8 Dividend Imports

Imported fields:

Field

Ex-Dividend Date

Payment Date

Dividend Amount

Currency

Validation requirements:

- Asset exists
- $\text{Amount} \geq 0$

- Date is valid

Dividend records must remain separate from price records.

2.13.9 Stock Split Imports

Imported fields:

Field

Split Date

Split Ratio

Validation requirements:

- Asset exists
- Ratio > 0
- Date is valid

Split events should be preserved exactly as reported by the provider.

2.13.10 Economic Data Imports

Supported indicators:

- CPI
- Inflation Rate
- GDP
- Unemployment Rate
- Federal Funds Rate

Validation requirements:

- Indicator exists
- Value is numeric
- Date is valid

Economic indicators should be stored separately from market assets.

2.13.11 Validation Rules

Every import must satisfy:

Rule	Purpose
Required Fields Present	Prevent incomplete records
Correct Data Types	Prevent malformed data
Constraint Validation	Enforce database integrity
Duplicate Detection	Prevent duplicate imports
Source Attribution	Preserve traceability

No data bypasses validation.

2.13.12 Normalization Standards

Different providers use different formats.

The normalization layer converts provider-specific formats into platform standards.

Examples:

Provider Format	Platform Format
BTC-USD	BTC
DateTime	DATE
Provider Symbol	Asset ID

The database should never depend on provider-specific schemas.

2.13.13 Duplicate Prevention

Duplicate records are prohibited.

Examples:

```
UNIQUE(asset_id, price_date, data_source)
UNIQUE(asset_id, ex_dividend_date, dividend_amount)
```

The ingestion layer should detect duplicates before insertion whenever possible.

2.13.14 Data Source Attribution

Every imported record must contain:

data_source

Examples:

```
yfinance
coingecko
fred
manual_import
```

Source attribution is mandatory for auditing and troubleshooting.

2.13.15 Import Scheduling

Recommended MVP schedule:

Dataset	Frequency
Asset Metadata	Daily
Historical Prices	Daily
Dividends	Daily
Splits	Daily
Economic Indicators	Daily

Schedules may be adjusted based on provider update frequency.

2.13.16 Failure Handling

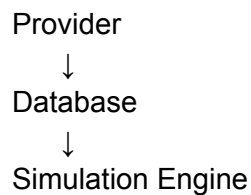
Import failures must:

1. Stop invalid data from entering the database.
2. Generate audit log entries.
3. Preserve error details.
4. Notify administrators when appropriate.

The platform should prefer missing data over corrupted data.

2.13.17 Historical Data Ownership

After successful import:



The simulation engine should never depend on live provider requests.

Benefits:

- Faster calculations
 - Reproducibility
 - Provider independence
 - Reduced outage risk
-

2.13.18 Reimport Strategy

The system must support reimporting historical datasets.

Reasons:

- Provider corrections
- Missing records
- Data quality improvements
- New assets

Reimports should be traceable through audit logs.

2.13.19 Future Expansion

Potential future ingestion sources:

Alpha Vantage
Polygon
IEX Cloud
SEC Filings
Alternative Data Providers

Potential future datasets:

Earnings Data
Options Data
Bond Data
Commodity Data
Forex Data

The ingestion architecture should support new providers without redesigning the platform.

2.13.20 Summary

The Data Ingestion Architecture ensures that historical data enters the platform through a controlled, validated, and auditable process.

Approved decisions:

- Local ownership of historical data
- Validation before storage
- Provider-independent schemas
- Duplicate prevention
- Source attribution

- Audit logging
- Daily import schedules
- No direct simulation dependence on external APIs

Historical data is the foundation of the platform, and the ingestion pipeline is responsible for protecting its integrity.

Status: Part 2.13 Complete

Next: Part 2.14 — Simulation Engine Architecture

Investment Time Machine

Founder Specification

Part 2.14 — Simulation Engine Architecture

Status: Draft

2.14.1 Purpose

The Simulation Engine is the core of the platform.

Every user-facing investment result originates from this component.

Its responsibility is to transform validated historical data into deterministic investment outcomes.

The Simulation Engine is the platform's most important business system and must prioritize:

- Accuracy
- Reproducibility
- Auditability
- Performance
- Explainability

2.14.2 Core Principle

The Simulation Engine is the sole source of financial truth.

No other system may calculate:

- Investment returns
- CAGR
- Dividend reinvestment
- Inflation-adjusted returns
- Portfolio performance

In particular:

Component	May Calculate Financial Results
Simulation Engine	Yes
AI Service	No
Frontend	No
Database	No
Data Ingestion System	No

This separation is mandatory.

2.14.3 Responsibilities

The Simulation Engine is responsible for:

Investment Growth Calculations

Example:

\$1,000 invested in AAPL from 2015 to 2025

Dividend Reinvestment

Example:

Reinvest all dividends during simulation period

Inflation Adjustment

Example:

Convert nominal returns into real returns

Portfolio Performance

Future versions will support:

Multiple assets

Rebalancing

Asset allocation analysis

Risk Metrics

Future versions may support:

Volatility

Drawdowns

Sharpe Ratio

Risk-adjusted returns

2.14.4 Simulation Workflow

Standard simulation flow:

User Request



Input Validation



Historical Data Retrieval



Simulation Calculation



Result Generation



Storage

↓
Optional AI Explanation

The AI layer always executes after calculations complete.

2.14.5 Input Requirements

Required inputs:

Field	Required
Asset	Yes
Investment Amount	Yes
Start Date	Yes
End Date	Yes

Optional inputs:

Field	Default
Include Dividends	False
Inflation Adjustment	False

All inputs must be validated before execution.

2.14.6 Historical Data Requirements

The engine may only use:

- Validated historical prices
- Validated dividend records
- Validated split records
- Validated economic indicators

The engine must never:

- Query live market data
- Generate missing prices
- Estimate missing returns

Missing data should result in a controlled error rather than a fabricated result.

2.14.7 Basic Investment Simulation

Formula:

$$\text{shares_purchased} = \text{investment_amount} / \text{initial_price}$$
$$\text{final_value} = \text{shares_purchased} \times \text{final_price}$$

Example:

Investment Amount: \$1,000

Initial Price: \$100

Shares Purchased: 10

Final Price: \$250

Final Value: \$2,500

This calculation forms the foundation of all future simulations.

2.14.8 Total Return Calculation

Formula:

$$\begin{aligned} \text{total_return_percent} = & \\ & ((\text{final_value} - \text{investment_amount}) \\ & / \text{investment_amount}) \times 100 \end{aligned}$$

Example:

Initial Investment: \$1,000

Final Value: \$2,500

Total Return: 150%

All return calculations must be deterministic.

2.14.9 CAGR Calculation

Formula:

$$\text{CAGR} = \left(\frac{\text{final_value}}{\text{investment_amount}} \right)^{1 / \text{years}} - 1$$

Example:

Initial Investment: \$1,000

Final Value: \$2,500

Period: 10 Years

The engine must preserve intermediate precision during calculations.

2.14.10 Dividend Reinvestment

When enabled:

1. Retrieve dividend events.
2. Calculate dividend cash received.
3. Purchase additional shares.
4. Update share count.
5. Continue simulation.

When disabled:

- Ignore dividend events.
- Use price appreciation only.

Dividend logic must remain transparent and auditable.

2.14.11 Inflation Adjustment

Inflation-adjusted returns require:

CPI

Inflation Rate

from economic indicator datasets.

Example:

Nominal Value: \$10,000

Inflation-Adjusted Value:
\$7,900

The platform should clearly distinguish:

Type	Meaning
Nominal Return	Market return
Real Return	Inflation-adjusted return

2.14.12 Determinism Requirements

Given identical:

- Inputs
- Historical data
- Calculation version

the engine must always produce identical outputs.

This requirement is non-negotiable.

Determinism is essential for:

- Trust
 - Debugging
 - Auditing
 - Testing
-

2.14.13 Calculation Versioning

Future simulations should store:

calculation_version

Example:

v1.0
v1.1
v2.0

Benefits:

- Reproducibility
- Backward compatibility
- Safer engine improvements

MVP may defer implementation but should reserve support for it.

2.14.14 Error Handling

Simulation failures should return structured errors.

Examples:

Error	Cause
Asset Not Found	Invalid symbol
Missing Price Data	Incomplete dataset
Invalid Date Range	End date before start date

Calculation Failure Internal processing issue

Errors should be explicit and actionable.

2.14.15 Performance Targets

Initial performance goals:

Operation	Target
Single Asset Simulation	< 2 seconds
Asset Lookup	< 250 ms
Historical Data Retrieval	< 500 ms

Correctness remains more important than speed.

2.14.16 Auditability Requirements

Every simulation should preserve:

- Original inputs
- Calculation assumptions
- Data source references
- Output values
- Execution timestamp

The platform must always be able to answer:

How was this result produced?

2.14.17 Future Simulation Types

Planned future support:

Dollar-Cost Averaging
Portfolio Simulations
Retirement Simulations
Tax-Aware Simulations
Risk Analysis
Scenario Analysis

The architecture should support these additions without redesigning the engine.

2.14.18 Testing Requirements

The Simulation Engine requires the highest test coverage in the platform.

Critical areas:

- Return calculations
- CAGR calculations
- Dividend logic
- Inflation adjustments
- Edge cases
- Error handling

Financial correctness must be continuously verified.

2.14.19 Summary

The Simulation Engine is the authoritative source of investment calculations.

Approved decisions:

- Deterministic calculations only
- No AI-generated financial results
- Local historical data only
- Dividend reinvestment support
- Inflation adjustment support
- Full auditability
- Structured error handling
- Future-ready architecture

The Simulation Engine is the heart of Investment Time Machine and the primary source of platform trust.

Status: Part 2.14 Complete

Next: Part 2.15 — Frontend Architecture

Investment Time Machine

Founder Specification

Part 2.15 — Frontend Architecture

Status: Draft

2.15.1 Purpose

The Frontend Architecture defines how users interact with the platform.

The frontend is responsible for presenting simulations, historical data, educational content, and analytics in a clear and intuitive manner.

The frontend does not perform business logic or financial calculations.

Its responsibilities are:

- User interaction
- Data visualization
- API communication
- State management
- User experience

All financial calculations remain in backend services.

2.15.2 Frontend Philosophy

The frontend should prioritize:

1. Understanding over aesthetics
2. Simplicity over complexity

3. Clarity over visual effects
4. Educational value over entertainment

The goal is not to build a trading terminal.

The goal is to help users understand historical investment outcomes.

2.15.3 Technology Stack

Component	Technology
Framework	Next.js
Language	TypeScript
Styling	Tailwind CSS
Charts	Recharts
API Communication	Fetch / Axios
Authentication	JWT
Deployment	Vercel

This stack aligns with the project's free-tier and maintainability goals.

2.15.4 Frontend Responsibilities

The frontend may:

- Display simulation results
- Display charts
- Collect user inputs
- Display AI explanations
- Manage authentication state
- Display historical data

The frontend may not:

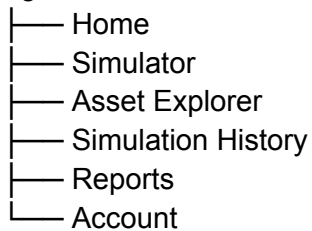
- Calculate returns
- Calculate CAGR

- Calculate dividends
- Modify financial results
- Access databases directly

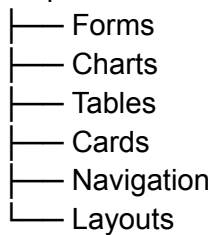
The frontend is a presentation layer only.

2.15.5 High-Level Structure

Pages



Components



This structure should remain modular and scalable.

2.15.6 Core Pages

Home Page

Purpose:

- Explain the product
- Showcase examples
- Guide users into simulations

Primary goal:

Convert visitors into active users

Simulator Page

Purpose:

Allow users to create historical investment simulations.

Inputs:

- Asset
- Investment amount
- Start date
- End date
- Dividend option
- Inflation option

This is the platform's most important page.

Asset Explorer

Purpose:

Allow users to discover supported assets.

Features:

- Search
 - Filtering
 - Asset metadata
 - Historical overview
-

Simulation History

Purpose:

Display previously created simulations.

Features:

- History
- Search

- Filtering
- Re-run simulations

Authentication required.

Reports Page

Purpose:

Display educational reports and AI-generated explanations.

Reports should clearly distinguish:

Content Type	Source
Financial Results	Simulation Engine
Educational Commentary	AI Service

2.15.7 Layout Architecture

Recommended layout:

Navigation



Main Content



Footer

The interface should remain consistent across pages.

2.15.8 Component Architecture

Components should be reusable.

Examples:

AssetSearch
SimulationForm
PerformanceChart
ReturnCard
MetricCard
AIExplanation

Business logic should never be embedded inside UI components.

2.15.9 State Management

MVP recommendation:

React State
Context API

Avoid introducing global state libraries until complexity requires them.

Potential future additions:

Redux
Zustand

These are intentionally deferred.

2.15.10 API Communication

Frontend communication should occur exclusively through backend APIs.

Frontend
↓
FastAPI
↓
Database

Direct database access is prohibited.

Benefits:

- Security
 - Maintainability
 - Consistency
-

2.15.11 Data Visualization

Charts are a core educational feature.

Primary chart types:

Chart	Purpose
Portfolio Growth	Investment performance
Asset Comparison	Relative performance
Inflation Comparison	Real vs nominal returns
Historical Timeline	Market context

Charts should support understanding rather than visual complexity.

2.15.12 Loading and Error States

Every page should handle:

Loading

Example:

Loading simulation...

Empty States

Example:

No simulations found.

Errors

Example:

Unable to retrieve historical data.

Error messages should be clear and actionable.

2.15.13 Responsive Design

The platform should support:

Device	Required
Desktop	Yes
Tablet	Yes
Mobile	Yes

Desktop remains the primary target for MVP development.

Mobile support should remain functional but does not require feature parity immediately.

2.15.14 Authentication Experience

Authenticated users should access:

- Saved simulations
- History
- Future portfolio features

Unauthenticated users may still use public simulation features if enabled.

The frontend should gracefully support both modes.

2.15.15 Accessibility Requirements

The platform should follow basic accessibility standards.

Requirements:

- Semantic HTML
- Keyboard navigation
- Accessible forms
- Meaningful labels
- Color-independent information

Accessibility should be built into components rather than added later.

2.15.16 Performance Targets

Initial goals:

Metric	Target
First Page Load	< 3 seconds
Asset Search Response	< 500 ms
Chart Rendering	< 1 second
Simulation Result Display	< 1 second after API response

Performance should support smooth educational exploration.

2.15.17 Future Frontend Expansion

Potential future features:

Portfolio Dashboard
Watchlists
Retirement Planner
Research Workspace
AI Assistant

The component architecture should support these additions without major redesign.

2.15.18 Summary

The Frontend Architecture exists to present historical investment information in a clear, educational, and intuitive manner.

Approved decisions:

- Next.js frontend
- TypeScript
- Tailwind CSS
- Recharts
- Backend-driven calculations
- API-only data access
- Responsive design
- Reusable components
- Accessibility support
- Educational-first user experience

The frontend should help users understand historical investment outcomes while keeping financial logic centralized in the backend.

Status: Part 2.15 Complete

Next: Part 2.16 — Product Roadmap Architecture

Investment Time Machine

Founder Specification

Part 2.16 — Product Roadmap Architecture

Status: Draft

2.16.1 Purpose

The Product Roadmap Architecture defines how the platform evolves from MVP to long-term vision.

The roadmap exists to:

- Prioritize development
- Prevent feature creep
- Align engineering effort with product goals
- Maintain focus on educational value

Every roadmap decision must align with the core principles defined in Part 1.

2.16.2 Roadmap Philosophy

The platform should be built in layers.

Each version should provide standalone value while creating a foundation for future capabilities.

Development should follow:

Foundation

→ Validation

→ Expansion

→ Intelligence

→ Platform

Features should only be added when they improve:

- Understanding
 - Transparency
 - Accuracy
 - Educational value
-

2.16.3 Version 1 — Historical Investment Simulator

Goal

Prove that users find value in historical investment exploration.

Core Features

- Asset search
- Historical investment simulations
- Dividend reinvestment
- Inflation adjustment
- AI explanations
- Historical asset comparison

Supported Assets

- Stocks
- ETFs
- Cryptocurrencies

Success Criteria

Users can answer:

What would have happened if I invested in this asset?

This version establishes the platform's core identity.

2.16.4 Version 2 — Portfolio Analysis Platform

Goal

Expand from single-asset simulations to portfolio-level analysis.

Core Features

- Portfolio creation
- Portfolio backtesting
- Allocation analysis
- Rebalancing simulations
- Portfolio comparison

Example Questions

Would diversification have improved my outcome?

What portfolio would have performed best?

Success Criteria

Users can evaluate historical portfolio decisions.

2.16.5 Version 3 — Research and Intelligence Layer

Goal

Transform simulations into deeper investment research.

Core Features

- Research workspaces
- Saved reports
- Historical event timelines
- Advanced AI explanations
- Economic context integration

Example Questions

Why did this investment outperform?

What historical events influenced these results?

Success Criteria

Users gain context, not just numbers.

2.16.6 Version 4 — Market Intelligence Platform

Goal

Provide broad historical market exploration capabilities.

Core Features

- Cross-asset analysis

- Sector analysis
- Market trend exploration
- Historical screening tools
- Economic regime analysis

Example Questions

How did technology stocks behave during recessions?

Which sectors performed best during inflationary periods?

Success Criteria

Users can explore market history at scale.

2.16.7 Version 5 — Financial Education Ecosystem

Goal

Become the leading educational platform for historical investing.

Core Features

- Interactive learning modules
- Guided investment lessons
- AI teaching assistant
- Educational pathways
- Scenario-based learning

Example Questions

How does compounding work?

How does diversification reduce risk?

Success Criteria

Users improve financial understanding through structured learning.

2.16.8 Feature Prioritization Framework

Every proposed feature should be evaluated against:

Question	Required
Improves understanding?	Yes
Improves transparency?	Preferred
Improves accuracy?	Preferred
Improves educational value?	Yes
Supports core mission?	Yes

Features failing these criteria should not be prioritized.

2.16.9 Features Explicitly Excluded

The roadmap intentionally excludes:

Brokerage integrations
Real-money trading
Investment recommendations
Copy trading
Social investing
Market predictions
Speculative forecasting

These features do not align with the platform's educational mission.

2.16.10 Development Order

Major development priorities:

Phase 1

Data

Simulation Engine
Database
API

Phase 2

Frontend
Visualization
User Experience

Phase 3

AI Layer
Research Features
Advanced Analysis

Backend functionality must exist before frontend enhancements.

2.16.11 Milestone Structure

Milestone 1

Historical data successfully stored.

Milestone 2

Simulation engine produces accurate results.

Milestone 3

Public API operational.

Milestone 4

Frontend simulator functional.

Milestone 5

AI explanations integrated.

Milestone 6

Public MVP release.

Each milestone should produce a measurable improvement.

2.16.12 Roadmap Risks

Common risks include:

Risk	Impact
Feature Creep	Delayed delivery
Overengineering	Slower progress
Data Quality Issues	Reduced trust
AI Hallucinations	User confusion
Premature Scaling	Increased costs

The roadmap should favor simplicity whenever possible.

2.16.13 Long-Term Vision Alignment

Every roadmap stage should move the platform toward:

The world's most educational historical investment simulator and market intelligence platform.

The roadmap should expand depth before breadth.

Better simulations are more valuable than more features.

2.16.14 Success Metrics by Version

Version	Primary Success Metric
V1	Accurate historical simulations
V2	Portfolio analysis adoption
V3	Research feature engagement
V4	Market intelligence usage
V5	Educational outcomes and retention

Each version should have a clearly defined objective.

2.16.15 Summary

The Product Roadmap Architecture defines a staged path from a focused historical simulator to a comprehensive investment education platform.

Approved roadmap sequence:

V1 Historical Simulator
V2 Portfolio Analysis
V3 Research Platform
V4 Market Intelligence
V5 Education Ecosystem

The roadmap prioritizes educational value, historical accuracy, and engineering quality over feature volume.

Status: Part 2.16 Complete

Next: Part 2.17 — MVP Scope Definition

Investment Time Machine

Founder Specification

Part 2.16 — Product Roadmap Architecture

Status: Draft

2.16.1 Purpose

The Product Roadmap Architecture defines how the platform evolves from MVP to long-term vision.

The roadmap exists to:

- Prioritize development
- Prevent feature creep
- Align engineering effort with product goals
- Maintain focus on educational value

Every roadmap decision must align with the core principles defined in Part 1.

2.16.2 Roadmap Philosophy

The platform should be built in layers.

Each version should provide standalone value while creating a foundation for future capabilities.

Development should follow:

Foundation

→ Validation

→ Expansion

→ Intelligence

→ Platform

Features should only be added when they improve:

- Understanding
 - Transparency
 - Accuracy
 - Educational value
-

2.16.3 Version 1 — Historical Investment Simulator

Goal

Prove that users find value in historical investment exploration.

Core Features

- Asset search
- Historical investment simulations
- Dividend reinvestment
- Inflation adjustment
- AI explanations
- Historical asset comparison

Supported Assets

- Stocks
- ETFs
- Cryptocurrencies

Success Criteria

Users can answer:

What would have happened if I invested in this asset?

This version establishes the platform's core identity.

2.16.4 Version 2 — Portfolio Analysis Platform

Goal

Expand from single-asset simulations to portfolio-level analysis.

Core Features

- Portfolio creation
- Portfolio backtesting
- Allocation analysis
- Rebalancing simulations
- Portfolio comparison

Example Questions

Would diversification have improved my outcome?

What portfolio would have performed best?

Success Criteria

Users can evaluate historical portfolio decisions.

2.16.5 Version 3 — Research and Intelligence Layer

Goal

Transform simulations into deeper investment research.

Core Features

- Research workspaces
- Saved reports
- Historical event timelines
- Advanced AI explanations
- Economic context integration

Example Questions

Why did this investment outperform?

What historical events influenced these results?

Success Criteria

Users gain context, not just numbers.

2.16.6 Version 4 — Market Intelligence Platform

Goal

Provide broad historical market exploration capabilities.

Core Features

- Cross-asset analysis
- Sector analysis
- Market trend exploration
- Historical screening tools
- Economic regime analysis

Example Questions

How did technology stocks behave during recessions?

Which sectors performed best during inflationary periods?

Success Criteria

Users can explore market history at scale.

2.16.7 Version 5 — Financial Education Ecosystem

Goal

Become the leading educational platform for historical investing.

Core Features

- Interactive learning modules
- Guided investment lessons
- AI teaching assistant
- Educational pathways
- Scenario-based learning

Example Questions

How does compounding work?

How does diversification reduce risk?

Success Criteria

Users improve financial understanding through structured learning.

2.16.8 Feature Prioritization Framework

Every proposed feature should be evaluated against:

Question	Required
Improves understanding?	Yes
Improves transparency?	Preferred
Improves accuracy?	Preferred
Improves educational value?	Yes
Supports core mission?	Yes

Features failing these criteria should not be prioritized.

2.16.9 Features Explicitly Excluded

The roadmap intentionally excludes:

Brokerage integrations
Real-money trading
Investment recommendations
Copy trading
Social investing
Market predictions
Speculative forecasting

These features do not align with the platform's educational mission.

2.16.10 Development Order

Major development priorities:

Phase 1

Data
Simulation Engine
Database
API

Phase 2

Frontend
Visualization
User Experience

Phase 3

AI Layer
Research Features
Advanced Analysis

Backend functionality must exist before frontend enhancements.

2.16.11 Milestone Structure

Milestone 1

Historical data successfully stored.

Milestone 2

Simulation engine produces accurate results.

Milestone 3

Public API operational.

Milestone 4

Frontend simulator functional.

Milestone 5

AI explanations integrated.

Milestone 6

Public MVP release.

Each milestone should produce a measurable improvement.

2.16.12 Roadmap Risks

Common risks include:

Risk	Impact
Feature Creep	Delayed delivery
Overengineering	Slower progress
Data Quality Issues	Reduced trust
AI Hallucinations	User confusion
Premature Scaling	Increased costs

The roadmap should favor simplicity whenever possible.

2.16.13 Long-Term Vision Alignment

Every roadmap stage should move the platform toward:

The world's most educational historical investment simulator and market intelligence platform.

The roadmap should expand depth before breadth.

Better simulations are more valuable than more features.

2.16.14 Success Metrics by Version

Version	Primary Success Metric
V1	Accurate historical simulations
V2	Portfolio analysis adoption
V3	Research feature engagement
V4	Market intelligence usage
V5	Educational outcomes and retention

Each version should have a clearly defined objective.

2.16.15 Summary

The Product Roadmap Architecture defines a staged path from a focused historical simulator to a comprehensive investment education platform.

Approved roadmap sequence:

V1 Historical Simulator
V2 Portfolio Analysis
V3 Research Platform
V4 Market Intelligence
V5 Education Ecosystem

The roadmap prioritizes educational value, historical accuracy, and engineering quality over feature volume.

Status: Part 2.16 Complete

Next: Part 2.17 — MVP Scope Definition

Investment Time Machine

Founder Specification

Part 2.17 — MVP Scope Definition

Status: Draft

2.17.1 Purpose

The MVP (Minimum Viable Product) defines the smallest version of Investment Time Machine that delivers meaningful value to users while validating the core product thesis.

The MVP is not intended to prove every feature in the long-term vision.

It is intended to answer one question:

Do users find value in exploring historical investment outcomes?

Every feature included in the MVP must directly support that objective.

2.17.2 MVP Philosophy

The MVP should prioritize:

- Accuracy
- Simplicity
- Educational value
- Speed of development

The MVP should not prioritize:

- Feature completeness
- Advanced analytics
- Complex infrastructure
- Enterprise scalability

Success comes from solving one problem extremely well.

2.17.3 MVP Goal

Allow a user to answer:

What would have happened if I invested in this asset during a specific period?

with:

- Accurate calculations
- Historical data
- Clear visualizations
- Educational explanations

If the platform accomplishes this reliably, the MVP is successful.

2.17.4 Included Features

Historical Investment Simulation

Users can:

- Select an asset
- Enter an investment amount
- Select a start date
- Select an end date

The platform calculates:

- Shares purchased
 - Final value
 - Total return
 - CAGR
-

Dividend Reinvestment

Users may choose:

Include Dividends

or

Exclude Dividends

The difference should be clearly visible.

Inflation Adjustment

Users may choose:

Nominal Return

or

Inflation-Adjusted Return

Economic data will be sourced from FRED.

Asset Search

Users can search supported assets.

Supported MVP assets:

- US Stocks
 - ETFs
 - Cryptocurrencies
-

AI Educational Explanation

Users may request:

Explain these results.

The AI layer may:

- Explain outcomes
- Explain historical context
- Explain investment concepts

The AI layer may not generate financial facts.

Basic Visualizations

MVP charts:

- Investment growth chart
- Performance summary
- Return comparison

Visualizations should prioritize clarity over complexity.

2.17.5 Included Data Sources

Source	Purpose
yfinance	Stocks and ETFs
CoinGecko	Cryptocurrency data
FRED	Economic indicators

No paid providers are required for MVP.

2.17.6 Included Infrastructure

Component	Technology
Frontend	Next.js
Backend	FastAPI
Database	PostgreSQL
Cache	Redis (Optional)
AI	External LLM Provider
Deployment	Vercel + Railway/Render
Containerization	Docker

This stack aligns with the project's low-cost strategy.

2.17.7 User Accounts

MVP options:

Option A

Public simulator without authentication.

Benefits:

- Faster development
- Lower complexity
- Easier testing

Option B

User accounts enabled.

Benefits:

- Saved simulations
- Simulation history
- Future personalization

Preferred recommendation:

Option B

if development effort remains reasonable.

2.17.8 Explicitly Excluded Features

The following are out of scope:

Portfolio Analysis

Watchlists

Retirement Planning

Tax Modeling

Real-Time Market Data

Brokerage Integration

Trading

Investment Recommendations

Social Features
Mobile Applications

These features belong to future versions.

2.17.9 MVP Success Criteria

Technical Success

The platform successfully:

- Stores historical data
- Generates simulations
- Calculates returns accurately
- Produces reproducible results
- Maintains stable APIs

Product Success

Users can:

- Discover assets
- Run simulations
- Understand outcomes
- Learn from explanations

Recruiter Success

A recruiter should immediately recognize:

- Backend engineering
 - Database design
 - API development
 - Data engineering
 - System architecture
 - AI integration
-

2.17.10 MVP Non-Goals

The MVP does not attempt to:

- Compete with brokerages
- Replace research terminals
- Predict future returns
- Offer investment advice
- Solve every investing problem

The MVP exists to validate the historical simulation concept.

2.17.11 Build Order

Recommended implementation sequence:

Priority	Component
1	Database
2	Historical Data Imports
3	Simulation Engine
4	API Layer
5	Frontend
6	Authentication
7	AI Explanations
8	Deployment

This order aligns with the principle:

Backend Before Frontend

2.17.12 MVP Release Checklist

Before launch:

- Historical data imports operational
- Asset search functional

- Simulation engine validated
- API documented
- Frontend deployed
- Error handling implemented
- Basic monitoring enabled
- AI explanations functional
- Database backups configured

Every item should be completed before public release.

2.17.13 Launch Definition

The MVP is considered launched when:

1. A user can access the platform.
2. A user can run a historical simulation.
3. Results are accurate and reproducible.
4. AI explanations are available.
5. The system operates reliably in production.

No additional features are required for launch.

2.17.14 Summary

The MVP is intentionally narrow.

It focuses on one core experience:

Historical investment simulation with educational explanations.

Included:

- Historical simulations
- Dividend reinvestment
- Inflation adjustment
- Asset search
- AI explanations
- Basic visualizations

Excluded:

- Portfolios
- Trading
- Recommendations
- Social features
- Advanced planning tools

A successful MVP proves that users value understanding historical investment outcomes.

Status: Part 2.17 Complete

Next: Part 2.18 — Engineering Workflow Architecture

Investment Time Machine

Founder Specification

Part 2.18 — Engineering Workflow Architecture

Status: Draft

2.18.1 Purpose

The Engineering Workflow Architecture defines how development work is planned, implemented, reviewed, tested, and deployed.

The objective is to maintain engineering quality while keeping development velocity high.

A good workflow should:

- Reduce mistakes
- Improve consistency
- Simplify collaboration
- Support long-term maintainability
- Keep documentation aligned with implementation

2.18.2 Workflow Philosophy

The project follows a documentation-first workflow.

Development sequence:

Decision

→ Documentation

→ Design

→ Implementation

→ Testing

→ Deployment

Major implementation work should not begin until the relevant architecture has been documented.

This ensures that engineering decisions remain intentional rather than reactive.

2.18.3 Development Lifecycle

Every feature should follow the same lifecycle.

Step 1 — Define

Document:

- Problem
- Goal
- Scope
- Success criteria

Step 2 — Design

Define:

- Architecture impact
- Database impact
- API impact
- UI impact

Step 3 — Implement

Build the feature according to the approved design.

Step 4 — Test

Verify:

- Functionality
- Edge cases
- Error handling
- Regression risks

Step 5 — Deploy

Release through the approved deployment workflow.

2.18.4 Branch Strategy

Recommended Git structure:

Branch	Purpose
main	Production-ready code
develop	Integration branch
feature/*	Feature development
hotfix/*	Production fixes

For solo development, a simplified workflow is acceptable:

main
feature/*

Complex Git workflows should not be introduced prematurely.

2.18.5 Feature Development Process

Every feature should begin with a written specification.

Example:

Feature:
Dividend Reinvestment

Goal:

Allow users to simulate dividend reinvestment.

Requirements:

- Retrieve dividend events
- Reinvest distributions
- Update share counts

Success Criteria:

Results match expected calculations

Specifications should exist before implementation.

2.18.6 Pull Request Standards

Every pull request should answer:

- What was changed?
- Why was it changed?
- What risks exist?
- How was it tested?

Large pull requests should be avoided.

Smaller, focused changes are easier to review and debug.

2.18.7 Code Review Principles

Review priorities:

Priority	Focus
1	Correctness
2	Security
3	Maintainability
4	Performance

Correct code is more important than clever code.

Readable code is preferred over complex optimizations.

2.18.8 Architecture Change Policy

Any change affecting:

- Database schema
- API contracts
- Core calculations
- Security architecture
- Data ingestion

must be documented before implementation.

Architecture should drive code, not the reverse.

2.18.9 Migration Workflow

Database changes require:

1. Schema update design.
2. Alembic migration creation.
3. Migration review.
4. Staging validation.
5. Production deployment.

Direct database modifications are prohibited.

Every schema change must be traceable.

2.18.10 Dependency Management

Before adding a dependency, evaluate:

- Is it necessary?
- Can existing tools solve the problem?
- Is it actively maintained?
- Does it introduce security risk?

Dependencies should be treated as long-term commitments.

Prefer fewer dependencies whenever practical.

2.18.11 Technical Debt Policy

Technical debt is acceptable when:

- Documented
- Intentional
- Temporary

Technical debt is unacceptable when:

- Hidden
- Untracked
- Avoidable

Every significant shortcut should include a plan for future resolution.

2.18.12 Bug Management

Bugs should be categorized by severity.

Severity	Description
Critical	Incorrect financial results
High	Broken core functionality
Medium	Partial feature failure
Low	Cosmetic or minor issues

Financial correctness bugs always receive highest priority.

2.18.13 Engineering Priorities

When priorities conflict:

1. Correctness
2. Security
3. Reliability
4. Maintainability
5. Performance
6. Convenience

This order should guide engineering decisions.

2.18.14 Documentation Requirements

The following must remain documented:

- Architecture decisions
- Database schema
- API contracts
- Deployment procedures
- Major engineering decisions

Documentation is considered part of the product.

Code and documentation should evolve together.

2.18.15 Definition of Done

A feature is considered complete only when:

- Requirements implemented
- Tests pass
- Documentation updated
- Errors handled
- Code reviewed
- Deployment validated

Implementation alone does not equal completion.

2.18.16 MVP Workflow Tools

Category	Tool
Source Control	GitHub
Project Planning	GitHub Projects
Documentation	Founder Specification
Database Migrations	Alembic
API Documentation	OpenAPI
Deployment	Vercel, Railway, Render

The workflow should remain lightweight during MVP development.

2.18.17 Future Workflow Expansion

Potential future additions:

- CI/CD Pipelines
- Automated Security Scanning
- Code Coverage Reporting
- Performance Testing
- Release Automation
- Infrastructure as Code

These should be added only when complexity justifies them.

2.18.18 Summary

The Engineering Workflow Architecture establishes a repeatable process for building and maintaining the platform.

Approved principles:

- Documentation first
- Architecture before implementation
- Small, focused changes
- Migration-driven database updates
- Correctness over speed
- Financial accuracy above all else

The workflow exists to ensure that the platform remains maintainable, reliable, and trustworthy as it grows.

Status: Part 2.18 Complete

Next: Part 2.19 — Testing Strategy Architecture

Investment Time Machine

Founder Specification

Part 2.18 — Engineering Workflow Architecture

Status: Draft

2.18.1 Purpose

The Engineering Workflow Architecture defines how development work is planned, implemented, reviewed, tested, and deployed.

The objective is to maintain engineering quality while keeping development velocity high.

A good workflow should:

- Reduce mistakes
- Improve consistency
- Simplify collaboration
- Support long-term maintainability
- Keep documentation aligned with implementation

2.18.2 Workflow Philosophy

The project follows a documentation-first workflow.

Development sequence:

Decision

→ Documentation

→ Design

→ Implementation

→ Testing

→ Deployment

Major implementation work should not begin until the relevant architecture has been documented.

This ensures that engineering decisions remain intentional rather than reactive.

2.18.3 Development Lifecycle

Every feature should follow the same lifecycle.

Step 1 — Define

Document:

- Problem
- Goal
- Scope
- Success criteria

Step 2 — Design

Define:

- Architecture impact
- Database impact
- API impact
- UI impact

Step 3 — Implement

Build the feature according to the approved design.

Step 4 — Test

Verify:

- Functionality
- Edge cases
- Error handling
- Regression risks

Step 5 — Deploy

Release through the approved deployment workflow.

2.18.4 Branch Strategy

Recommended Git structure:

Branch	Purpose
main	Production-ready code
develop	Integration branch
feature/*	Feature development
hotfix/*	Production fixes

For solo development, a simplified workflow is acceptable:

main
feature/*

Complex Git workflows should not be introduced prematurely.

2.18.5 Feature Development Process

Every feature should begin with a written specification.

Example:

Feature:
Dividend Reinvestment

Goal:
Allow users to simulate dividend reinvestment.

Requirements:
- Retrieve dividend events
- Reinvest distributions
- Update share counts

Success Criteria:
Results match expected calculations

Specifications should exist before implementation.

2.18.6 Pull Request Standards

Every pull request should answer:

- What was changed?
- Why was it changed?
- What risks exist?
- How was it tested?

Large pull requests should be avoided.

Smaller, focused changes are easier to review and debug.

2.18.7 Code Review Principles

Review priorities:

Priority	Focus
1	Correctness
2	Security

3 Maintainability

4 Performance

5 Style

Correct code is more important than clever code.

Readable code is preferred over complex optimizations.

2.18.8 Architecture Change Policy

Any change affecting:

- Database schema
- API contracts
- Core calculations
- Security architecture
- Data ingestion

must be documented before implementation.

Architecture should drive code, not the reverse.

2.18.9 Migration Workflow

Database changes require:

1. Schema update design.
2. Alembic migration creation.
3. Migration review.
4. Staging validation.
5. Production deployment.

Direct database modifications are prohibited.

Every schema change must be traceable.

2.18.10 Dependency Management

Before adding a dependency, evaluate:

- Is it necessary?
- Can existing tools solve the problem?
- Is it actively maintained?
- Does it introduce security risk?

Dependencies should be treated as long-term commitments.

Prefer fewer dependencies whenever practical.

2.18.11 Technical Debt Policy

Technical debt is acceptable when:

- Documented
- Intentional
- Temporary

Technical debt is unacceptable when:

- Hidden
- Untracked
- Avoidable

Every significant shortcut should include a plan for future resolution.

2.18.12 Bug Management

Bugs should be categorized by severity.

Severity	Description
Critical	Incorrect financial results
High	Broken core functionality

Medium Partial feature failure

Low Cosmetic or minor issues

Financial correctness bugs always receive highest priority.

2.18.13 Engineering Priorities

When priorities conflict:

1. Correctness
2. Security
3. Reliability
4. Maintainability
5. Performance
6. Convenience

This order should guide engineering decisions.

2.18.14 Documentation Requirements

The following must remain documented:

- Architecture decisions
- Database schema
- API contracts
- Deployment procedures
- Major engineering decisions

Documentation is considered part of the product.

Code and documentation should evolve together.

2.18.15 Definition of Done

A feature is considered complete only when:

- Requirements implemented

- Tests pass
- Documentation updated
- Errors handled
- Code reviewed
- Deployment validated

Implementation alone does not equal completion.

2.18.16 MVP Workflow Tools

Category	Tool
Source Control	GitHub
Project Planning	GitHub Projects
Documentation	Founder Specification
Database Migrations	Alembic
API Documentation	OpenAPI
Deployment	Vercel, Railway, Render

The workflow should remain lightweight during MVP development.

2.18.17 Future Workflow Expansion

Potential future additions:

CI/CD Pipelines
Automated Security Scanning
Code Coverage Reporting
Performance Testing
Release Automation
Infrastructure as Code

These should be added only when complexity justifies them.

2.18.18 Summary

The Engineering Workflow Architecture establishes a repeatable process for building and maintaining the platform.

Approved principles:

- Documentation first
- Architecture before implementation
- Small, focused changes
- Migration-driven database updates
- Correctness over speed
- Financial accuracy above all else

The workflow exists to ensure that the platform remains maintainable, reliable, and trustworthy as it grows.

Status: Part 2.18 Complete

Next: Part 2.19 — Testing Strategy Architecture

Investment Time Machine

Founder Specification

Part 2.19 — Testing Strategy Architecture

Status: Draft

2.19.1 Purpose

The Testing Strategy defines how the platform verifies correctness, reliability, and stability.

Because Investment Time Machine performs financial calculations, testing is not optional.

The primary objective is to ensure that users can trust every result produced by the platform.

Testing exists to verify:

- Financial correctness
- System reliability
- Data integrity

- API stability
 - Regression prevention
-

2.19.2 Testing Philosophy

The platform follows a simple rule:

If a calculation affects money, it must be tested.

Testing priorities:

Priority	Focus
1	Financial correctness
2	Data integrity
3	API reliability
4	Security
5	User experience

A visually imperfect feature is acceptable.

An incorrect financial result is not.

2.19.3 Testing Pyramid

The platform should follow a standard testing pyramid.

Integration Tests



MVP focus:

- Heavy unit testing

- Essential integration testing

Avoid excessive end-to-end complexity early in development.

2.19.4 Unit Testing

Unit tests verify individual components in isolation.

Primary targets:

- Simulation calculations
- Utility functions
- Validation logic
- Data transformations

Examples:

CAGR calculation

Return calculation

Dividend reinvestment

Inflation adjustment

The Simulation Engine should receive the highest unit test coverage in the platform.

2.19.5 Simulation Engine Testing

Every financial calculation must have deterministic test cases.

Required coverage:

Area	Required
Investment Growth	Yes
Total Return	Yes
CAGR	Yes
Dividend Reinvestment	Yes

Inflation Adjustment Yes

Error Handling Yes

The Simulation Engine is the highest-risk component and therefore receives the highest testing priority.

2.19.6 Known Answer Tests

Financial calculations should use known-answer test cases.

Example:

Investment: \$1,000

Initial Price: \$100

Final Price: \$250

Expected Value: \$2,500

Expected Return: 150%

Tests should compare actual outputs against predetermined expected results.

This improves confidence in financial correctness.

2.19.7 Edge Case Testing

Required edge cases:

Scenario	Expected Behavior
Zero investment	Validation error
Invalid dates	Validation error
Missing price data	Controlled failure
Negative values	Validation error

Empty datasets Controlled failure

Edge cases are often more important than normal cases.

2.19.8 Integration Testing

Integration tests verify interactions between components.

Examples:

API → Database

API → Simulation Engine

Simulation Engine → Historical Data

AI Service → Simulation Results

The goal is to ensure systems work correctly together.

2.19.9 API Testing

Every public endpoint should be tested.

Areas to verify:

- Request validation
- Response structure
- Authentication
- Error handling
- Status codes

Examples:

Endpoint	Test Required
Asset Search	Yes
Simulation Creation	Yes
User Authentication	Yes

2.19.10 Database Testing

Database tests should verify:

- Constraints
- Foreign keys
- Unique indexes
- Migrations
- Data integrity

Examples:

Duplicate asset prevention

Foreign key enforcement

Migration execution

Database correctness is critical because it stores financial truth.

2.19.11 Data Ingestion Testing

Data import systems should verify:

- Valid imports succeed
- Invalid imports fail
- Duplicates are rejected
- Data normalization works correctly
- Source attribution is preserved

Import quality directly impacts simulation quality.

2.19.12 AI Testing

AI output should not be tested for exact wording.

Instead verify:

- Successful generation
- Proper data inputs
- Prompt version tracking
- Error handling
- Storage behavior

Financial correctness should never depend on AI output.

2.19.13 Security Testing

Security tests should verify:

- Authentication requirements
- Authorization controls
- Protected endpoints
- Secret handling
- Input validation

Security failures can compromise user trust and system integrity.

2.19.14 Regression Testing

Every bug fix should include a test.

Rule:

A bug should never be fixed without adding a test that prevents it from returning.

This creates a growing safety net over time.

2.19.15 Performance Testing

Initial performance targets:

Operation	Target
Asset Search	< 250ms

Simulation	< 2s
API Response	< 500ms
Database Query	< 100ms

Performance testing becomes more important as data volume grows.

2.19.16 Test Data Strategy

Tests should use controlled datasets.

Benefits:

- Deterministic results
- Faster execution
- Easier debugging

Avoid relying on live APIs during testing.

Tests should remain reproducible.

2.19.17 Coverage Goals

Recommended minimum coverage:

Component	Target
Simulation Engine	90%+
Data Ingestion	85%+
API Layer	80%+
Database Logic	80%+
Frontend	60%+

Coverage is a useful metric, but correctness remains more important than percentages.

2.19.18 MVP Testing Stack

Category	Tool
Backend Testing	Pytest
API Testing	Pytest + FastAPI Test Client
Coverage	Coverage.py
Frontend Testing	Vitest
End-to-End Testing	Playwright (Optional)

The MVP should prioritize simple and reliable tooling.

2.19.19 Testing Requirements Before Launch

Before MVP release:

- Simulation calculations validated
- Core APIs tested
- Database migrations tested
- Historical imports tested
- Authentication tested
- Critical bugs resolved

No launch should occur without simulation validation.

2.19.20 Summary

The Testing Strategy exists to protect platform trust.

Approved principles:

- Financial correctness first
- Deterministic testing
- Known-answer validation
- Strong Simulation Engine coverage

- Integration testing for critical workflows
- Regression tests for bug fixes
- No dependency on live APIs during tests

A simulation platform is only as trustworthy as its testing strategy.

Status: Part 2.19 Complete

Next: Part 2.20 — Documentation Strategy Architecture

Investment Time Machine

Founder Specification

Part 2.20 — Documentation Strategy Architecture

Status: Draft

2.20.1 Purpose

Documentation is a core component of the platform, not a secondary artifact.

The Documentation Strategy defines how decisions, systems, APIs, workflows, and operational procedures are recorded and maintained throughout the project lifecycle.

The objective is to ensure:

- Knowledge preservation
- Faster development
- Easier onboarding
- Better maintainability
- Long-term project sustainability

2.20.2 Documentation Philosophy

Investment Time Machine follows a documentation-first approach.

Major decisions should be documented before implementation.

The platform treats documentation as a product asset alongside:

- Source code
- Historical data
- Infrastructure

A feature is not fully complete if its documentation is missing or outdated.

2.20.3 Documentation Principles

Accuracy

Documentation should reflect actual system behavior.

Outdated documentation is considered a defect.

Clarity

Documentation should be understandable by future developers without requiring tribal knowledge.

Brevity

Documentation should maximize decision density.

Avoid unnecessary repetition, excessive formatting, and filler content.

Traceability

Important decisions should explain:

- What was decided
 - Why it was decided
 - What alternatives were considered
-

2.20.4 Documentation Hierarchy

The project should maintain five documentation layers.

Layer

Purpose

Founder Specification	Product and architecture decisions
Technical Specifications	Detailed implementation design
API Documentation	Endpoint contracts
Operational Documentation	Deployment and maintenance
Code Documentation	Implementation details

Each layer serves a different audience and should not duplicate information unnecessarily.

2.20.5 Founder Specification

This document serves as the project's highest-level source of truth.

Responsibilities:

- Product vision
- Architecture
- Engineering principles
- Roadmaps
- System design decisions

The Founder Specification should answer:

Why does the system exist?

How is it designed?

How should it evolve?

2.20.6 Technical Specifications

Technical specifications provide implementation-level detail.

Examples:

Database Schema Specification

API Specification

Simulation Formula Specification

Data Import Specification

These documents should be detailed enough to support development without redefining architecture.

2.20.7 API Documentation

API documentation should be generated automatically whenever possible.

Required information:

- Endpoint
- Method
- Request schema
- Response schema
- Authentication requirements
- Example requests
- Example responses

OpenAPI serves as the primary API documentation source.

2.20.8 Database Documentation

Database documentation should include:

- Tables
- Columns
- Constraints
- Relationships
- Indexes
- Migration history

Database documentation must remain synchronized with schema changes.

2.20.9 Architecture Decision Records

Major decisions should be recorded.

Examples:

Why PostgreSQL was selected

Why FastAPI was selected

Why AI cannot generate financial data

Why historical data is stored locally

Decision records help future contributors understand tradeoffs.

2.20.10 Code Documentation

Code should be self-explanatory whenever practical.

Documentation should focus on:

- Intent
- Assumptions
- Non-obvious behavior

Avoid comments that merely repeat code.

Preferred:

Explain why.

Avoid:

Explain what obvious code already shows.

2.20.11 Operational Documentation

Operational documentation should cover:

- Deployment procedures
- Environment setup
- Backup procedures
- Recovery procedures
- Monitoring workflows

The objective is to reduce operational dependency on a single individual.

2.20.12 Documentation Ownership

Documentation ownership follows the same ownership model as code.

Rule:

Whoever changes the system is responsible for updating the documentation.

Documentation updates should be part of the definition of done.

2.20.13 Documentation Review Policy

Documentation should be reviewed whenever:

- Architecture changes
- Database schema changes
- API contracts change
- Deployment procedures change

Reviews help prevent documentation drift.

2.20.14 Documentation Storage

Recommended locations:

Documentation Type	Location
Founder Specification	Repository /docs
API Documentation	OpenAPI
Database Documentation	Repository /docs
Deployment Guides	Repository /docs

Decision Records

Repository
/docs/adr

Documentation should remain version-controlled alongside code.

2.20.15 Documentation Quality Standards

Good documentation should be:

- Accurate
- Current
- Searchable
- Concise
- Actionable

Bad documentation:

- Duplicates information
 - Becomes outdated
 - Lacks ownership
 - Omits rationale
-

2.20.16 MVP Documentation Requirements

Before MVP launch, the project should have:

- Founder Specification
- Database Schema Documentation
- API Documentation
- Deployment Guide
- Setup Guide
- Migration Procedures

These documents provide sufficient operational and engineering coverage.

2.20.17 Future Documentation Expansion

Potential future additions:

User Documentation
Developer Handbook
Architecture Diagrams
Contribution Guide
Runbooks
Incident Response Guides

These should be added as platform complexity increases.

2.20.18 Documentation Success Metrics

Documentation is successful when:

- New contributors can onboard quickly
- Architecture decisions are understandable
- Deployments are repeatable
- APIs are discoverable
- Database structures are understandable

Documentation should reduce uncertainty.

2.20.19 Summary

The Documentation Strategy ensures that knowledge remains accessible, accurate, and maintainable as the platform evolves.

Approved principles:

- Documentation first
- Version-controlled documentation
- Decision traceability
- Automatic API documentation
- Documentation ownership
- Documentation as part of the definition of done

Investment Time Machine treats documentation as a product asset, not an afterthought.

Status: Part 2.20 Complete

Founder-Level Architecture Status:

- Product Vision Complete
- System Architecture Complete
- Database Architecture Complete
- AI Architecture Complete
- Security Architecture Complete
- API Architecture Complete
- Infrastructure Architecture Complete
- Deployment Architecture Complete
- Monitoring Architecture Complete
- Data Ingestion Architecture Complete
- Simulation Architecture Complete
- Frontend Architecture Complete
- Product Roadmap Complete
- MVP Scope Complete
- Engineering Workflow Complete
- Testing Strategy Complete
- Documentation Strategy Complete

Next Recommended Section: Part 2.21 — Risk Analysis Architecture

Investment Time Machine

Founder Specification

Part 2.21 — Risk Analysis Architecture

Status: Draft

2.21.1 Purpose

Risk Analysis Architecture identifies the major risks that could impact the platform and defines strategies to reduce or manage them.

The objective is not to eliminate all risk.

The objective is to:

- Identify risks early

- Reduce avoidable failures
- Improve decision-making
- Protect platform trust

Risk management is especially important because the platform depends on historical financial accuracy.

2.21.2 Risk Philosophy

The platform follows three principles:

Identify Risks Early

Known risks are easier to manage than unexpected risks.

Prioritize High-Impact Risks

Not all risks deserve equal attention.

Focus should remain on risks that threaten:

- Financial correctness
- User trust
- Platform reliability

Prefer Prevention Over Recovery

Preventing a problem is generally cheaper than fixing it later.

2.21.3 Risk Categories

The platform's risks fall into six categories:

Category	Description
Data Risks	Historical data quality issues
Technical Risks	Software and infrastructure failures
Security Risks	Unauthorized access or compromise

Product Risks	Poor user adoption
AI Risks	Incorrect or misleading AI output
Operational Risks	Deployment and maintenance failures

2.21.4 Data Risks

Incorrect Historical Data

Risk:

Provider supplies inaccurate data.

Impact:

High

Mitigation:

- Data validation
 - Source attribution
 - Audit logging
 - Future multi-provider verification
-

Missing Historical Data

Risk:

Required price history unavailable.

Impact:

High

Mitigation:

- Validation during imports
- Missing-data detection

- Explicit simulation failures

The platform should fail clearly rather than fabricate results.

Duplicate Records

Risk:

Duplicate imports corrupt calculations.

Impact:

Medium

Mitigation:

- Unique constraints
 - Duplicate detection
 - Import validation
-

2.21.5 Simulation Risks

Incorrect Calculations

Risk:

Simulation engine produces incorrect results.

Impact:

Critical

Mitigation:

- Unit testing
- Known-answer tests
- Formula verification
- Code review

This is the highest-priority technical risk.

Non-Reproducible Results

Risk:

Same inputs produce different outputs.

Impact:

Critical

Mitigation:

- Deterministic calculations
- Versioned logic
- Local data ownership

2.21.6 AI Risks

Hallucinated Financial Facts

Risk:

AI invents prices or returns.

Impact:

Critical

Mitigation:

- AI cannot calculate results
- Structured inputs only
- Prompt controls
- Audit logging

Misleading Explanations

Risk:

AI generates plausible but incorrect explanations.

Impact:

Medium

Mitigation:

- Historical context validation
 - Prompt versioning
 - Human review during development
-

2.21.7 Security Risks

Credential Exposure

Risk:

API keys or secrets become public.

Impact:

Critical

Mitigation:

- Environment variables
 - Secret isolation
 - Access controls
-

Account Compromise

Risk:

Unauthorized user access.

Impact:

High

Mitigation:

- Password hashing
 - Authentication controls
 - Rate limiting
-

2.21.8 Infrastructure Risks

Database Failure

Risk:

Database becomes unavailable.

Impact:

Critical

Mitigation:

- Backups
 - Recovery procedures
 - Monitoring
-

Hosting Provider Outage

Risk:

Railway, Render, or Vercel downtime.

Impact:

Medium

Mitigation:

- Provider-independent architecture
- Backup deployment plans
- Data ownership

2.21.9 Product Risks

Low User Adoption

Risk:

Users do not find value in the product.

Impact:

Critical

Mitigation:

- Focused MVP
- User feedback
- Educational value
- Fast iteration

Feature Creep

Risk:

Too many features slow development.

Impact:

High

Mitigation:

- Strict MVP scope
- Roadmap discipline
- Feature prioritization framework

2.21.10 Operational Risks

Failed Deployments

Risk:

Deployment introduces outages.

Impact:

High

Mitigation:

- Staging environment
 - Rollback strategy
 - Deployment validation
-

Documentation Drift

Risk:

Documentation becomes inaccurate.

Impact:

Medium

Mitigation:

- Documentation ownership
 - Documentation reviews
 - Definition of done requirements
-

2.21.11 Risk Severity Matrix

Severity
y

Definition

Critical	Threatens platform trust or correctness
----------	---

High	Major user or operational impact
Medium	Noticeable but manageable impact
Low	Limited impact

Priority order:

Critical
High
Medium
Low

Engineering effort should align with risk severity.

2.21.12 Highest Priority Risks

The platform's most important risks are:

1. Incorrect simulation calculations
2. Corrupted historical data
3. AI-generated financial misinformation
4. Security credential exposure
5. Loss of user trust

These risks should receive disproportionate attention.

2.21.13 Risk Review Process

Risks should be reviewed whenever:

- Architecture changes
- New providers are added
- New AI functionality is added
- Major infrastructure changes occur

Risk analysis is an ongoing process, not a one-time activity.

2.21.14 Summary

The Risk Analysis Architecture provides a structured approach for identifying and managing threats to platform success.

Approved priorities:

- Protect historical accuracy
- Protect simulation correctness
- Prevent AI misinformation
- Protect user trust
- Maintain operational reliability

Trust is the platform's most valuable asset, and risk management exists to protect it.

Status: Part 2.21 Complete

Next: Part 2.22 — Scalability Strategy Architecture

Investment Time Machine

Founder Specification

Part 2.22 — Scalability Strategy Architecture

Status: Draft

2.22.1 Purpose

The Scalability Strategy defines how the platform grows while maintaining:

- Accuracy
- Reliability
- Performance
- Cost efficiency
- Maintainability

The objective is not to build for millions of users on day one.

The objective is to ensure today's decisions do not prevent future growth.

2.22.2 Scalability Philosophy

The platform follows a simple rule:

Scale based on evidence, not assumptions.

Infrastructure complexity should only be introduced when supported by:

- User growth
- Data growth
- Performance bottlenecks
- Operational requirements

Premature optimization is prohibited.

2.22.3 Scaling Priorities

When growth occurs, scaling priorities should be:

1. Correctness
2. Reliability
3. Performance
4. Cost Efficiency

A faster system with incorrect calculations is unacceptable.

Financial correctness remains the highest priority.

2.22.4 Growth Stages

Stage 1 — MVP

Target:

0–1,000 users

Infrastructure:

- Single FastAPI instance
- Single PostgreSQL instance

- Optional Redis
- Vercel frontend

Characteristics:

- Lowest cost
 - Simplest operations
 - Fastest development
-

Stage 2 — Early Growth

Target:

1,000–10,000 users

Infrastructure additions:

- Multiple backend instances
- Redis caching
- Improved monitoring
- Background job processing

Primary focus:

- API performance
 - Reduced database load
-

Stage 3 — Growth

Target:

10,000–100,000 users

Infrastructure additions:

- Database read replicas
- Dedicated background workers
- Advanced observability
- Enhanced caching

Primary focus:

- Database scalability
 - Query optimization
-

Stage 4 — Large Scale

Target:

100,000+ users

Infrastructure additions:

- Distributed services
- Data warehouse
- Advanced analytics
- Multi-region deployments

This stage is intentionally outside MVP planning.

2.22.5 Database Scalability

The database is the most important scalability concern.

Expected growth:

Dataset	Growth Rate
Historical Prices	High
Simulations	Medium
Audit Logs	Medium
AI Explanations	Medium
Users	Low

Scaling order:

1. Query optimization
2. Index optimization
3. Read replicas

4. Partitioning
5. Advanced database architecture

Database redesign should be a last resort.

2.22.6 Historical Data Scaling

Historical prices will become the largest table in the platform.

Growth estimate:

Assets	Years	Approximate Records
1,000	10	~2.5M
5,000	20	~25M
10,000	20	~50M

PostgreSQL can comfortably support this scale with proper indexing.

Partitioning should only be introduced when actual performance metrics justify it.

2.22.7 API Scalability

API scaling should occur horizontally.

Preferred approach:

Load Balancer



Backend Instance 1

Backend Instance 2

Backend Instance 3

Benefits:

- Simplicity
- Reliability

- Provider independence

FastAPI is well suited for horizontal scaling.

2.22.8 Simulation Engine Scalability

Simulation requests are computational workloads.

Scaling options:

Vertical Scaling

Increase:

- CPU
- Memory

Simplest approach.

Horizontal Scaling

Multiple simulation workers.

Recommended once simulation volume increases significantly.

2.22.9 Caching Strategy

Redis should be introduced when performance data justifies it.

Primary cache candidates:

Data	Cache Worthy
Asset Metadata	Yes
Historical Prices	Yes
Economic Indicators	Yes
Simulation Results	Maybe

The cache should improve performance but never become the source of truth.

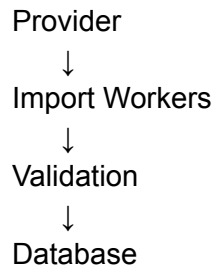
PostgreSQL remains authoritative.

2.22.10 Data Ingestion Scalability

As asset coverage expands:

- Import volume increases
- Validation volume increases
- Storage volume increases

Scaling approach:



Background processing should absorb ingestion growth.

2.22.11 AI Scalability

AI costs and latency increase with usage.

Scaling strategy:

1. Prompt optimization
2. Caching explanations
3. Model selection optimization
4. Background generation

The AI layer should scale independently from simulations.

Simulation performance must never depend on AI availability.

2.22.12 Frontend Scalability

The frontend should remain stateless.

Benefits:

- Easy deployment
- CDN support
- Horizontal scaling

Vercel can absorb significant traffic before custom infrastructure becomes necessary.

2.22.13 Monitoring Scalability

Monitoring complexity should increase gradually.

MVP:

Provider Metrics

Sentry

Uptime Monitoring

Future:

Prometheus

Grafana

OpenTelemetry

Observability should scale with operational complexity.

2.22.14 Cost-Conscious Scaling

Every scaling decision should answer:

1. What bottleneck exists?
2. How severe is it?
3. What is the cheapest effective solution?
4. Can optimization solve it before new infrastructure?

Infrastructure should be justified by measured needs.

2.22.15 Anti-Patterns

The platform should avoid:

Premature microservices
Premature Kubernetes
Premature partitioning
Premature distributed systems
Premature multi-region deployments

These add complexity without delivering user value during early stages.

2.22.16 Scalability Success Metrics

Scalability is successful when:

- Simulations remain accurate
- Performance remains acceptable
- Costs remain reasonable
- Reliability remains high
- Complexity remains manageable

Growth should not compromise platform quality.

2.22.17 Summary

The Scalability Strategy provides a roadmap for platform growth while preserving simplicity.

Approved principles:

- Scale based on evidence
- Database first optimization
- Horizontal API scaling
- Independent AI scaling
- Cost-conscious infrastructure growth

- Avoid premature complexity

The platform should evolve from a simple MVP into a large-scale educational platform through incremental, measured scaling decisions.

Status: Part 2.22 Complete

Next: Part 2.23 — Cost Architecture

Investment Time Machine

Founder Specification

Part 2.23 — Cost Architecture

Status: Draft

2.23.1 Purpose

The Cost Architecture defines how the platform manages infrastructure and operational expenses throughout its lifecycle.

The objective is to:

- Minimize unnecessary spending
- Maximize learning per dollar spent
- Maintain sustainability during development
- Delay major infrastructure costs until justified

Cost efficiency is a strategic advantage during the MVP stage.

2.23.2 Cost Philosophy

The platform follows three rules:

Build Before Spending

Prove user value before increasing infrastructure investment.

Prefer Free Tiers

Use free or low-cost services whenever they satisfy requirements.

Scale Costs With Usage

Infrastructure spending should increase only when user growth creates measurable demand.

2.23.3 Cost Categories

The platform's primary costs fall into five categories:

Category	Examples
Hosting	Backend, frontend, databases
AI Usage	LLM API costs
Storage	Historical datasets and backups
Monitoring	Error tracking and uptime monitoring
Future Services	Analytics and scaling infrastructure

2.23.4 MVP Infrastructure Costs

Recommended MVP stack:

Component	Provider	Target Cost
Frontend	Vercel	Free
Backend	Railway / Render	Free–\$5
PostgreSQL	Railway / Render	Free–\$5
GitHub	GitHub	Free
Docker	Docker	Free
Monitoring	Sentry Free Tier	Free

Target monthly infrastructure cost:

\$0–\$10/month

This aligns with the project's original engineering principles.

2.23.5 AI Cost Management

AI is the most likely source of variable costs.

Cost drivers:

- Number of requests
- Prompt size
- Response size
- Model selection

Control strategies:

- Keep prompts concise
- Limit unnecessary generation
- Cache explanations when appropriate
- Use AI only where educational value exists

AI should enhance the platform, not dominate operating costs.

2.23.6 Historical Data Costs

MVP providers:

yfinance
CoinGecko
FRED

Cost:

\$0/month

Because the platform stores data locally after import, provider costs remain minimal.

Local ownership reduces long-term dependency costs.

2.23.7 Storage Cost Strategy

Stored assets include:

- Historical prices
- Dividends
- Stock splits
- Economic indicators
- Simulations
- Audit logs
- AI explanations

Expected MVP storage requirements remain modest.

Estimated scale:

< 10 GB

Storage costs should remain negligible during early development.

2.23.8 Development Cost Strategy

Development spending should focus on:

1. Data quality
2. Product validation
3. User feedback

Avoid spending on:

Enterprise infrastructure
Premium analytics
Complex DevOps tooling
Large-scale cloud services

before user demand exists.

2.23.9 Scaling Cost Stages

Stage 1 — MVP

Target:

\$0–\$10/month

Primary focus:

- Build product
 - Validate idea
-

Stage 2 — Early Users

Target:

\$10–\$50/month

Potential additions:

- Better database plans
 - Increased AI usage
 - Improved monitoring
-

Stage 3 — Growth

Target:

\$50–\$500/month

Potential additions:

- Redis
- Read replicas

- Background workers
 - Enhanced observability
-

Stage 4 — Scale

Target:

Usage-dependent

Infrastructure should scale only after proven demand.

2.23.10 Cost Monitoring

The platform should track:

Cost Area	Monitor
AI Usage	Monthly spend
Hosting	Monthly spend
Database	Storage growth
Data Imports	Resource usage
Infrastructure	Total monthly cost

Unexpected cost growth should be investigated.

2.23.11 Cost Optimization Principles

Before purchasing additional infrastructure:

1. Measure the bottleneck.
2. Verify the bottleneck exists.
3. Attempt optimization.
4. Evaluate alternatives.
5. Upgrade only when justified.

Optimization should generally precede spending.

2.23.12 Cost Risks

Risk	Impact	Mitigation
AI Overuse	High	Usage limits
Premature Scaling	Medium	Evidence-based upgrades
Provider Pricing Changes	Medium	Provider independence
Excessive Monitoring Tools	Low	Start simple
Feature Creep	High	Strict roadmap discipline

Cost discipline is a product strategy, not merely a financial strategy.

2.23.13 Cost Success Metrics

Cost management is successful when:

- MVP remains affordable
 - Infrastructure costs grow slower than value delivered
 - AI costs remain controlled
 - Development remains sustainable
 - Scaling decisions remain data-driven
-

2.23.14 Summary

The Cost Architecture ensures that Investment Time Machine remains financially sustainable during development and growth.

Approved principles:

- Free-tier first
- Pay only when justified
- Local data ownership

- Controlled AI usage
- Evidence-based scaling
- Continuous cost monitoring

The platform should maximize learning and product progress while minimizing unnecessary infrastructure spending.

Status: Part 2.23 Complete

Next: Part 2.24 — Disaster Recovery & Business Continuity Architecture

Investment Time Machine

Founder Specification

Part 2.24 — Disaster Recovery & Business Continuity Architecture

Status: Draft

2.24.1 Purpose

Disaster Recovery and Business Continuity define how the platform responds to failures, outages, data loss, and operational disruptions.

The objective is to ensure:

- Historical data remains recoverable
- Critical services can be restored
- User trust is preserved
- Platform downtime is minimized

Because historical data is a core product asset, recovery planning is a mandatory architectural requirement.

2.24.2 Recovery Philosophy

The platform follows three principles:

Recover Critical Data First

Historical data is more valuable than infrastructure.

Servers can be recreated.

Lost historical datasets are significantly harder to replace.

Automate Recovery Where Practical

Recovery procedures should minimize manual intervention.

Prepare Before Failure

Recovery plans must exist before disasters occur.

2.24.3 Critical Assets

Assets are classified by recovery priority.

Tier 1 — Critical

Must be recoverable.

Historical Prices
Dividends
Stock Splits
Economic Indicators
Simulations
Database Schema
Migration History
Source Code

Tier 2 — Important

Should be recoverable.

Audit Logs
AI Explanations
Monitoring Data

Tier 3 — Rebuildable

Can be recreated if necessary.

Application Containers

Temporary Caches

Redis Data

2.24.4 Failure Scenarios

The architecture should prepare for:

Scenario	Expected Response
Database Failure	Restore database backup
Data Corruption	Restore validated snapshot
Deployment Failure	Rollback deployment
Provider Outage	Continue using local data
AI Provider Failure	Disable explanations only
Hosting Failure	Redeploy infrastructure
Human Error	Restore backup and audit changes

2.24.5 Backup Strategy

Database Backups

Minimum requirement:

Daily Automated Backups

Recommended future state:

Daily Full Backup

Hourly Incremental Backup

Backups must include:

- Data
- Schema
- Indexes
- Constraints

Source Code Backups

Primary source:

GitHub Repository

Benefits:

- Version history
- Change tracking
- Easy restoration

Source code should never exist only on local machines.

Documentation Backups

Required documents:

- Founder Specification
- Database Schema
- Deployment Procedures
- Architecture Records

Documentation should remain version-controlled alongside code.

2.24.6 Recovery Objectives

Recovery Time Objective (RTO)

Maximum acceptable downtime:

24 Hours

for MVP.

Recovery Point Objective (RPO)

Maximum acceptable data loss:

24 Hours

for MVP.

Future versions may require stricter targets.

2.24.7 Database Recovery

Recovery process:

Database Failure



Identify Backup



Restore Database



Validate Data Integrity



Resume Services

Validation should include:

- Record counts
 - Schema verification
 - Critical query testing
-

2.24.8 Deployment Recovery

If deployment introduces failures:

Failed Deployment



Rollback



Restore Previous Version



Validate System Health

Frontend and backend deployments should support independent rollback.

2.24.9 Historical Data Protection

Historical market data should be treated as a strategic asset.

Protection mechanisms:

- Import validation
- Source attribution
- Audit logging
- Backup retention

Historical truth must remain recoverable.

2.24.10 AI Failure Handling

AI failures should not affect core functionality.

Example:

Simulation Success

AI Failure

Result:

Simulation Available

Explanation Unavailable

The platform remains operational.

2.24.11 Business Continuity Strategy

The platform should continue functioning during:

- AI outages
- Provider outages
- Import failures
- Partial service disruptions

Core simulation functionality should remain the highest operational priority.

2.24.12 Recovery Testing

Backups are only useful if restoration works.

Recovery testing should verify:

- Database restoration
- Deployment rollback
- Backup integrity
- Recovery procedures

Recovery procedures should be tested periodically.

2.24.13 Single Points of Failure

The platform should minimize dependence on:

One Developer
One Hosting Provider
One Data Provider
One AI Provider

Provider independence improves resilience.

2.24.14 Incident Response Workflow

Standard workflow:

Detect

↓

Assess

↓

Contain

↓

Recover

↓

Review

Post-incident reviews should document:

- Cause
 - Impact
 - Resolution
 - Prevention measures
-

2.24.15 Future Enhancements

Potential future additions:

Cross-Region Backups

Automated Failover

Read Replicas

Multi-Provider Hosting

Disaster Recovery Drills

These features are intentionally deferred until platform scale justifies additional complexity.

2.24.16 Summary

The Disaster Recovery & Business Continuity Architecture protects the platform from data loss, outages, and operational failures.

Approved principles:

- Historical data is a critical asset
- Daily backups required
- Recovery procedures documented
- AI failures must not impact simulations
- Rollback capability required
- Provider independence preferred
- Recovery testing encouraged

A trustworthy platform must not only operate correctly—it must be able to recover when failures occur.

Status: Part 2.24 Complete

Next: Part 2.25 — Long-Term Evolution Strategy

Investment Time Machine

Founder Specification

Part 2.25 — Long-Term Evolution Strategy

Status: Draft

2.25.1 Purpose

The Long-Term Evolution Strategy defines how Investment Time Machine evolves over the next several years while remaining aligned with its core mission.

The objective is to ensure that future growth remains intentional, sustainable, and consistent with the platform's educational focus.

This section answers:

- Where is the platform heading?
 - What should it become?
 - What should it never become?
-

2.25.2 Mission Preservation

As the platform evolves, the following principles must remain unchanged:

- Historical accuracy
- Educational value
- Transparency
- Reproducibility
- Evidence-based analysis

Future features should strengthen these principles rather than dilute them.

2.25.3 Evolution Path

The platform should evolve through five stages.

Stage 1 — Historical Investment Simulator

Core capabilities:

- Historical simulations
- Dividend analysis
- Inflation adjustment
- Educational explanations

Primary question:

What would have happened if I invested?

Stage 2 — Portfolio Intelligence Platform

Core capabilities:

- Portfolio construction
- Portfolio backtesting
- Allocation analysis
- Rebalancing simulations

Primary question:

How would a portfolio have performed?

Stage 3 — Historical Research Platform

Core capabilities:

- Research workspaces
- Historical event timelines
- Economic context analysis
- Saved reports

Primary question:

Why did these outcomes occur?

Stage 4 — Market Intelligence Platform

Core capabilities:

- Cross-asset analysis
- Sector research
- Economic regime analysis
- Historical screening tools

Primary question:

What patterns exist across market history?

Stage 5 — Financial Education Ecosystem

Core capabilities:

- Interactive learning experiences
- AI teaching systems
- Guided educational pathways
- Investment learning curriculum

Primary question:

How can users become better investors through understanding?

2.25.4 Product Expansion Areas

Future expansion may include:

Portfolio Analysis

Examples:

- Multi-asset portfolios
- Allocation optimization
- Historical diversification analysis

Retirement Planning

Examples:

- Contribution schedules
- Withdrawal simulations
- Retirement projections

Risk Analysis

Examples:

- Drawdowns
- Volatility
- Risk-adjusted returns

Market Research

Examples:

- Sector performance
- Historical comparisons
- Economic trend analysis

These expansions should build upon existing simulation infrastructure.

2.25.5 Geographic Expansion

Initial focus:

United States Markets

Future expansion:

Europe
Canada
Asia-Pacific
Global Markets

International expansion should occur only after domestic functionality is mature.

2.25.6 Asset Expansion

Initial assets:

- Stocks
- ETFs
- Cryptocurrencies

Future assets:

Bonds
Commodities
Mutual Funds
Forex
Real Estate Indexes
Alternative Assets

Asset coverage should expand gradually while preserving data quality.

2.25.7 AI Evolution

Current AI role:

Educational Explanation

Future AI roles:

Research Assistant
Learning Assistant
Historical Context Engine
Educational Tutor

AI should remain an educational tool rather than a source of financial truth.

2.25.8 Platform Capabilities to Avoid

The platform should avoid evolving into:

Brokerage Platform
Trading Platform
Prediction Engine
Stock Picking Service
Social Trading Network
Investment Recommendation Service

These directions conflict with the platform's mission.

2.25.9 Technical Evolution

Expected evolution:

Phase	Focus
Early Stage	Simplicity
Growth Stage	Reliability
Scale Stage	Performance
Maturity Stage	Intelligence

Infrastructure complexity should increase only when justified by actual usage.

2.25.10 Data Evolution

Future datasets may include:

Earnings Data
Macroeconomic Data
Sector Data
Options Data
Fundamentals
Alternative Datasets

Every new dataset must satisfy:

- Accuracy requirements
 - Validation requirements
 - Source attribution requirements
-

2.25.11 Competitive Positioning

The platform should position itself differently from:

Category	Focus
Brokerages	Trading
Financial News	Information
Research Platforms	Analysis
Investment Time Machine	Understanding

The platform's advantage is educational historical exploration.

2.25.12 Success Definition

Long-term success means users can:

- Understand investment outcomes

- Learn financial concepts
- Explore market history
- Evaluate risk
- Improve decision-making

Success is measured by understanding, not trading volume.

2.25.13 Five-Year Vision

Within five years, the platform should aspire to become:

The world's most trusted educational platform for exploring and understanding historical investment decisions.

This vision should guide future roadmap decisions.

2.25.14 Architectural Preservation Rules

Future growth must preserve:

- Deterministic calculations
- AI separation
- Local data ownership
- Database integrity
- Reproducibility
- Documentation-first development

These principles are foundational and should not be compromised.

2.25.15 Summary

The Long-Term Evolution Strategy defines a path from a focused historical simulator to a comprehensive financial education platform.

Approved evolution path:

Historical Simulator
→ Portfolio Platform

- Research Platform
- Market Intelligence Platform
- Education Ecosystem

The platform should expand depth before breadth, prioritize understanding over prediction, and remain committed to accuracy, transparency, and education.

Status: Part 2.25 Complete

Part 2 — System Architecture Complete

Founder Specification Architecture Phase Status:

- ✓ Vision & Strategy
- ✓ System Architecture
- ✓ Database Architecture
- ✓ AI Architecture
- ✓ Security Architecture
- ✓ API Architecture
- ✓ Infrastructure Architecture
- ✓ Deployment Architecture
- ✓ Monitoring Architecture
- ✓ Data Ingestion Architecture
- ✓ Simulation Architecture
- ✓ Frontend Architecture
- ✓ Product Roadmap
- ✓ MVP Scope
- ✓ Engineering Workflow
- ✓ Testing Strategy
- ✓ Documentation Strategy
- ✓ Risk Analysis
- ✓ Scalability Strategy
- ✓ Cost Architecture
- ✓ Disaster Recovery
- ✓ Long-Term Evolution Strategy

Part 2 Status: Complete

Investment Time Machine

Founder Specification

Part 3 — Product & Execution Architecture

Status: Planned

Purpose

Part 1 defined the vision.

Part 2 defined the architecture.

Part 3 defines exactly what the product does, how users interact with it, how features behave, how financial analytics are calculated, and how future AI coding agents should implement the platform.

Part 3 becomes the bridge between architecture and implementation.

Part 3.1 — Product Requirements Document (PRD)

Defines:

- Product Overview
- User Personas
- Core User Journeys
- Screens
- Features
- Acceptance Criteria
- Success Metrics

Purpose:

Answer:

What are we building?

Part 3.2 — User Flows

Defines:

- First-Time User Flow
- Historical Simulation Flow
- Asset Discovery Flow
- AI Explanation Flow
- Account Creation Flow
- Future Portfolio Flow

Purpose:

Answer:

How does a user move through the product?

Part 3.3 — Functional Requirements

Defines every major feature.

For each feature:

- Inputs
- Outputs
- Validation Rules
- Error States
- Edge Cases
- Future Extensions

Examples:

- Historical Simulation
- Dividend Reinvestment
- Inflation Adjustment
- Asset Search
- Authentication
- AI Explanations

Purpose:

Answer:

Exactly how should the product behave?

Part 3.4 — Non-Functional Requirements

Defines:

- Performance Requirements
- Reliability Requirements
- Availability Targets
- Scalability Requirements
- Security Requirements
- Accessibility Requirements

Purpose:

Answer:

How well must the system perform?

Part 3.5 — Financial Analytics Architecture

Defines every financial metric used by the platform.

Examples:

- Total Return
- CAGR
- Volatility
- Sharpe Ratio
- Sortino Ratio
- Max Drawdown
- Calmar Ratio
- Rolling Returns
- Opportunity Cost Analysis

For every metric:

- Purpose

- Formula
- Inputs
- Outputs
- Edge Cases
- API Exposure
- Storage Requirements

Purpose:

Answer:

How are financial insights generated?

Part 3.6 — Threat Modeling

Defines:

- Threat
- Impact
- Likelihood
- Mitigation

Examples:

- API Abuse
- Credential Stuffing
- Data Corruption
- Prompt Injection
- Secret Leakage
- Provider Compromise

Purpose:

Answer:

What can go wrong?

Part 3.7 — Business Intelligence Architecture

Defines platform metrics.

Examples:

- Monthly Active Users
- Daily Active Users
- Simulations Created
- Simulation Success Rate
- AI Usage Rate
- Asset Popularity
- Retention Metrics

Purpose:

Answer:

How do we measure success?

Part 3.8 — AI Development Standards

Defines mandatory implementation rules for:

- Claude Code
- Codex
- Future AI Coding Agents

Examples:

- SQLAlchemy 2.0 only
- UUID primary keys
- Pydantic v2
- Type hints required
- Tests required
- Alembic migrations required
- No business logic in frontend

Purpose:

Answer:

How should code be generated?

Part 3 Completion Criteria

Part 3 is complete when:

- Every screen is defined
- Every user flow is defined
- Every feature is defined
- Every financial metric is defined
- Every coding agent has implementation standards
- Every major threat is documented

At that point the Founder Specification becomes implementation-ready.

Status:

Part 1 Complete

Part 2 Complete

Part 3 Planned

Investment Time Machine

Founder Specification

Part 3.1 — Product Requirements Document (PRD)

Status: Draft

3.1.1 Purpose

The Product Requirements Document defines what the product does from a user perspective.

Part 2 defined how the system is built.

Part 3.1 defines what users experience.

The PRD serves as the primary reference for:

- Product decisions
- Feature development
- UI planning
- Backend requirements
- Future roadmap prioritization

It answers:

What are we building?

3.1.2 Product Overview

Investment Time Machine is a historical investment intelligence platform that allows users to explore historical investment outcomes using real market data.

The platform helps users answer questions such as:

- What would have happened if I invested in Bitcoin in 2011?
- How much would \$1,000 invested in Apple be worth today?
- Would SPY have outperformed Tesla?
- How did inflation affect my returns?

The product combines:

- Historical simulations
- Financial analytics
- Educational explanations
- Market intelligence

into a single experience.

3.1.3 Product Goal

The MVP has one primary goal:

Allow users to understand historical investment outcomes through accurate simulations and educational analysis.

Everything included in the MVP should directly support this goal.

3.1.4 Primary User Personas

Persona 1 — College Student

Characteristics:

- Learning personal finance
- Beginning investor
- Limited market experience

Primary Questions:

- What if I started investing earlier?
- How powerful is compounding?
- How much difference does time make?

Success Outcome:

Better understanding of investing fundamentals.

Persona 2 — Retail Investor

Characteristics:

- Familiar with investing
- Interested in historical performance
- Interested in portfolio decisions

Primary Questions:

- Was this investment worth the risk?
- How would diversification have changed results?

Success Outcome:

Better understanding of historical outcomes and risk.

Persona 3 — Research-Oriented User

Characteristics:

- Curious about market history
- Enjoys data analysis
- Interested in comparisons

Primary Questions:

- Which assets performed best?
- How did sectors behave during market downturns?

Success Outcome:

Deeper understanding of market behavior.

3.1.5 Core Product Experience

The primary product experience follows a simple workflow:

Choose Asset



Choose Investment Amount



Choose Time Period



Run Simulation



View Results



Learn Why

Every major feature should strengthen this workflow.

3.1.6 MVP Screens

The MVP consists of six primary screens.

Screen	Purpose
Home	Product introduction

Simulator	Create simulations
Asset Explorer	Discover assets
Results	View outcomes
Reports	AI explanations
Account	User history and settings

These screens define the initial user experience.

3.1.7 Home Screen

Purpose

Introduce the platform and guide users toward simulations.

Core Components

- Hero section
- Search bar
- Example simulations
- Feature overview
- Call-to-action

Primary User Actions

- Search an asset
- Start a simulation
- Learn about the platform

Success Metric

User successfully begins a simulation.

3.1.8 Simulator Screen

Purpose

Collect simulation inputs.

Inputs

Field	Required
Asset	Yes
Investment Amount	Yes
Start Date	Yes
End Date	Yes
Include Dividends	Optional
Adjust for Inflation	Optional

User Actions

- Configure simulation
- Submit simulation request

Success Metric

Simulation executes successfully.

3.1.9 Results Screen

Purpose

Present historical investment outcomes.

Core Components

- Final value
- Total return
- CAGR
- Growth chart
- Key metrics

User Actions

- Review results
- Compare outcomes
- Request AI explanation

Success Metric

User understands simulation outcome.

3.1.10 Asset Explorer

Purpose

Allow users to discover supported assets.

Core Components

- Asset search
- Asset details
- Asset metadata
- Historical overview

User Actions

- Search assets
- View asset information
- Launch simulations

Success Metric

User finds an asset and starts analysis.

3.1.11 Reports Screen

Purpose

Provide educational explanations.

Core Components

- AI explanation
- Historical context
- Economic context
- Key takeaways

User Actions

- Read explanation
- Learn investment concepts
- Explore related topics

Success Metric

User gains understanding beyond raw numbers.

3.1.12 Account Screen

Purpose

Store user-specific information.

Core Components

- Profile information
- Simulation history
- Saved reports

User Actions

- View previous simulations
- Revisit reports
- Manage account

Success Metric

User returns to prior work.

3.1.13 Core MVP Features

The MVP includes:

Feature	Included
Historical Simulation	Yes
Dividend Reinvestment	Yes
Inflation Adjustment	Yes
Asset Search	Yes
AI Explanation	Yes
User Accounts	Yes
Simulation History	Yes

Everything else is deferred.

3.1.14 Excluded Features

The MVP excludes:

Portfolio Analysis
Watchlists
Retirement Planning
Tax Modeling
Trading
Brokerage Integrations
Social Features
Investment Recommendations

These belong to future versions.

3.1.15 Product Success Metrics

User Metrics

- Simulations created

- Repeat users
- AI explanation usage
- Session completion rate

Product Metrics

- Simulation success rate
- API reliability
- Historical data coverage

Educational Metrics

- Explanation engagement
 - Report completion
 - User retention
-

3.1.16 Acceptance Criteria

The MVP is successful when:

1. Users can discover assets.
2. Users can run simulations.
3. Results are historically accurate.
4. Results are reproducible.
5. AI explanations are available.
6. The platform remains stable in production.

All six criteria must be satisfied before MVP completion.

3.1.17 Summary

The Product Requirements Document defines the user-facing experience of Investment Time Machine.

Approved MVP scope:

- Six core screens
- Historical simulations
- Dividend reinvestment
- Inflation adjustment

- AI explanations
- User accounts
- Simulation history

The PRD acts as the bridge between architecture and implementation and serves as the foundation for future user flows and functional requirements.

Status: Part 3.1 Complete

Next: Part 3.2 — User Flows

Investment Time Machine

Founder Specification

Part 3.2 — User Flows

Status: Draft

3.2.1 Purpose

User Flows define how users interact with the platform to accomplish specific goals.

The objective is to:

- Clarify product behavior
- Guide frontend design
- Guide API design
- Support feature development
- Reduce ambiguity during implementation

Each flow should represent a complete user journey from intent to outcome.

3.2.2 User Flow Philosophy

A good user flow should be:

- Simple
- Predictable
- Educational

- Efficient

The platform should minimize unnecessary steps between:

Question



Simulation



Understanding

Every additional step must provide value.

3.2.3 Primary User Journey

The primary product journey is:

User Arrives



Selects Asset



Configures Simulation



Runs Simulation



Views Results



Reads Explanation



Learns Something

The entire platform is designed around this flow.

3.2.4 First-Time Visitor Flow

Goal

Introduce new users to the platform.

Flow

Landing Page



View Product Overview



Search Asset



Open Simulator



Run First Simulation



View Results

Success Criteria

User completes their first simulation.

3.2.5 Historical Simulation Flow

Goal

Allow users to analyze a historical investment.

Flow

Select Asset



Enter Investment Amount



Choose Start Date



Choose End Date



Configure Options



Run Simulation



View Results

Inputs

- Asset
- Investment amount
- Start date
- End date
- Dividend option
- Inflation option

Output

- Final value
- Total return
- CAGR
- Growth chart

Success Criteria

User receives accurate results.

3.2.6 Asset Discovery Flow

Goal

Help users find assets worth exploring.

Flow

Open Asset Explorer



Search Asset



Review Asset Information



Select Asset



Launch Simulation

Success Criteria

User successfully transitions from exploration to analysis.

3.2.7 AI Explanation Flow

Goal

Help users understand simulation outcomes.

Flow

Simulation Complete



Request Explanation



AI Generates Report



Display Educational Analysis

Output

- Historical context
- Economic context
- Key insights
- Educational explanation

Success Criteria

User gains understanding beyond raw performance metrics.

3.2.8 Comparison Flow

Goal

Compare multiple assets.

Flow

Select Asset A



Select Asset B



Run Comparison



View Performance Differences



Review Explanation

Example

AAPL vs SPY

BTC vs Gold

Tesla vs Nvidia

Success Criteria

User understands relative performance.

3.2.9 Account Creation Flow

Goal

Create a user account.

Flow

Open Registration



Enter Information



Create Account



Login



Access Dashboard

Inputs

- Email
- Password
- Display Name

Success Criteria

User successfully authenticates.

3.2.10 Login Flow

Goal

Authenticate returning users.

Flow

Open Login
↓
Enter Credentials
↓
Authenticate
↓
Access Account

Success Criteria

User accesses saved simulations and reports.

3.2.11 Simulation History Flow

Goal

Allow users to revisit previous analyses.

Flow

Login
↓
Open History
↓
Select Simulation
↓
Review Results
↓
Optionally Re-Run

Success Criteria

Users can easily revisit previous work.

3.2.12 Error Flow — Invalid Asset

Scenario

User requests an unsupported asset.

Flow

User Submits Asset
↓
Validation Fails
↓
Display Error
↓
Suggest Valid Assets

User Message

Asset not found.

Success Criteria

User can recover without confusion.

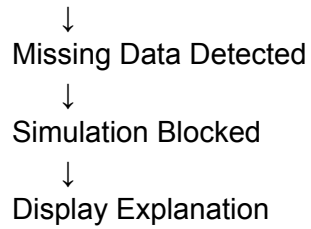
3.2.13 Error Flow — Missing Historical Data

Scenario

Required market data is unavailable.

Flow

Simulation Request



User Message

Historical data unavailable for the selected period.

Success Criteria

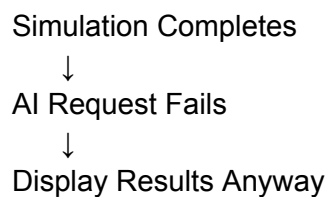
System fails transparently.

3.2.14 Error Flow — AI Failure

Scenario

AI service becomes unavailable.

Flow



User Message

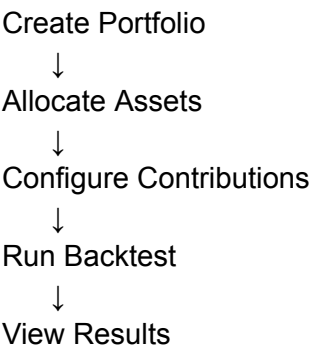
Explanation unavailable.

Success Criteria

Simulation remains accessible.

3.2.15 Future Portfolio Flow

Future versions may support:



This flow is intentionally deferred beyond MVP.

3.2.16 User Flow Success Metrics

Key metrics:

Metric	Purpose
Simulation Completion Rate	Core engagement
Asset Search Success Rate	Discovery effectiveness
AI Explanation Usage Rate	Educational engagement
Account Creation Rate	User adoption
Return Visit Rate	Product retention

User flows should be measured continuously.

3.2.17 Summary

User Flows define how users move through the platform to achieve meaningful outcomes.

Core flows include:

- First-time visitor flow
- Historical simulation flow
- Asset discovery flow
- AI explanation flow
- Authentication flow
- Simulation history flow

The platform's primary objective remains:

Question



Simulation



Understanding

Every future feature should strengthen this journey.

Status: Part 3.2 Complete

Next: Part 3.3 — Functional Requirements

Investment Time Machine

Founder Specification

Part 3.3 — Functional Requirements

Status: Draft

3.3.1 Purpose

Functional Requirements define exactly how the platform behaves.

Unlike architecture documents, which describe system design, functional requirements describe user-visible behavior.

Each feature must specify:

- Inputs
- Outputs

- Validation Rules
- Error Conditions
- Edge Cases
- Future Expansion Opportunities

The objective is to eliminate ambiguity during implementation.

3.3.2 Historical Investment Simulation

Purpose

Allow users to simulate a historical investment using real market data.

Inputs

Field	Required
Asset	Yes
Investment Amount	Yes
Start Date	Yes
End Date	Yes
Include Dividends	No
Adjust for Inflation	No

Outputs

Output	Required
Final Value	Yes
Total Return	Yes
CAGR	Yes
Growth Chart	Yes

Validation Rules

- Asset must exist.

- Investment amount must be greater than zero.
- Start date must be valid.
- End date must be valid.
- End date must be after start date.

Error Conditions

Error	Response
Asset Not Found	Validation error
Missing Historical Data	Simulation blocked
Invalid Dates	Validation error
Invalid Investment Amount	Validation error

Edge Cases

- Asset delisted during period
- Single-day simulation
- Extremely long date ranges
- Market closures

3.3.3 Dividend Reinvestment

Purpose

Allow users to include dividend reinvestment in historical simulations.

Inputs

include_dividends = true

Outputs

- Updated share count
- Updated final value
- Updated return calculations

Validation Rules

- Asset must support dividend history.
- Dividend records must exist.

Error Conditions

Error	Response
Missing Dividend Data	Inform user
Invalid Dividend Record	Reject calculation

Edge Cases

- No dividends paid
- Special dividends
- Dividend frequency changes

3.3.4 Inflation Adjustment

Purpose

Allow users to view inflation-adjusted results.

Inputs

`adjust_for_inflation = true`

Outputs

- Inflation-adjusted value
- Real return
- Inflation comparison

Validation Rules

- CPI data available
- Economic indicator data available

Error Conditions

Error	Response
-------	----------

Missing CPI Data Inflation adjustment unavailable

Edge Cases

- High inflation periods
 - Deflation periods
 - Long historical simulations
-

3.3.5 Asset Search

Purpose

Allow users to discover supported assets.

Inputs

- Symbol
- Asset name
- Partial search term

Outputs

- Matching assets
- Asset metadata

Validation Rules

- Search query must not be empty.

Error Conditions

Error	Response
No Results Found	Display empty state

Edge Cases

- Similar ticker symbols
- Multiple asset matches
- Case-insensitive searches

3.3.6 Asset Details

Purpose

Provide information about an asset before simulation.

Outputs

- Symbol
- Name
- Asset type
- Currency
- Exchange
- Historical availability

Error Conditions

Error	Response
Asset Missing	Asset unavailable

3.3.7 AI Explanation Generation

Purpose

Generate educational explanations for simulation results.

Inputs

simulation_id

Outputs

- Historical context
- Educational analysis
- Economic context
- Key insights

Validation Rules

- Simulation must exist.
- Simulation must be completed.

Error Conditions

Error	Response
Simulation Missing	Explanation blocked
AI Service Failure	Explanation unavailable

Edge Cases

- Extremely large returns
 - Negative returns
 - Long simulation periods
-

3.3.8 User Registration

Purpose

Create a platform account.

Inputs

- Email
- Password
- Display Name

Outputs

- User account
- Authentication credentials

Validation Rules

- Email format valid
- Password meets requirements
- Email unique

Error Conditions

Error	Response
Duplicate Email	Registration denied
Invalid Email	Validation error
Weak Password	Validation error

3.3.9 User Authentication

Purpose

Authenticate users.

Inputs

- Email
- Password

Outputs

- Access token
- Authenticated session

Error Conditions

Error	Response
Invalid Credentials	Authentication denied
Disabled Account	Authentication denied

3.3.10 Simulation History

Purpose

Allow users to review previous simulations.

Inputs

- Authenticated user

Outputs

- Simulation list
- Search results
- Historical records

Validation Rules

- User authenticated

Error Conditions

Error	Response
Unauthorized	Access denied

3.3.11 Asset Comparison

Purpose

Allow users to compare multiple assets.

Inputs

- Asset A
- Asset B
- Time period

Outputs

- Relative performance
- Return comparison
- CAGR comparison
- Visual comparison

Validation Rules

- Both assets exist.

- Shared historical period available.

Edge Cases

- Different inception dates
 - Missing data periods
 - Extreme performance differences
-

3.3.12 Report Generation

Purpose

Create educational reports from simulation results.

Inputs

- Simulation result
- AI explanation

Outputs

- Educational report
- Summary insights

Error Conditions

Error	Response
Missing Simulation	Report unavailable
AI Failure	Partial report available

3.3.13 Global Validation Rules

The following rules apply platform-wide:

Assets

- Must exist in asset catalog.

Dates

- Must be valid.
- End date must follow start date.

Numeric Inputs

- Must be positive.
- Must be within supported ranges.

Authentication

- Required for user-specific features.
-

3.3.14 Global Error Handling

All errors should:

- Be user-friendly
- Be actionable
- Avoid exposing internal details

Standard error response:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Human-readable explanation"
  }
}
```

3.3.15 Functional Requirement Priorities

Critical

- Historical Simulation
- Dividend Reinvestment
- Inflation Adjustment
- Asset Search

High

- AI Explanations
- Authentication
- Simulation History

Medium

- Asset Comparison
- Report Generation

Implementation priority should follow this order.

3.3.16 Summary

The Functional Requirements document defines the expected behavior of every major MVP feature.

Approved MVP functions:

- Historical simulations
- Dividend reinvestment
- Inflation adjustment
- Asset search
- Asset details
- AI explanations
- User accounts
- Simulation history
- Asset comparison
- Report generation

These requirements serve as the implementation contract between product design and engineering.

Status: Part 3.3 Complete

Next: Part 3 **Investment Time Machine**

Founder Specification

Part 3.4 — Non-Functional Requirements

Status: Draft

3.4.1 Purpose

Non-Functional Requirements define how well the platform must operate rather than what features it provides.

These requirements establish standards for:

- Performance
- Reliability
- Security
- Scalability
- Maintainability
- Accessibility

Functional requirements define behavior.

Non-functional requirements define quality.

3.4.2 Philosophy

The platform prioritizes:

1. Accuracy
2. Reliability
3. Security
4. Maintainability
5. Performance

When tradeoffs occur, correctness takes precedence over speed.

A fast but incorrect simulation is unacceptable.

3.4.3 Performance Requirements

API Performance

Operation	Target
Asset Search	< 250 ms
Historical Data Retrieval	< 500 ms
Authentication	< 500 ms
Simulation Creation Request	< 500 ms

Simulation Performance

Operation	Target
Single Asset Simulation	< 2 seconds
Asset Comparison	< 3 seconds
AI Explanation Generation	< 15 seconds

Simulation accuracy remains more important than simulation speed.

Frontend Performance

Metric	Target
First Page Load	< 3 seconds
Navigation Between Pages	< 1 second
Chart Rendering	< 1 second
Search Interaction	< 500 ms

3.4.4 Reliability Requirements

System Availability

MVP target:

99% uptime

Future target:

99.9% uptime

Data Reliability

The platform must ensure:

- Historical data remains accurate

- Imports are validated
 - Duplicates are prevented
 - Simulations remain reproducible
-

Failure Isolation

Failures in one system should not cascade.

Example:

AI Failure



Simulation Continues

The AI layer must never become a dependency for simulation correctness.

3.4.5 Security Requirements

Required controls:

- HTTPS in production
- Password hashing
- Authentication
- Authorization
- Rate limiting
- Secret management
- Input validation

Security requirements defined in Part 2.8 remain mandatory.

3.4.6 Scalability Requirements

The architecture should support:

Stage	Users
MVP	1,000
Early Growth	10,000
Growth	100,000
Future Scale	100,000+

Scaling should occur without major architectural redesign.

3.4.7 Maintainability Requirements

The system should support:

- Modular services
- Version-controlled migrations
- Documentation-first development
- Automated testing
- Clear API contracts

Code should be understandable by future developers without requiring undocumented knowledge.

3.4.8 Testability Requirements

The platform must support:

- Unit testing
- Integration testing
- Regression testing

- Database testing

Critical financial calculations should be independently testable.

The Simulation Engine must maintain the highest test coverage.

3.4.9 Observability Requirements

The system must provide visibility into:

- Errors
- Performance
- Usage
- Failures
- Imports

Required capabilities:

Health Checks

Structured Logs

Error Tracking

Audit Logs

Monitoring

A production issue should be diagnosable using platform telemetry.

3.4.10 Accessibility Requirements

The frontend should support:

- Keyboard navigation
- Semantic HTML
- Accessible forms
- Screen reader compatibility
- Color-independent communication

Accessibility should be incorporated during development rather than added later.

3.4.11 Compatibility Requirements

Supported platforms:

Platform	Required
----------	----------

Desktop	Yes
---------	-----

Tablet	Yes
--------	-----

Mobile	Yes
--------	-----

Supported browsers:

Browser	Required
---------	----------

Chrome	Yes
--------	-----

Firefox	Yes
---------	-----

Safari	Yes
--------	-----

Edge	Yes
------	-----

3.4.12 Data Integrity Requirements

The platform must guarantee:

- Foreign key integrity
- Constraint enforcement
- Source attribution
- Auditability
- Reproducibility

Historical financial data must never be silently modified.

3.4.13 Compliance Requirements

The MVP should follow basic industry best practices for:

- Data protection
- Credential security
- User privacy

The platform is not initially designed for:

SOC 2

HIPAA

PCI DSS

but future architecture should avoid blocking compliance initiatives.

3.4.14 Backup and Recovery Requirements

Minimum requirements:

- Daily database backups
- Version-controlled source code
- Recovery procedures
- Backup verification

Target recovery objectives:

Metric	Target
--------	--------

RTO	24 Hours
-----	----------

RPO	24 Hours
-----	----------

3.4.15 Documentation Requirements

The following must remain documented:

- Architecture
- Database schema
- API contracts
- Deployment procedures
- Engineering decisions

Documentation updates are part of the definition of done.

3.4.16 Cost Requirements

Target MVP operating cost:

\$0–\$10/month

Infrastructure decisions should support sustainable development without sacrificing core functionality.

3.4.17 Quality Requirements

The platform should maintain:

- Accurate calculations
- Stable APIs
- Reliable imports
- Clear user experience
- Consistent behavior

Quality should be measurable and continuously monitored.

3.4.18 Non-Functional Success Metrics

The platform is successful when:

- Simulations are accurate
- APIs remain responsive
- Data remains trustworthy
- Failures are observable
- Costs remain controlled
- Users can rely on results

These metrics define operational quality.

3.4.19 Summary

The Non-Functional Requirements establish the quality standards that govern the entire platform.

Approved priorities:

- Accuracy first
- Reliability second
- Security third
- Maintainability fourth
- Performance fifth

These requirements ensure that Investment Time Machine remains trustworthy, scalable, and sustainable as it grows.

Status: Part 3.4 Complete

Next: Part 3.5 — Financial Analytics Architecture

.4 — Non-Functional Requirements

Investment Time Machine

Founder Specification

Part 3.5 — Financial Analytics Architecture

Status: Draft

3.5.1 Purpose

The Financial Analytics Architecture defines every financial metric used by the platform.

This section establishes:

- Metric definitions
- Formulas
- Inputs
- Outputs
- Interpretation
- Edge cases
- Storage requirements
- API exposure rules

The objective is to ensure every financial insight is:

- Reproducible
- Auditable
- Explainable
- Consistent

The Financial Analytics Engine operates as a deterministic system and is independent of AI-generated content.

3.5.2 Analytics Philosophy

Every metric must satisfy three requirements:

Reproducible

The same inputs must always produce the same result.

Explainable

Users must understand what the metric means.

Auditable

The platform must be able to show how the metric was calculated.

Metrics that cannot satisfy these requirements should not be included.

3.5.3 Return on Investment (ROI)

Purpose

Measure total investment growth relative to initial capital.

Formula

ROI =
$$\frac{((\text{Final Value} - \text{Initial Investment}) / \text{Initial Investment}) \times 100}{}$$

Inputs

- Initial Investment
- Final Value

Output

Percentage return.

Interpretation

ROI	Meaning
Positive	Investment gained value

Zero No change
Negative Investment lost value

API Exposure

Required.

Storage

Stored in simulations table.

3.5.4 Compound Annual Growth Rate (CAGR)

Purpose

Measure average annual growth over time.

Formula

CAGR =
$$\left(\frac{\text{Final Value}}{\text{Initial Investment}} \right)^{\frac{1}{\text{Years}}} - 1$$

Inputs

- Initial Investment
- Final Value
- Time Period

Output

Annualized percentage return.

Interpretation

Allows fair comparison between investments with different durations.

API Exposure

Required.

Storage

Stored in simulations table.

3.5.5 Volatility

Purpose

Measure variability of returns.

Formula

Standard deviation of periodic returns.

Inputs

- Historical returns

Output

Volatility percentage.

Interpretation

Volatility	Meaning
Low	More stable
High	More variable

MVP Status

Deferred to Version 2.

3.5.6 Sharpe Ratio

Purpose

Measure risk-adjusted return.

Formula

$$\frac{(\text{Return} - \text{Risk Free Rate})}{\text{Volatility}}$$

Inputs

- Average Return
- Risk-Free Rate
- Volatility

Output

Sharpe Ratio

Interpretation

Higher values indicate better return per unit of risk.

MVP Status

Deferred.

3.5.7 Sortino Ratio

Purpose

Measure risk-adjusted return using downside risk only.

Formula

$$\frac{(\text{Return} - \text{Risk Free Rate})}{\text{Downside Deviation}}$$

Inputs

- Average Return

- Risk-Free Rate
- Downside Deviation

Output

Sortino Ratio

Interpretation

Penalizes harmful volatility rather than total volatility.

MVP Status

Deferred.

3.5.8 Maximum Drawdown

Purpose

Measure largest peak-to-trough decline.

Formula

$$\frac{(\text{Peak Value} - \text{Lowest Value})}{\text{Peak Value}}$$

Inputs

- Historical portfolio values

Output

Percentage decline.

Interpretation

Represents worst historical loss experienced.

MVP Status

Deferred.

3.5.9 Calmar Ratio

Purpose

Measure return relative to maximum drawdown.

Formula

$$\frac{\text{Annual Return}}{\text{Maximum Drawdown}}$$

Inputs

- CAGR
- Maximum Drawdown

Output

Calmar Ratio

Interpretation

Higher values indicate stronger risk-adjusted performance.

MVP Status

Deferred.

3.5.10 Inflation-Adjusted Return

Purpose

Measure purchasing-power growth.

Inputs

- Final Value
- CPI Data

Output

Real Return
Inflation-Adjusted Value

Interpretation

Shows what investment gains are worth after inflation.

API Exposure

Required.

Storage

Stored in simulations table.

3.5.11 Dividend Contribution

Purpose

Measure the impact of dividends on total return.

Inputs

- Dividend events
- Share ownership history

Output

Dividend value contribution.

Interpretation

Shows how much growth came from dividends versus price appreciation.

MVP Status

Supported through dividend reinvestment calculations.

3.5.12 Opportunity Cost Analysis

Purpose

Compare one investment against an alternative.

Example

AAPL vs SPY

BTC vs Gold

Tesla vs Nasdaq

Inputs

- Asset A
- Asset B
- Same investment amount
- Same time period

Output

Performance difference.

Interpretation

Helps users understand alternative outcomes.

API Exposure

Required for comparison features.

3.5.13 Best Entry Date Analysis

Purpose

Identify historically optimal investment entry points.

Inputs

- Historical price series

Output

Best historical purchase dates.

Interpretation

Educational analysis only.

Not predictive.

MVP Status

Deferred.

3.5.14 Best Exit Date Analysis

Purpose

Identify historically optimal exit dates.

Inputs

- Historical price series

Output

Best historical sale dates.

Interpretation

Historical analysis only.

Not predictive.

MVP Status

Deferred.

3.5.15 Rolling Return Analysis

Purpose

Measure performance across repeated holding periods.

Example

1-Year Rolling Return

5-Year Rolling Return

10-Year Rolling Return

Output

Distribution of historical outcomes.

Interpretation

Shows consistency of performance over time.

MVP Status

Deferred.

3.5.16 Analytics Storage Strategy

Metrics should be divided into two categories.

Persisted Metrics

Stored in database:

- Final Value
- ROI
- CAGR
- Inflation-Adjusted Value

Derived Metrics

Calculated on demand:

- Volatility
- Sharpe Ratio
- Sortino Ratio
- Drawdown Analysis
- Rolling Returns

This minimizes storage requirements while preserving flexibility.

3.5.17 Analytics API Exposure

Public MVP metrics:

Metric	API Exposure
Final Value	Yes
ROI	Yes
CAGR	Yes
Inflation-Adjusted Value	Yes
Dividend Contribution	Yes

Future metrics:

Metric	Future Exposure
Volatility	V2
Sharpe Ratio	V2
Sortino Ratio	V2
Drawdown	V2
Rolling Returns	V3

3.5.18 Analytics Validation

Every metric must be validated through:

- Known-answer tests
- Formula verification
- Historical examples
- Regression tests

Financial metrics are part of the platform's trust model.

3.5.19 Future Analytics Expansion

Potential future additions:

Alpha
Beta
Treynor Ratio
Information Ratio
Correlation Analysis
Factor Exposure
Risk Attribution

New metrics should only be added when they provide meaningful educational value.

3.5.20 Summary

The Financial Analytics Architecture defines the quantitative foundation of the platform.

Approved MVP metrics:

- Final Value
- ROI
- CAGR
- Inflation-Adjusted Return
- Dividend Contribution
- Opportunity Cost Analysis

Future versions may expand into advanced risk and portfolio analytics while preserving the platform's commitment to transparency, accuracy, and reproducibility.

Status: Part 3.5 Complete

Next: Part 3.6 — Threat Modeling

Investment Time Machine

Founder Specification

Part 3.6 — Threat Modeling

Status: Draft

3.6.1 Purpose

Threat Modeling identifies potential security, operational, data, and platform risks before they become incidents.

The objective is to systematically evaluate:

- What could go wrong
- How likely it is
- What impact it would have
- How it should be mitigated

Threat modeling complements Security Architecture by focusing on realistic attack and failure scenarios.

3.6.2 Threat Modeling Philosophy

The platform follows four principles:

Assume Failure Is Possible

Every component can fail.

Protect Trust First

Threats that impact financial correctness receive highest priority.

Prioritize Realistic Threats

Focus on practical threats before theoretical threats.

Mitigate Before Scaling

Security and reliability controls should be established before growth increases risk.

3.6.3 Risk Classification

Threats are evaluated using:

Impact

Level	Description
Critical	Threatens platform trust or correctness
High	Major operational impact
Medium	Noticeable disruption
Low	Limited impact

Likelihood

Level	Description
High	Expected to occur
Medium	Possible
Low	Uncommon

3.6.4 Threat: Incorrect Simulation Results

Category

Financial Integrity

Impact

Critical

Likelihood

Medium

Risk

Users receive incorrect investment outcomes.

Mitigations

- Known-answer testing
- Formula validation
- Code review
- Regression testing
- Deterministic calculations

Detection

- Failed tests
- User reports
- Analytics validation

This is the highest-priority threat in the platform.

3.6.5 Threat: Corrupted Historical Data

Category

Data Integrity

Impact

Critical

Likelihood

Medium

Risk

Invalid historical data contaminates simulations.

Mitigations

- Validation layer
- Import verification
- Duplicate detection
- Source attribution
- Audit logging

Detection

- Import validation failures
 - Data quality checks
-

3.6.6 Threat: AI Hallucinated Financial Facts

Category

AI Risk

Impact

Critical

Likelihood

Medium

Risk

AI invents prices, returns, or market data.

Mitigations

- AI cannot calculate results
- Structured inputs only
- Prompt controls
- Input auditing
- Output review during development

Detection

- AI audits
- User reports
- Explanation validation

3.6.7 Threat: Credential Stuffing

Category

Authentication

Impact

High

Likelihood

Medium

Risk

Automated login attempts compromise accounts.

Mitigations

- Argon2 password hashing
- Rate limiting
- Account lockout policies
- Future MFA support

Detection

- Failed login spikes
 - Security monitoring
-

3.6.8 Threat: API Abuse

Category

Availability

Impact

Medium

Likelihood

High

Risk

Automated traffic overwhelms APIs.

Mitigations

- Rate limiting
- Request validation
- Caching
- Monitoring

Detection

- Traffic anomalies
 - Error-rate increases
-

3.6.9 Threat: Secret Exposure

Category

Infrastructure Security

Impact

Critical

Likelihood

Medium

Risk

API keys or credentials become public.

Mitigations

- Environment variables
- Secret isolation

- Repository scanning
- Access control

Detection

- Security audits
 - Automated scanning
-

3.6.10 Threat: Database Failure

Category

Infrastructure

Impact

Critical

Likelihood

Low

Risk

Database becomes unavailable.

Mitigations

- Automated backups
- Recovery procedures
- Monitoring
- Hosting provider redundancy

Detection

- Health checks
 - Database alerts
-

3.6.11 Threat: Data Loss

Category

Business Continuity

Impact

Critical

Likelihood

Low

Risk

Historical data or simulations become unrecoverable.

Mitigations

- Daily backups
- Source control
- Recovery testing
- Backup verification

Detection

- Backup monitoring
 - Recovery audits
-

3.6.12 Threat: Malicious Data Import

Category

Data Ingestion

Impact

High

Likelihood

Low

Risk

Bad provider data enters the platform.

Mitigations

- Validation pipeline
- Schema checks
- Import auditing
- Source attribution

Detection

- Import failures
 - Data anomaly detection
-

3.6.13 Threat: Unauthorized Administrative Access

Category

Authorization

Impact

Critical

Likelihood

Low

Risk

Unauthorized user gains elevated privileges.

Mitigations

- Role-based access control
- Principle of least privilege
- Audit logging
- Administrative monitoring

Detection

- Audit logs
 - Security alerts
-

3.6.14 Threat: Deployment Failure

Category

Operations

Impact

High

Likelihood

Medium

Risk

Deployment introduces production instability.

Mitigations

- Staging validation
- Rollback strategy
- Deployment monitoring

Detection

- Health checks
 - Error tracking
-

3.6.15 Threat: Provider Outage

Category

External Dependency

Impact

Medium

Likelihood

Medium

Risk

Data provider or AI provider becomes unavailable.

Mitigations

- Local data ownership
- Provider abstraction
- Failure isolation

Detection

- Import failures
 - Service monitoring
-

3.6.16 Threat: Documentation Drift

Category

Operational Risk

Impact

Medium

Likelihood

High

Risk

Documentation no longer reflects system behavior.

Mitigations

- Documentation-first workflow

- Review requirements
- Definition of done updates

Detection

- Documentation reviews
 - Engineering audits
-

3.6.17 Threat Prioritization Matrix

Threat	Impact	Likelihood	Priority
Incorrect Simulations	Critical	Medium	Highest
Corrupted Data	Critical	Medium	Highest
AI Hallucinations	Critical	Medium	Highest
Secret Exposure	Critical	Medium	Highest
Database Failure	Critical	Low	High
Credential Stuffing	High	Medium	High
Deployment Failure	High	Medium	High
API Abuse	Medium	High	Medium
Provider Outage	Medium	Medium	Medium
Documentation Drift	Medium	High	Medium

3.6.18 Security Monitoring Requirements

The following events should generate alerts:

- Repeated authentication failures
- Secret exposure detection
- Database outages
- Import failures
- Elevated error rates
- Administrative access anomalies

Threat detection should be integrated into platform monitoring.

3.6.19 Future Threats

Future platform growth may introduce:

Prompt Injection
Data Poisoning
Model Manipulation
Supply Chain Attacks
Social Engineering
Advanced Credential Theft

Threat modeling should be revisited as architecture evolves.

3.6.20 Summary

Threat Modeling provides a structured framework for identifying and mitigating risks before they become incidents.

Highest-priority threats:

- Incorrect simulations
- Corrupted historical data
- AI-generated misinformation
- Secret exposure
- Unauthorized administrative access

Platform trust depends on protecting both financial correctness and operational integrity.

Status: Part 3.6 Complete

Next: Part 3.7 — Business Intelligence Architecture

Investment Time Machine

Founder Specification

Part 3.7 — Business Intelligence Architecture

Status: Draft

3.7.1 Purpose

Business Intelligence (BI) Architecture defines how platform usage, engagement, growth, and product effectiveness are measured.

The objective is to provide quantitative visibility into:

- User behavior
- Product adoption
- Feature usage
- Platform performance
- Business growth

The platform should make decisions using data rather than assumptions.

3.7.2 BI Philosophy

The platform follows three principles:

Measure What Matters

Collect metrics that directly support product improvement.

Avoid Vanity Metrics

Metrics should drive decisions, not presentations.

Respect User Privacy

Analytics should prioritize aggregated product insights over excessive user tracking.

3.7.3 Measurement Categories

The platform should track five categories:

Category	Purpose
User Metrics	Understand user behavior
Product Metrics	Measure feature adoption
Simulation Metrics	Measure core platform usage
AI Metrics	Measure educational engagement
Technical Metrics	Measure operational health

3.7.4 User Metrics

Monthly Active Users (MAU)

Definition:

Number of unique users active during a 30-day period.

Purpose:

Measure overall platform adoption.

Weekly Active Users (WAU)

Definition:

Number of unique users active during a 7-day period.

Purpose:

Measure short-term engagement.

Daily Active Users (DAU)

Definition:

Number of unique users active during a 24-hour period.

Purpose:

Measure daily platform activity.

User Retention

Definition:

Percentage of users returning after their first visit.

Suggested checkpoints:

Day 1

Day 7

Day 30

Purpose:

Measure long-term value.

3.7.5 Simulation Metrics

Simulations Created

Definition:

Total simulations executed.

Purpose:

Measure core product usage.

Simulation Success Rate

Formula:

Successful Simulations

/

Total Simulations

Target:

> 99%

Purpose:

Measure platform reliability.

Simulations Per User

Definition:

Average simulations created per active user.

Purpose:

Measure engagement depth.

Most Simulated Assets

Examples:

AAPL

SPY

BTC

TSLA

Purpose:

Understand user interests.

3.7.6 AI Metrics

AI Explanation Requests

Definition:

Number of generated explanations.

Purpose:

Measure educational feature usage.

AI Usage Rate

Formula:

AI Explanation Requests
/
Total Simulations

Purpose:

Measure educational engagement.

Explanation Completion Rate

Definition:

Percentage of requested explanations successfully generated.

Purpose:

Measure AI reliability.

Average Generation Time

Definition:

Average time required to generate an explanation.

Purpose:

Measure user experience quality.

3.7.7 Product Metrics

Asset Search Usage

Definition:

Number of asset searches performed.

Purpose:

Measure discovery behavior.

Asset Search Success Rate

Formula:

Successful Searches
/
Total Searches

Purpose:

Evaluate search effectiveness.

Simulation Completion Funnel

Track:

Asset Selected
↓
Simulation Configured
↓
Simulation Submitted
↓
Results Viewed
↓
Explanation Viewed

Purpose:

Identify user drop-off points.

3.7.8 Educational Metrics

Explanation Engagement

Definition:

Percentage of users who read AI explanations after running simulations.

Purpose:

Measure educational value.

Report Usage

Definition:

Number of generated reports viewed.

Purpose:

Measure research engagement.

Learning Retention Indicators

Future metric.

Potential signals:

- Repeat simulations
- Repeated asset comparisons
- Return visits

Purpose:

Estimate educational effectiveness.

3.7.9 Technical Metrics

API Reliability

Track:

- Success rate
- Error rate
- Latency

Purpose:

Monitor platform health.

Data Import Reliability

Track:

- Import success rate
- Failed imports
- Data freshness

Purpose:

Protect historical accuracy.

Database Growth

Track:

- Storage usage
- Record counts
- Query performance

Purpose:

Support scaling decisions.

3.7.10 Executive Dashboard Metrics

Founder dashboard should include:

Metric	Importance
MAU	High
Simulations Created	High
Simulation Success Rate	High
AI Usage Rate	High
User Retention	High
Most Simulated Assets	Medium
API Reliability	High

These metrics provide a high-level view of platform health.

3.7.11 Data Collection Standards

Collected analytics should be:

- Aggregated when possible
- Securely stored
- Auditable
- Useful

The platform should avoid collecting unnecessary personal information.

3.7.12 MVP BI Requirements

Required MVP metrics:

Users
Simulations
AI Usage
Asset Searches
API Reliability

Import Success Rate

Anything beyond this can be added after launch.

3.7.13 Future BI Expansion

Potential future metrics:

Portfolio Usage
Research Workspace Usage
Educational Progress
Feature Adoption
Revenue Metrics
Conversion Metrics

These metrics become relevant as the platform evolves.

3.7.14 Decision-Making Framework

Product decisions should prioritize:

1. User behavior data
2. Product metrics
3. Technical metrics
4. Subjective opinions

The platform should continuously learn from real usage.

3.7.15 Success Criteria

Business Intelligence is successful when the platform can answer:

- Are users returning?
- Are users running simulations?
- Are users reading explanations?
- Which assets are most popular?

- Where are users dropping off?
- Which features create value?

If these questions can be answered confidently, the BI architecture is working.

3.7.16 Summary

Business Intelligence Architecture provides the measurement framework for platform growth and product decision-making.

Approved MVP metrics:

- MAU
- DAU
- Simulation volume
- Simulation success rate
- AI usage rate
- Asset search usage
- API reliability
- Import reliability

The purpose of Business Intelligence is to transform user activity into actionable product insight.

Status: Part 3.7 Complete

Next: Part 3.8 — AI Development Standards

Investment Time Machine

Founder Specification

Part 3.8 — AI Development Standards

Status: Draft

3.8.1 Purpose

AI Development Standards define mandatory implementation rules for all AI coding agents used during development.

These standards apply to:

- Claude Code
- Codex
- OpenAI Coding Agents
- Future AI Development Tools

The objective is to ensure that AI-generated code remains:

- Consistent
- Maintainable
- Secure
- Testable
- Aligned with platform architecture

These standards are mandatory unless explicitly overridden by a documented engineering decision.

3.8.2 Core Principle

AI agents must follow the architecture.

AI agents must not redesign the architecture.

The Founder Specification is the source of truth.

If generated code conflicts with the Founder Specification:

Founder Specification Wins

3.8.3 Backend Standards

Required technologies:

Category	Standard
Language	Python
Framework	FastAPI
ORM	SQLAlchemy 2.0

Validation	Pydantic v2
Migrations	Alembic
Database	PostgreSQL
Cache	Redis

AI agents must not introduce alternative frameworks without explicit approval.

Examples of prohibited substitutions:

Django
Flask
MongoDB
Marshmallow

unless specifically requested.

3.8.4 Database Standards

Primary Keys

All production tables must use:

UUID PRIMARY KEY

Foreign Keys

Database relationships must be enforced.

Application-only relationships are prohibited.

Migrations

Schema changes require Alembic migrations.

Direct schema modifications are prohibited.

Naming

Required:

snake_case
plural table names

Examples:

users
assets
historical_prices

3.8.5 Python Standards

Required:

- Type hints
- Explicit return types
- Small functions
- Clear naming
- Docstrings for public interfaces

Avoid:

single-letter variables
deep nesting
hidden side effects

Readability is preferred over cleverness.

3.8.6 API Standards

All endpoints must:

- Use Pydantic models
- Return structured responses
- Validate inputs
- Handle errors explicitly

Preferred response format:


```
{  
  "success": true,  
  "data": {}  
}
```

Error format:

```
{  
  "success": false,  
  "error": {  
    "code": "ERROR_CODE",  
    "message": "Description"  
  }  
}
```

3.8.7 Simulation Engine Standards

The Simulation Engine is the highest-risk component.

AI-generated code must:

- Preserve determinism
- Preserve reproducibility
- Avoid floating-point errors
- Support testing

Required:

NUMERIC values
known-answer tests
explicit formulas

Prohibited:

hidden calculations
AI-generated financial outputs

3.8.8 Frontend Standards

Required technologies:

Category	Standard
----------	----------

Framework	Next.js
-----------	---------

Language	TypeScript
----------	------------

Styling	Tailwind
---------	----------

Charts	Recharts
--------	----------

Business logic must remain in backend services.

Frontend responsibilities:

- Display data
- Collect inputs
- Render charts
- Manage user interactions

Frontend must not calculate financial metrics.

3.8.9 Security Standards

Generated code must:

- Validate inputs
- Use parameterized queries
- Protect secrets
- Follow least privilege

Prohibited:

hardcoded secrets
plaintext passwords
unsafe SQL

Passwords must use:

Argon2

or approved alternatives.

3.8.10 Testing Standards

Every significant feature must include tests.

Required coverage:

Area	Requirement
Simulation Engine	Mandatory
API Endpoints	Mandatory
Data Validation	Mandatory
Authentication	Mandatory

A feature is not complete without tests.

3.8.11 Documentation Standards

Generated code must remain consistent with documentation.

Required updates when relevant:

- API documentation
- Database documentation
- Architecture documentation
- Migration documentation

Documentation is part of the implementation.

3.8.12 Error Handling Standards

Errors must:

- Be explicit

- Be actionable
- Avoid exposing secrets

Avoid:

Something went wrong.

Prefer:

Historical data unavailable for selected date range.

3.8.13 Dependency Standards

Before adding dependencies, AI agents should evaluate:

1. Is it necessary?
2. Is it maintained?
3. Does it align with architecture?
4. Can existing tools solve the problem?

Prefer fewer dependencies.

3.8.14 Code Review Checklist

Generated code should satisfy:

- Architecture compliant
- Type-safe
- Tested
- Secure
- Documented
- Readable

Code that fails these criteria should be revised.

3.8.15 Prohibited AI Behaviors

AI coding agents must not:

- Change architecture without approval
- Introduce unsupported technologies
- Remove validation
- Bypass testing
- Generate fake financial data
- Store secrets in source code

The Founder Specification always takes precedence.

3.8.16 Future Expansion

Future standards may include:

CI/CD Rules

Infrastructure-as-Code Rules

Performance Standards

Observability Standards

Multi-Agent Workflows

These additions should remain consistent with existing architecture.

3.8.17 Summary

The AI Development Standards provide a common rule set for all current and future coding agents.

Approved standards:

- FastAPI
- SQLAlchemy 2.0
- Pydantic v2
- PostgreSQL
- UUID primary keys
- Alembic migrations
- Type hints
- Mandatory testing
- Documentation-first development

- No AI-generated financial facts

These standards ensure that AI-generated code remains consistent with the architecture and long-term goals of Investment Time Machine.

Status: Part 3.8 Complete

Part 3 Status:

- ✓ Product Requirements Document
- ✓ User Flows
- ✓ Functional Requirements
- ✓ Non-Functional Requirements
- ✓ Financial Analytics Architecture
- ✓ Threat Modeling
- ✓ Business Intelligence Architecture
- ✓ AI Development Standards

Part 3 Complete

Next Recommendation: **Founder Specification Consolidation Pass v1.0** before creating implementation-specific chats (Backend, Frontend, Database, AI, Infrastructure).

Investment Time Machine

Founder Specification

Part 3.9 — Engineering Standards

Status: Draft

3.9.1 Purpose

Engineering Standards define the mandatory rules that govern implementation across the platform.

These standards apply to:

- Human developers
- Claude Code
- Codex
- GPT-based coding agents

- Future contributors

The objective is to ensure:

- Consistency
- Maintainability
- Reliability
- Security
- Architectural alignment

All implementation work must follow these standards unless a documented engineering decision explicitly overrides them.

3.9.2 Engineering Philosophy

The platform follows five engineering principles:

Correctness Over Speed

Accurate systems are more valuable than rapidly built systems.

Simplicity Over Cleverness

Readable solutions are preferred over complex solutions.

Architecture Over Convenience

Implementation must follow approved architecture.

Documentation Over Assumptions

Important decisions should be documented.

Long-Term Maintainability Over Short-Term Optimization

Code should remain understandable years after it is written.

3.9.3 Python Standards

Required:

- Python 3.12+
- Type hints
- Explicit return types
- Clear function names
- Small functions
- Descriptive variable names

Preferred:

`calculate_cagr()`

Avoid:

`calc()`

Code should prioritize readability.

3.9.4 FastAPI Standards

FastAPI is the approved backend framework.

Requirements:

- APIRouter organization
- Dependency injection
- Pydantic request models
- Pydantic response models
- OpenAPI documentation

Business logic must not exist inside route handlers.

Route handlers should:

Validate

↓

Call Service

↓

Return Response

3.9.5 SQLAlchemy Standards

Required:

- SQLAlchemy 2.0 style
- Declarative models
- Explicit relationships
- Typed ORM models

Avoid legacy SQLAlchemy patterns.

Database models should remain focused on persistence rather than business logic.

3.9.6 Alembic Standards

All schema changes require migrations.

Required:

- Migration file
- Migration review
- Version control
- Rollback consideration

Prohibited:

Manual production schema changes

The database schema must evolve through migrations only.

3.9.7 PostgreSQL Standards

Requirements:

- UUID primary keys
- Foreign key constraints
- UTC timestamps
- NUMERIC financial precision
- Indexed query paths

Financial calculations must never use floating-point storage.

Required:

NUMERIC(20,8)

Prohibited:

FLOAT

REAL

DOUBLE PRECISION

for financial values.

3.9.8 API Standards

All APIs must:

- Validate requests
- Return structured responses
- Handle errors explicitly
- Use versioned routes

Example:

/api/v1/assets

/api/v1/simulations

Preferred success response:

```
{  
  "success": true,  
  "data": {}  
}
```

Preferred error response:

```
{  
  "success": false,  
  "error": {
```

```
"code": "ERROR_CODE",  
"message": "Description"  
}  
}
```

3.9.9 Simulation Engine Standards

The Simulation Engine is the highest-risk component.

Requirements:

- Deterministic calculations
- Explicit formulas
- Reproducible outputs
- Known-answer tests

The Simulation Engine must never:

- Depend on AI
- Query live market data
- Generate estimated prices

Financial correctness is mandatory.

3.9.10 Frontend Standards

Approved stack:

- Next.js
- TypeScript
- Tailwind CSS
- Recharts

Frontend responsibilities:

- Collect inputs
- Display outputs
- Render charts
- Manage user interaction

Frontend must not:

- Calculate returns
- Calculate CAGR
- Modify simulation outputs

Financial logic belongs in backend services.

3.9.11 Testing Standards

Every significant feature requires tests.

Required coverage:

Component	Requirement
Simulation Engine	Mandatory
API Endpoints	Mandatory
Authentication	Mandatory
Validation Logic	Mandatory
Data Imports	Mandatory

Bug fixes should include regression tests whenever practical.

3.9.12 Git Standards

Recommended workflow:

main
feature/*

Future expansion:

develop
hotfix/*
release/*

Requirements:

- Descriptive commit messages
- Small pull requests
- Atomic changes

Avoid mixing unrelated work within a single commit.

3.9.13 Documentation Standards

Every significant implementation change should update:

- Architecture documentation
- Database documentation
- API documentation
- Deployment documentation

Documentation is part of the implementation.

A feature is not complete if documentation is outdated.

3.9.14 Security Standards

Generated code must:

- Validate inputs
- Use parameterized queries
- Protect secrets
- Follow least privilege

Prohibited:

- Hardcoded credentials
- Plaintext passwords
- Unsafe SQL execution

Secrets must be provided through environment variables.

3.9.15 Code Review Checklist

Before approval, code should satisfy:

- Architecture compliant
- Tested
- Documented
- Type-safe
- Secure
- Readable

Reviewers should prioritize:

1. Correctness
 2. Security
 3. Maintainability
 4. Performance
 5. Style
-

3.9.16 AI Coding Agent Standards

Claude Code, Codex, and future coding agents must:

- Follow the Founder Specification
- Follow approved technology choices
- Generate tests
- Generate documentation when required
- Preserve architectural boundaries

AI agents must not:

- Redesign architecture
- Introduce unsupported frameworks
- Bypass testing
- Generate fake financial data

When conflicts occur:

Founder Specification Wins

3.9.17 Definition of Done

A task is complete only when:

- Implementation finished
- Tests pass
- Documentation updated
- Errors handled
- Code reviewed
- Deployment validated

Implementation alone does not equal completion.

3.9.18 Summary

Engineering Standards define how Investment Time Machine is built and maintained.

Approved standards:

- Python 3.12+
- FastAPI
- SQLAlchemy 2.0
- PostgreSQL
- Alembic
- UUID primary keys
- Type hints
- Mandatory testing
- Documentation-first development
- Security-first implementation
- Architecture-driven development

These standards serve as the implementation rulebook for all future development.

Status: Part 3.9 Complete

Part 3 Status:

- ✓ Product Requirements Document
- ✓ User Flows
- ✓ Functional Requirements
- ✓ Non-Functional Requirements
- ✓ Financial Analytics Architecture
- ✓ Threat Modeling

- ✓ Business Intelligence Architecture
- ✓ AI Development Standards
- ✓ Engineering Standards

Founder Specification Status:

Part 1 Complete

Part 2 Complete

Part 3 Complete

Founder Specification v1 Ready for Consolidation.